NextPlay · Propuestas

52 pendientes · 13 en curso · 23 hechas · actualizado el 04/11/2026 18:50

Revisión de la lista · 01/11/2026

He repasado las tareas abiertas contra el código, la base de datos local y las APIs reales. Cada una lleva ahora un bloque aparte con cómo está hoy, cómo la haría y formas de profundizar. Aquí va lo que solo se ve mirándolas todas juntas. Los «comprobado» de este documento están probados el 01/11/2026; lo que dice «comprobar» no he podido probarlo. Las cifras de tester_steam son de antes de arreglar el emparejado, que le ha subido la biblioteca de 348 a 361 juegos.

Lo primero

Cambia las claves antes de subir nada más. Las claves de IGDB, IsThereAnyDeal, Steam y GG.deals de tu .env son las mismas que quedaron en el historial de git, y el repositorio es público. Sacar .env de git no las protegió: siguen ahí. Y este documento lo dice, así que conviene cambiarlas antes de subirlo.

1. Sacar .env del repositorio y cambiar las claves

Dos pruebas que dependen de ti. Un NPSSO para medir los trofeos de esta rama y crear la aplicación de Azure, para probar Xbox por primera vez con una cuenta real.

29. Trofeos de PlayStation emparejados por el ID del juego, no por el nombre
5. Importar juegos y horas de Xbox

Lo que cambia al mirarla entera

Había horas que no contaban por cómo se emparejaba. Ya arreglado. En tester_steam, 33 títulos sin emparejar dejaban fuera 689 h, entre ellas las 382,6 h de Rainbow Six Siege. Con el emparejado nuevo salen 22 títulos y 585 h, y quedan 4 títulos por emparejar a mano; lo demás no son juegos. Le habría pasado a cualquiera que importe de PlayStation o Xbox.

72. Emparejar los juegos por nombre sin contar signos: recuperar horas perdidas

Completado está casi vacío. Solo 1 de los 348 juegos de tester_steam está en Completado, porque las importaciones nunca completan; otros 220 están en Jugado sin saber si se terminaron. Los retos de terminar juegos, las estadísticas y la duración de la comunidad se apoyan en esa columna. Un repaso guiado con pistas («llevas 40 h y suele durar 30: ¿lo terminaste?») la llena: cruzando las horas con la duración de IGDB, 121 de esos 220 tendrían ya una respuesta propuesta.

55. Retos del año y control del pendiente
7. Logros uno a uno: cuáles tienes, cuándo los conseguiste y lo raros que son

Las notas, también. 2 de 348 juegos valorados en tester_steam y 7 de 259 en tester. Compatibilidad y recomendaciones no pueden depender solo de las estrellas: la señal fuerte son las horas, relativas a lo que dura cada juego. Y hace falta valorar con menos esfuerzo, en ráfaga o por duelos.

8. La gente que sigues en la ficha y en los perfiles
36. Recomendaciones «Qué jugar después»
20. Primer uso: traer tus juegos nada más crear la cuenta
57. Ranking personal por duelos

Los logros traen fecha, y eso recupera el pasado. Comprobado con tester_steam: 118 de 128 juegos con logros traen fechas, con actividad en cada año desde 2022. 36 tienen logros de 2025 y 14 de esos se perderían mirando solo la última partida. Es la única fuente retroactiva de cuándo se ha jugado, y hoy steam.player_achievements la cuenta y la tira. Por eso le daría más prioridad.

7. Logros uno a uno: cuáles tienes, cuándo los conseguiste y lo raros que son
6. Diario de partidas que se escribe solo
3. Tier lists que se rellenan solas con tu biblioteca: «Jugados en 2026», tu pendiente, tus platinos…
37. Estadísticas del perfil y resumen anual

Lo que no se guarda hoy no se recupera. Pasa con los precios de Xbox y Nintendo, que se sobrescriben cada noche y no tienen historial, con las entradas y salidas de Game Pass y con las sesiones de juego. Mientras NextPlay esté en local solo se pierde tu propio historial, y por eso la tarea de cada noche puede esperar al despliegue; ese día conviene que sea de lo primero. Guardar esos datos cuesta muy poco y dentro de un año no se podrán conseguir.

53. Sincronización automática de la biblioteca
45. Precios de Nintendo eShop y PlayStation Store
39. Saber qué juegos están en Game Pass

Varias tareas no necesitan usuarios, aunque lo parezca. IGDB da nota de usuarios en 2148 de 2206 juegos y duración en 1523, además de datos de popularidad de Steam y Twitch. Steam da porcentajes de logros sin clave. Las joyas ocultas, el precio por hora, la tendencia del catálogo y la tasa de finalización funcionan desde el primer día.

60. Joyas ocultas y juegos en los que la comunidad discrepa
61. Cuándo se abandona, si mejora con las horas y cuánto sale la hora
13. Catálogo con filtros y orden al nivel de Backloggd (y más)
30. ¿Cuánta gente lo termina? Tasa de finalización con los logros de Steam

Piezas que sirven a varias tareas

Una pasada a IGDB. Un comando fill_game_fields, con 5 peticiones para todo el catálogo, trae de una vez tipo, estado, modos, temas, parecidos, compañías y nota de usuarios. Mejor una sola vez que una por propuesta.

25. Quitar DLC y actualizaciones de las búsquedas en IGDB
12. Ficha del juego más completa
13. Catálogo con filtros y orden al nivel de Backloggd (y más)
33. ¿A qué jugamos? Juegos en común con tus amigos
36. Recomendaciones «Qué jugar después»
60. Joyas ocultas y juegos en los que la comunidad discrepa

Avisos en un solo sitio. Antes de que ocho propuestas añadan avisos nuevos y de que el bloqueo tenga que tocarlos todos.

81. Un solo sitio para crear los avisos, con preferencias por tipo
74. Retos: crea los tuyos y únete a los de otros
31. Bloquear, silenciar y quitar seguidores

Horas totales en la base de datos. El primer paso, total_minutes, es pequeño y deja de obligar a cargar bibliotecas enteras para ordenarlas. El resto abre la puerta a más plataformas.

19. Horas por plataforma sin una columna por tienda
18. Biblioteca del perfil por tandas en vez de entera
49. Importar Epic, GOG, EA y Battle.net con Playnite, y CSV de otros trackers
34. Importar Nintendo Switch con capturas de la app Nintendo Store

Imágenes para compartir con Pillow. Una sola función con varias plantillas, en PNG, porque las redes no aceptan SVG. Sirve para perfiles, tier lists, retos conseguidos y el resumen anual.

- 40. Vistas previas al compartir y tarjeta para incrustar
- 37. Estadísticas del perfil y resumen anual
- 3. Tier lists que se rellenan solas con tu biblioteca: «Jugados en 2026», tu pendiente, tus platinos…

Una sola regla de quién ve qué. Las listas repiten user__profile__is_public en vez de usar Profile.visible_filter. Conviene unificarlo antes de añadir bloqueos y el nivel «solo seguidores».

- 31. Bloquear, silenciar y quitar seguidores
- 32. Perfil solo para seguidores, con solicitudes

En qué orden lo haría

1. Esta semana: casi todo es cosa tuya. Seguridad y las dos pruebas con cuentas reales.

- 1. Sacar .env del repositorio y cambiar las claves
- 29. Trofeos de PlayStation emparejados por el ID del juego, no por el nombre
- 5. Importar juegos y horas de Xbox

2. Remates rápidos. Poco trabajo, y dejan la lista y la web más limpias.

- 14. Que ningún juego se quede sin precio: pedirlo al importar o al abrir la ficha, y el de Steam a Steam
- 15. Arreglos pequeños que han salido al medir: mejor precio, redirección y consultas de más
- 65. Traducir al español los comentarios antiguos en inglés
- 23. Listas colaborativas: invitar solo a quien te sigue y avisar al aceptar
- 51. Detalles sueltos: recuadros de los formularios, el corazón de las listas y pestañas cortadas
- 22. Formularios que avisan mientras trabajan
- 21. Inicio en el móvil: la actividad antes que tu resumen
- 20. Primer uso: traer tus juegos nada más crear la cuenta
- 24. Avisar al salir de Editar perfil con cambios sin guardar
- 54. Páginas de error propias: no encontrada, fallo del servidor y formulario caducado
- 10. Rediseño de toda la web con aspecto de red social
- 16. Que las pantallas de cada función se comprueben solas
- 82. Partir las vistas más grandes por pantalla

3. Retos, listas y perfil: lo que pediste primero. Es el núcleo de lo que quieres construir, y los logros le dan los datos que le faltan.

- 81. Un solo sitio para crear los avisos, con preferencias por tipo
- 74. Retos: crea los tuyos y únete a los de otros
- 7. Logros uno a uno: cuáles tienes, cuándo los conseguiste y lo raros que son
- 3. Tier lists que se rellenan solas con tu biblioteca: «Jugados en 2026», tu pendiente, tus platinos…
- 4. Tier lists de la comunidad, con su propia pestaña como Retos
- 6. Diario de partidas que se escribe solo
- 55. Retos del año y control del pendiente
- 8. La gente que sigues en la ficha y en los perfiles
- 11. Publicaciones con imágenes asociadas a un juego
- 9. Cabecera y color automáticos según lo que juegas
- 17. Intercambio de pendientes: elige un juego del pendiente de un amigo

4. Una pasada a IGDB y el catálogo. Cada una es corta si la consulta de IGDB se hace una sola vez.

- 25. Quitar DLC y actualizaciones de las búsquedas en IGDB
- 12. Ficha del juego más completa
- 35. Duración estimada de los juegos
- 19. Horas por plataforma sin una columna por tienda
- 18. Biblioteca del perfil por tandas en vez de entera

13. Catálogo con filtros y orden al nivel de Backloggd (y más)
26. Contadores y reparto de valoraciones en la ficha
28. Próximos lanzamientos: fechas por plataforma y aviso cuando sale
60. Joyas ocultas y juegos en los que la comunidad discrepa
27. Ediciones agrupadas bajo su juego (GOTY, Complete, Definitive…)

5. Lo que se apoya en todo lo anterior. Salen mucho mejor con logros, duración y más datos del juego ya guardados.

37. Estadísticas del perfil y resumen anual
36. Recomendaciones «Qué jugar después»
30. ¿Cuánta gente lo termina? Tasa de finalización con los logros de Steam
33. ¿A qué jugamos? Juegos en común con tus amigos
40. Vistas previas al compartir y tarjeta para incrustar
56. Etiquetas de la comunidad y «¿Qué juego esta noche?»
57. Ranking personal por duelos
63. ¿Qué edición compro? Precios de cada edición y remaster
61. Cuándo se abandona, si mejora con las horas y cuánto sale la hora
38. Recuerdos: «un día como hoy» con tus fechas reales
50. Preguntas sobre un juego que no destripan nada

6. Precios y tiendas, cuando apetezca. Independientes entre sí. «¿Cuándo bajará?» tiene más gracia si llega antes de las rebajas de invierno.

46. ¿Cuándo bajará? Próxima rebaja prevista en el veredicto
39. Saber qué juegos están en Game Pass
42. Juegos gratis para quedártelos
43. Steam Deck, parches y «jugando ahora»
45. Precios de Nintendo eShop y PlayStation Store
58. Directos de Twitch en la ficha del juego
48. Cuánto has gastado y ahorrado: compras detectadas al sincronizar

7. Más plataformas. Mejor después de las horas por plataforma, y tras probar con datos reales que la lectura funciona.

47. Tareas en segundo plano para lo que tarda
49. Importar Epic, GOG, EA y Battle.net con Playnite, y CSV de otros trackers
34. Importar Nintendo Switch con capturas de la app Nintendo Store
62. Colección física con escáner de código de barras

8. Antes de abrir al público (aparcado). Tu decisión del 21/10 sigue en pie: primero el producto. Esto es lo que haría falta el día que quieras abrirlo, incluida la tarea de cada noche, que programarás al desplegar.

2. Borrar la cuenta, descargar mis datos y privacidad (RGPD)
31. Bloquear, silenciar y quitar seguidores
32. Perfil solo para seguidores, con solicitudes
41. Entrar con Steam o con cuenta Microsoft
44. Encontrar a tus amigos de Steam y Xbox en NextPlay
64. Miniatura ya encuadrada de la foto de perfil para las listas
59. Buscar grupo para jugar
52. Publicar NextPlay y abrir una beta antes de las rebajas de invierno
53. Sincronización automática de la biblioteca

Ajustes que propongo a la lista

No los he aplicado: es decisión tuya.

- Subir «Logros uno a uno» a prioridad alta, por lo que desbloquea.
- Cerrar «Rediseño de toda la web» cuando se decida lo de las tres columnas: lo que queda ya tiene sus propias propuestas.
- Dividir «Precios de Nintendo eShop y PlayStation Store»: Nintendo, hecha; PlayStation, aparte, pendiente de la clave de PlatPrices.
- Renombrar «Retos del año y control del pendiente» a «Valor del pendiente y repaso de la biblioteca» y subirla a media: los retos ya cubren lo demás, y el repaso es lo que llena Completado.
- Quitar «antes de las rebajas de invierno» del título de «Publicar NextPlay» mientras siga aparcada.

Índice

#	Propuesta	Prioridad	Tipo	Estado
1	Sacar .env del repositorio y cambiar las claves	alta	Seguridad	en curso
2	Borrar la cuenta, descargar mis datos y privacidad (RGPD)	alta	Legal	pendiente
3	Tier lists que se rellenan solas con tu biblioteca: «Jugados en 2026», tu pendiente, tus platinos…	alta	Funcionalidad	en curso
4	Tier lists de la comunidad, con su propia pestaña como Retos	alta	Funcionalidad	en curso
5	Importar juegos y horas de Xbox	alta	Funcionalidad	en curso
6	Diario de partidas que se escribe solo	alta	Funcionalidad	en curso
7	Logros uno a uno: cuáles tienes, cuándo los conseguiste y lo raros que son	alta	Funcionalidad	en curso
8	La gente que sigues en la ficha y en los perfiles	alta	Funcionalidad	pendiente
9	Cabecera y color automáticos según lo que juegas	alta	Funcionalidad	pendiente
10	Rediseño de toda la web con aspecto de red social	alta	Diseño	en curso
11	Publicaciones con imágenes asociadas a un juego	alta	Funcionalidad	en curso
12	Ficha del juego más completa	alta	Funcionalidad	pendiente
13	Catálogo con filtros y orden al nivel de Backloggd (y más)	alta	Funcionalidad	pendiente
14	Que ningún juego se quede sin precio: pedirlo al importar o al abrir la ficha, y el de Steam a Steam	media	Funcionalidad	en curso
15	Arreglos pequeños que han salido al medir: mejor precio, redirección y consultas de más	media	Funcionalidad	pendiente
16	Que las pantallas de cada función se comprueben solas	media	Documentación	pendiente
17	Intercambio de pendientes: elige un juego del pendiente de un amigo	media	Funcionalidad	pendiente
18	Biblioteca del perfil por tandas en vez de entera	media	Diseño	pendiente
19	Horas por plataforma sin una columna por tienda	media	Infraestructura	pendiente
20	Primer uso: traer tus juegos nada más crear la cuenta	media	Diseño	pendiente
21	Inicio en el móvil: la actividad antes que tu resumen	media	Diseño	pendiente
22	Formularios que avisan mientras trabajan	media	Diseño	pendiente
23	Listas colaborativas: invitar solo a quien te sigue y avisar al aceptar	media	Funcionalidad	pendiente
24	Avisar al salir de Editar perfil con cambios sin guardar	media	Funcionalidad	pendiente

#	Propuesta	Prioridad	Tipo	Estado
25	Quitar DLC y actualizaciones de las búsquedas en IGDB	media	Funcionalidad	pendiente
26	Contadores y reparto de valoraciones en la ficha	media	Funcionalidad	pendiente
27	Ediciones agrupadas bajo su juego (GOTY, Complete, Definitive…)	media	Funcionalidad	pendiente
28	Próximos lanzamientos: fechas por plataforma y aviso cuando sale	media	Funcionalidad	pendiente
29	Trofeos de PlayStation emparejados por el ID del juego, no por el nombre	media	Funcionalidad	en curso
30	¿Cuánta gente lo termina? Tasa de finalización con los logros de Steam	media	Funcionalidad	pendiente
31	Bloquear, silenciar y quitar seguidores	media	Funcionalidad	pendiente
32	Perfil solo para seguidores, con solicitudes	media	Funcionalidad	pendiente
33	¿A qué jugamos? Juegos en común con tus amigos	media	Funcionalidad	pendiente
34	Importar Nintendo Switch con capturas de la app Nintendo Store	media	Funcionalidad	pendiente
35	Duración estimada de los juegos	media	Funcionalidad	pendiente
36	Recomendaciones «Qué jugar después»	media	Funcionalidad	pendiente
37	Estadísticas del perfil y resumen anual	media	Funcionalidad	pendiente
38	Recuerdos: «un día como hoy» con tus fechas reales	media	Funcionalidad	pendiente
39	Saber qué juegos están en Game Pass	media	Funcionalidad	pendiente
40	Vistas previas al compartir y tarjeta para incrustar	media	Funcionalidad	pendiente
41	Entrar con Steam o con cuenta Microsoft	media	Funcionalidad	pendiente
42	Juegos gratis para quedártelos	media	Funcionalidad	pendiente
43	Steam Deck, parches y «jugando ahora»	media	Funcionalidad	pendiente
44	Encontrar a tus amigos de Steam y Xbox en NextPlay	media	Funcionalidad	pendiente
45	Precios de Nintendo eShop y PlayStation Store	media	Funcionalidad	en curso
46	¿Cuándo bajará? Próxima rebaja prevista en el veredicto	media	Funcionalidad	pendiente
47	Tareas en segundo plano para lo que tarda	media	Infraestructura	pendiente
48	Cuánto has gastado y ahorrado: compras detectadas al sincronizar	media	Funcionalidad	pendiente
49	Importar Epic, GOG, EA y Battle.net con Playnite, y CSV de otros trackers	media	Funcionalidad	pendiente
50	Preguntas sobre un juego que no destripan nada	baja	Funcionalidad	pendiente

#	Propuesta	Prioridad	Tipo	Estado
51	Detalles sueltos: recuadros de los formularios, el corazón de las listas y pestañas cortadas	baja	Diseño	en curso
52	Publicar NextPlay y abrir una beta antes de las rebajas de invierno	baja	Lanzamiento	pendiente
53	Sincronización automática de la biblioteca	baja	Infraestructura	en curso
54	Páginas de error propias: no encontrada, fallo del servidor y formulario caducado	baja	Diseño	pendiente
55	Retos del año y control del pendiente	baja	Funcionalidad	pendiente
56	Etiquetas de la comunidad y «¿Qué juego esta noche?»	baja	Funcionalidad	pendiente
57	Ranking personal por duelos	baja	Funcionalidad	pendiente
58	Directos de Twitch en la ficha del juego	baja	Funcionalidad	pendiente
59	Buscar grupo para jugar	baja	Funcionalidad	pendiente
60	Joyas ocultas y juegos en los que la comunidad discrepa	baja	Funcionalidad	pendiente
61	Cuándo se abandona, si mejora con las horas y cuánto sale la hora	baja	Funcionalidad	pendiente
62	Colección física con escáner de código de barras	baja	Funcionalidad	pendiente
63	¿Qué edición compro? Precios de cada edición y remaster	baja	Funcionalidad	pendiente
64	Miniatura ya encuadrada de la foto de perfil para las listas	baja	Rendimiento	pendiente
65	Traducir al español los comentarios antiguos en inglés	baja	Documentación	pendiente
66	Perfil muy personalizable: foto, cabecera, fondo, colores y escaparate	alta	Funcionalidad	hecha
67	Reseñas completas: medias estrellas, una por partida y página propia	alta	Funcionalidad	hecha
68	Reseñas: lo mejor y lo peor, datos de la partida rellenos solos e imágenes	alta	Funcionalidad	hecha
69	Me gusta y comentarios en las reseñas	alta	Funcionalidad	hecha
70	Listas personalizadas de juegos	alta	Funcionalidad	hecha
71	Listas: colocar las carátulas arrastrándolas	alta	Funcionalidad	hecha
72	Emparejar los juegos por nombre sin contar signos: recuperar horas perdidas	alta	Funcionalidad	hecha
73	Horas de cada plataforma, a la vista	media	Funcionalidad	hecha
74	Retos: crea los tuyos y únete a los de otros	alta	Funcionalidad	hecha
75	Revisar los juegos que no se emparejaron al importar	—	Funcionalidad	hecha

#	Propuesta	Prioridad	Tipo	Estado
76	Lista de deseos con alertas de precio	—	Funcionalidad	hecha
77	Historial de precios y «¿compro ahora o espero?»	—	Funcionalidad	hecha
78	Seguir a otros jugadores y feed de actividad	alta	Funcionalidad	hecha
79	Rediseño de la ficha: cabecera con arte y carrusel de imágenes y vídeos	alta	Diseño	hecha
80	Fichas en castellano: doblaje, subtítulos y sinopsis en español	alta	Funcionalidad	hecha
81	Un solo sitio para crear los avisos, con preferencias por tipo	media	Infraestructura	hecha
82	Partir las vistas más grandes por pantalla	media	Infraestructura	hecha
83	Reseñas con horas verificadas y spoilers según tu progreso	media	Funcionalidad	hecha
84	Importar logros y trofeos	media	Funcionalidad	hecha
85	Importar la lista de deseos de Steam	media	Funcionalidad	hecha
86	Vincular otras plataformas sin importar (PSN, Switch, Epic, GOG, Battle.net)	—	Funcionalidad	hecha
87	Importar juegos y horas de PlayStation (experimental)	—	Funcionalidad	hecha
88	Varias cuentas de Steam por usuario	—	Funcionalidad	hecha

1. Sacar .env del repositorio y cambiar las claves

PRIORIDAD ALTA · EN CURSO · Seguridad · propuesta el 17/10/2026

Qué es

Dejar de versionar el archivo .env para que las claves (IGDB, IsThereAnyDeal, Steam...) no acaben en GitHub.

Por qué

.gitignore incluye .env, pero el archivo se subió en el commit «Create .env» y git lo sigue versionando. Cualquier clave que se ponga ahí se sube con el siguiente commit.

Pasos previstos

1. git rm --cached .env para que git deje de versionarlo sin borrarlo del ordenador.
2. Commit del cambio; a partir de ahí .gitignore lo ignora de verdad.
3. Cambiar en cada servicio las claves que ya se subieron (están en el historial de git aunque se quite el archivo).
4. Mantener .env.example con los nombres de las variables y sin valores.

Notas

25/10/2026: al revisar el proyecto, el repositorio de GitHub es público y el .env de main lleva claves con pinta de reales: el Client ID y el secreto de Twitch (IGDB) y las claves de IsThereAnyDeal, GG.deals y Steam (la de Steam entró con «Update .env», hecho desde la web de GitHub). Hay que cambiar el secreto de Twitch y las claves de IsThereAnyDeal y Steam; la de GG.deals no la usa el código, así que basta con revocarla y quitarla del .env. .claude/launch.json también está versionado y es de cada ordenador: sacarlo a la vez y añadirlo a .gitignore. 25/10/2026: hecho en la rama sacar-env-del-repositorio (PR abierto, sin fusionar): git deja de versionar .env y .claude/launch.json (que pasa a .gitignore) sin borrarlos del ordenador. Falta que el usuario cambie el secreto de

Twitch y las claves de IsThereAnyDeal y Steam y revoque la de GG.deals; entonces pasa a «hecha». No se reescribe el historial: con las claves cambiadas, las que se quedan en él ya no sirven.

REVISIÓN DEL 01/11/2026

Cómo está hoy

La parte de git está hecha y fusionada: .env ya no se versiona. La parte que importa, no: IGDB_CLIENT_ID, IGDB_CLIENT_SECRET, ITAD_API_KEY, STEAM_API_KEY y GGDEALS_API_KEY del .env de este ordenador son idénticas a las de los commits c0d413f y 7915833, y el repositorio es público. Cualquiera puede usarlas. DJANGO_SECRET_KEY nunca se subió. Lo he comparado sin enseñar ningún valor.

Cómo lo haría

1. Tú, en cada servicio, porque son tus cuentas: en la consola de Twitch, un secreto nuevo para la aplicación (el Client ID se queda); en IsThereAnyDeal y en Steam, una clave nueva que anule la anterior; en GG.deals, revocarla y borrar GGDEALS_API_KEY del .env, porque el código no la usa (comprobado: 0 referencias).
2. Pegar las nuevas en el .env, reiniciar el servidor y probar cada una con una llamada real: import_igdb --search con un juego, update_prices --game con otro y sincronizar Steam desde el perfil.
3. Repetir la comparación de hoy (los valores de .env contra esos dos commits, sin mostrarlos) para confirmar que ya no coincide ninguno, y pasar la propuesta a «hecha».
4. No reescribiría el historial: con las claves cambiadas, lo que queda en él no sirve, y reescribirlo rompe todas las ramas y los PR.

Para profundizar

- Que no vuelva a pasar: «Secret scanning» y «Push protection» en GitHub, gratis en repositorios públicos (rechazan el push si lleva una clave conocida), y gitleaks como hook de pre-commit para lo que se les escape.
- Una comprobación propia de Django (system check) que avise si .env aparece en git ls-files o si falta una clave que el código necesita: sale al arrancar el servidor, sin depender de acordarse.
- Un juego de claves por entorno, para poder cortar las de desarrollo sin tocar producción cuando se publique.
- Revisar qué más es público: este PDF y propuestas.json están en el repositorio y cuentan qué falta por asegurar. Si el código no necesita ser público, hacerlo privado es lo más sencillo; no arregla lo ya copiado, por eso primero van las claves.

---

2. Borrar la cuenta, descargar mis datos y privacidad (RGPD)

PRIORIDAD ALTA · PENDIENTE · Legal · propuesta el 17/10/2026

Qué es

Botones para borrar la cuenta y para descargar todos tus datos, más una página de privacidad que explique qué se guarda de cada plataforma.

Por qué

Ahora no hay forma de borrar la cuenta ni de exportar los datos, y el RGPD obliga a ofrecer las dos cosas antes de abrir la web al público en España.

Pasos previstos

1. «Borrar mi cuenta» en Editar perfil, con confirmación de contraseña; borra biblioteca, reseñas y cuentas vinculadas.
2. «Descargar mis datos» en JSON o CSV: perfil, biblioteca, valoraciones, reseñas y cuentas vinculadas.

3. Página de privacidad: qué se guarda (IDs de Steam, PSN y Xbox; el NPSSO no se guarda), a qué servicios se llama (IGDB, IsThereAnyDeal, Steam, Sony, Microsoft) y cómo pedir el borrado.
4. Enlaces en el pie de página.

Notas

Si solo se usan las cookies de sesión y CSRF no hace falta banner de cookies; si algún día se añaden analíticas, sí.

REVISIÓN DEL 01/11/2026

Cómo está hoy

No hay ninguna de las tres cosas (el pie solo cita a IGDB). La base ayuda: casi todo cuelga del usuario con CASCADE y las señales ya borran del disco las imágenes del perfil, de las reseñas y de las publicaciones. El problema está en lo compartido: GameList.user y Challenge.creator también son CASCADE, así que borrar a alguien se lleva listas con colaboradores y retos con más participantes.

Cómo lo haría

1. accounts/privacy.py con export_user(user) y delete_user(user). Un test recorre apps.get_models() y falla si aparece un modelo nuevo con relación al usuario sin decidir qué pasa con él: así la lista no se queda vieja.
2. Borrar: en Editar perfil, pidiendo la contraseña y que escriba su nombre de usuario. Antes de borrar, las listas con colaboradores pasan al primero que aceptó y los retos con más gente se quedan sin creador (SET_NULL), para no quitarles nada a los demás.
3. Descargar: un ZIP con un JSON por tipo de dato (perfil, biblioteca, títulos importados, sesiones, reseñas, listas, retos, publicaciones, seguidos) y las imágenes que ha subido. Con una biblioteca como la de tester_steam (348 juegos) se genera dentro de la petición sin problema.
4. Página /privacidad/ con una tabla: qué se guarda y de dónde sale (SteamID, ID de PSN, XUID, horas, logros, correo), qué no se guarda (NPSSO, tokens de Microsoft), a quién se llama (IGDB y Twitch, IsThereAnyDeal, Steam, Sony, Microsoft, Nintendo y YouTube solo al pulsar) y cómo reclamar ante la AEPD. Enlaces en el pie y en el registro.
5. Revisar lo que se carga de fuera en cada página: htmx sale de unpkg y SortableJS de jsDelivr, y eso les da la IP de cada visitante. Servir los dos desde static y citar en la página los CDN de imágenes. Sin analíticas sigue sin hacer falta banner de cookies.

Para profundizar

- Periodo de gracia: marcar la cuenta, ocultarla al momento y borrarla de verdad a los 30 días desde nightly; entrar antes la recupera. Protege de borrados por impulso y de cuentas robadas.
- Olvidar por partes: al desvincular una plataforma hoy se quedan sus juegos y sus horas. Ofrecer «borrar también lo importado de esta cuenta» y apagar el diario sin tener que irse.
- Exportar en un CSV que otros trackers entiendan: la portabilidad es un argumento a favor (Backloggd aún no la tiene) y sirve de copia personal.
- Edad mínima en el registro: en España son 14 años para consentir el tratamiento (LOPDGDD, art. 7), con una reforma en marcha para subirla a 16; comprobarlo antes de publicar.

3. Tier lists que se rellenan solas con tu biblioteca: «Jugados en 2026», tu pendiente, tus platinos…

PRIORIDAD ALTA · EN CURSO · Funcionalidad · propuesta el 29/10/2026

Qué es

Al crear una tier list, poder empezar desde una plantilla que mete sola los juegos de tu biblioteca en «Sin clasificar», para que solo haya que arrastrarlos: jugados este año, terminados este año, los que más horas llevan, tus platinos y 100 %, lo jugado en cada plataforma o tu pendiente. Y que las tier lists tengan su propia sección, aparte de las listas.

Por qué

Idea del usuario (29/10/2026). Las tier lists y el tablero para arrastrar ya existen (GameList en modo tiers, la fila «Sin clasificar», SortableJS), pero los juegos hay que buscarlos y añadirlos uno a uno. NextPlay ya sabe qué has jugado y cuándo gracias a las importaciones: con la cuenta de prueba tester_steam, «Jugados en 2026» saldría con 50 juegos sin escribir ninguno.

Pasos previstos

1. En «Nueva tier list», un bloque «Empieza desde tu biblioteca» con las plantillas y cuántos juegos metería cada una; al elegir una se crea la tier list con su título («Jugados en 2026») y sus juegos sin clasificar, ordenados por horas, y se abre el tablero.
2. Sección propia para las tier lists (idea del usuario, 02/11/2026: «es algo más grande»): su página «Nueva tier list», con las plantillas arriba y debajo el formulario para empezarla en blanco; su pestaña en el perfil y su sección en el escaparate, con un tablero en miniatura en cada tarjeta; y «Mis tier lists» en el menú. «Nueva lista» crea las normales y las numeradas. Por dentro siguen siendo listas (GameList en modo tiers), con sus «me gusta», comentarios y colaboradores, y una lista puede cambiar de tipo en su editor.
3. Plantillas: jugados este año (última partida de Steam, PlayStation o Xbox, completed_at o una reseña terminada este año), terminados este año, tus 30 más jugados por horas, tus platinos y 100 % (accounts.achievements), lo jugado en cada plataforma y tu pendiente.
4. Niveles propios de cada plantilla: S, A, B, C y D para lo jugado; «Ya mismo», «Pronto», «Algún día» y «Nunca» para el pendiente, que así sirve para decidir qué jugar después.
5. Si los juegos tienen nota, ofrecer colocarlos de entrada según ella (5 estrellas en S, 4 en A…) para que solo haya que retocar.
6. Lista viva: la del año en curso sigue metiendo en «Sin clasificar» los juegos nuevos que juegas al sincronizar, con un aviso («3 juegos nuevos por colocar»); el 1 de enero se cierra y empieza la del año siguiente.
7. En el perfil, la serie de tus años juntos («Tus años en tier lists»).

Notas

Los años pasados no pueden ser exactos: Steam solo da la última partida y el historial de sesiones (PlaySession) empieza el 25/10/2026, así que «Jugados en 2025» se perdería los juegos de 2025 que se han vuelto a jugar en 2026 (PlayStation sí da la primera partida). Mejor ofrecer solo el año en curso y, desde 2027, los anteriores con el historial. Solo 2 de los 348 juegos de tester_steam tienen nota, así que colocar por nota es un extra: lo importante es el orden por horas. Encaja con «Estadísticas del perfil y resumen anual» y con la tarjeta para compartir de «Vistas previas al compartir y tarjeta para incrustar». 02/11/2026: hechos en la rama tierlists-biblioteca los cinco primeros pasos. Las plantillas están en lists/presets.py y las crea lists.views.from_library; colocar por nota va de serie, sin casilla, porque solo mueve los juegos que la tienen. Faltan la lista viva y la serie de tus años.

REVISIÓN DEL 02/11/2026

Cómo está hoy

Primera fase hecha en la rama tierlists-biblioteca. Hay siete plantillas en lists/presets.py; con tester_steam, «Jugados en 2026» sale con 51 juegos, «Tus platinos y 100 %» con 10 y el pendiente con 119, y la de Xbox no sale porque no tiene horas ahí. Cada plantilla crea la tier list privada, ordenada por horas y con los juegos con nota ya en su nivel. Las tier lists tienen ya su sección: la página «Nueva tier list», su pestaña y su sección del escaparate en el perfil, con un tablero en miniatura en cada tarjeta (lists.views.mark_boards, dos consultas para todas), y «Mis tier lists» en el menú.

Cómo lo haría

1. Lista viva: al crearla desde una plantilla, guardar en GameList.source la plantilla y el año (JSON); update_library mete en «Sin clasificar» lo que la plantilla elegiría ahora y aún no está, con un aviso («3 juegos nuevos por colocar»), y nightly la cierra el 1 de enero y ofrece la del año nuevo.
2. «Tus años en tier lists»: en la pestaña Tier lists del perfil, las de las plantillas por año juntas, de la más reciente a la más antigua.
3. Los años pasados, con las fechas de los logros en cuanto «Logros uno a uno» las guarde (ver abajo).

Para profundizar

- Una entrada «Tier lists» en el menú de arriba, como la de «Retos», cuando haya algo que explorar además de las tuyas: la tier list de la comunidad por plantilla y las de la gente que sigues. Ahora va en «Tier lists de la comunidad, con su propia pestaña como Retos».
- Años pasados con los logros: comprobado con tester_steam, 118 de sus 128 juegos con logros traen fechas, 36 tienen logros de 2025 y 14 de esos se perderían mirando solo la última partida, porque se volvieron a jugar en 2026. Con «Logros uno a uno», «Jugados en 2025» se puede ofrecer ya, diciendo «según tus logros».
- «Haz tu versión»: en la tier list de otra persona, un botón copia sus juegos a una tuya sin clasificar y, al terminar, enseña en qué discrepáis («S para ti, C para Ana»). Da conversación y se comparte sola. Ahora va en «Tier lists de la comunidad, con su propia pestaña como Retos».
- Tier list de la comunidad por plantilla: la mediana del nivel de cada juego entre todos los que han hecho «Jugados en 2026», como resumen del año sin pedir nada nuevo a nadie. Ahora va en «Tier lists de la comunidad, con su propia pestaña como Retos».
- Que decida algo: la fila «Ya mismo» de la plantilla del pendiente alimenta el bloque «Elige de tu pendiente» de las recomendaciones, o se convierte en reto con el «Convertir en reto» que ya existe.
- Listas inteligentes: además de las plantillas, una lista definida por una regla («mis RPG de más de 50 h», «pendiente en castellano de menos de 10 h») que se rellena sola cada vez que se abre, con los mismos filtros de los retos (_game_filters) y del catálogo.

---

4. Tier lists de la comunidad, con su propia pestaña como Retos

PRIORIDAD ALTA · EN CURSO · Funcionalidad · propuesta el 03/11/2026

Qué es

Una pestaña «Tier lists» en el menú de arriba, como la de Retos, y tier lists que se dejan abiertas para que las haga la comunidad. Quien la abre pone los juegos y los niveles, y puede no colocar ninguno. Cada uno hace su versión, y la página enseña la tier list de la comunidad, que junta todas, y en qué te apartas tú de ella.

Por qué

Idea del usuario (03/11/2026): apartar todavía más las tier lists, con una pestaña propia como la de los retos, y poder dejarlas para que las haga la comunidad. Ya estaba apuntado como forma de profundizar en la propuesta de las tier lists que se rellenan solas con tu biblioteca: la entrada del menú, «Haz tu

versión» y la tier list de la comunidad. La pestaña sola aporta poco, porque hoy hay 3 tier lists de 2 personas y ya salen en los perfiles y en el feed. Lo que la justifica es tener algo que explorar y en lo que participar, como pasa con los retos.

Pasos previstos

1. Pestaña «Tier lists» (tier-lists/), como la de Retos: las tuyas, las de la gente que sigues, las abiertas con más versiones y las últimas que se han abierto, con un botón «Crear una tier list» que lleva a la página que ya existe. En el ordenador va en el menú de arriba, entre Retos y Deseos. En el móvil ese menú no se ve y la barra de abajo ya tiene cinco botones, así que va en el menú de la cuenta, junto a «Mis tier lists» y Retos.
2. Abrir una tier list a la comunidad: una casilla en «Nueva tier list» y en su editor, solo si es pública. Quien la abre elige los juegos y los niveles, y puede dejarlos todos en «Sin clasificar»: así la deja para que la haga la comunidad.
3. «Haz tu versión»: un botón en la página de una tier list abierta crea una tuya con los mismos juegos y niveles en «Sin clasificar» y abre el tablero, como hacen las plantillas de la biblioteca. Es una tier list normal y tuya, con sus «me gusta» y comentarios, y sale en tu perfil y en el feed («hizo su versión de…»).
4. La tier list de la comunidad: en la página de la abierta, un tablero que junta todas las versiones y pone cada juego en el nivel mediano de quienes lo han colocado. Al tocar un juego se ve el reparto («S 40 %, A 35 %…») y cuánta gente lo ha colocado. Hasta que haya tres versiones, sale «Faltan 2 versiones para ver la de la comunidad».
5. En qué te apartas: al terminar la tuya, los juegos en los que más te alejas de la comunidad («S para ti, C para la comunidad») y los más polémicos, en los que nadie se pone de acuerdo.
6. Aviso a quien la abrió cuando alguien hace su versión, agrupados («Ana y 3 más han hecho su versión de…»).

Notas

La comunidad no arrastra juegos en un mismo tablero: cada uno hace su versión y la de la comunidad se calcula. Así nadie puede estropearla, no hay peleas por dónde va un juego y tu versión sigue siendo tuya. Para editar un mismo tablero entre unos pocos ya están las listas colaborativas. Por dentro siguen siendo listas (GameList en modo tiers): se aparta la navegación, no el modelo, para no repetir los «me gusta», los comentarios, los colaboradores, las denuncias y el feed. Recoge también, para las tier lists abiertas, la «Lista por votos» de la propuesta de las listas colaborativas. 03/11/2026: hecha la fase 1 (pasos 1 a 4) en la rama tierlists-comunidad, fusionada con main en el PR #33 (04/11/2026): GameList.is_open y GameList.parent, con su migración (0004); «Haz tu versión» (lists.views.make_version, una por persona); la tier list de la comunidad en la página de la abierta (lists/community.py: el nivel del medio de los votos, desde 3 personas, solo con las versiones públicas); los niveles fijos en cuanto hay versiones; los juegos que se añaden después, también en las versiones; «hizo su versión de…» en el feed, y la pestaña «Tier lists» (lists/views/tier_lists.py) en el menú. Faltan «En qué te apartas», los polémicos y el aviso a quien la abrió (pasos 5 y 6).

REVISIÓN DEL 04/11/2026

Cómo está hoy

Fase 1 hecha, revisada en el servidor y fusionada con main (PR #33). Con la semilla de prueba, «Mis juegos, por niveles» de tester está abierta y tiene las versiones de demo1 y demo2, así que la de la comunidad ya sale con tres personas. Cada juego lleva debajo una barra con los colores de los niveles en los que lo ha puesto la gente; la mediana se calcula en Python con una sola consulta por página.

Cómo lo haría

1. «En qué te apartas»: en la página de tu versión, los juegos en los que tu nivel queda más lejos del de la comunidad, con «S para ti, C para la comunidad». community_board ya da el nivel de cada juego; falta compararlo con el tuyo.
2. Polémicos: los juegos con los votos más repartidos (dos niveles o más de distancia entre el primer y el último cuarto de los votos), con su marca en la de la comunidad y una sección propia debajo.

3. Aviso a quien la abrió cuando alguien hace su versión: un Notification.Kind nuevo, con su migración de choices, agrupado mientras no lo lea («Ana y 3 más han hecho su versión de…»).
4. Tests de las tres cosas, y que la semilla deje un juego polémico para verlo.

Para profundizar

- Tier list de la comunidad por plantilla: juntar todas las «Jugados en 2026» de la gente, aunque cada una tenga juegos distintos, como resumen del año de NextPlay.
- Colocar por duelos en el móvil («¿cuál te gustó más?»), como en «Ranking personal por duelos»: con 30 juegos sin clasificar, es más cómodo que arrastrar.
- Una imagen para compartir «Mi versión contra la de la comunidad», con la pieza común de imágenes con Pillow.
- Tier lists abiertas con fecha de cierre, como los retos («Los Resident Evil, de mejor a peor: hasta el 30 de noviembre»). Al cerrarse, la de la comunidad queda fija y sale en el feed.
- Convertir la de la comunidad en reto («Juega los de la fila S») con el «Convertir en reto» que ya existe.

---

5. Importar juegos y horas de Xbox

PRIORIDAD ALTA · EN CURSO · Funcionalidad · propuesta el 17/10/2026

Qué es

Vincular cuentas de Xbox (cuenta Microsoft) e importar sus juegos con los minutos jugados, igual que con Steam.

Por qué

Después de Steam, Xbox es la única plataforma que permite hacerlo de forma fiable: el usuario inicia sesión con su cuenta Microsoft y se puede leer su historial de juegos y estadísticas.

Pasos previstos

1. Registrar una aplicación gratuita en Azure (o pedir una clave de OpenXBL, más sencillo) y guardar sus credenciales en el .env.
2. Inicio de sesión con cuenta Microsoft (OAuth) y obtención del token de Xbox Live del usuario.
3. Modelo XboxAccount (usuario, XUID único, gamertag, avatar), con varias cuentas por usuario como en Steam.
4. Leer el historial de juegos y los minutos jugados, y emparejar con el catálogo vía IGDB (que guarda los IDs de Microsoft Store).
5. Sumar las horas con las de Steam cuando un juego está en las dos plataformas.

Notas

Algunos juegos antiguos o de Xbox 360 no tienen estadística de minutos jugados; entrarían sin horas. Programada el 17/10/2026 con la vía oficial (aplicación de Azure + XboxLive.signin), sin guardar tokens. El 18/10/2026 pasó a guardar cada título en la tabla de títulos importados y a desempatar por los dispositivos en los que se jugó. Falta crear la aplicación de Azure (pasos en el README), poner XBOX_CLIENT_ID y XBOX_CLIENT_SECRET en el .env y probarla con una cuenta real.

REVISIÓN DEL 01/11/2026

Cómo está hoy

El código está completo y probado con respuestas simuladas, pero nunca ha corrido de verdad: no hay XBOX_CLIENT_ID en el .env y hay 0 cuentas de Xbox en la base de datos.

Cómo lo haría

1. Tú: crear la aplicación de Azure con los pasos del README y poner las dos variables en el .env. Son diez minutos y no se puede hacer por ti.

2. Probar con una cuenta real y mirar los tres puntos frágiles: que XSTS dé el token con esa aplicación, que titlehub traiga todo el historial (played_titles no pagina: comprobar si corta) y que MinutesPlayed responda.
3. Si XSTS no deja, el plan B es OpenXBL (xbl.io), que tiene su propio inicio de sesión: solo cambia cómo se obtienen los datos; save_titles y update_library siguen igual.
4. Convertir lo que salga en tests con respuestas reales anonimizadas, apuntar en «Importar logros y trofeos» que Xbox ya está probado y pasar a «hecha».

Para profundizar

- Sincronizar solo: SCOPE ya pide offline_access y el token de refresco se tira. Con consentimiento explícito y guardado cifrado (clave fuera de la base de datos), Xbox tendría sesiones cada noche, como Steam. Se borra al desvincular.
- Logros con fecha y rareza: achievements.xboxlive.com los da por título, y sirven para lo mismo que los de Steam (años pasados y fecha de fin).
- Amigos: peoplehub, con el mismo token de cada inicio de sesión, da los gamertags de tus amigos para «Encontrar a tus amigos de Steam y Xbox en NextPlay».
- Cruzar con Game Pass: marcar qué has jugado con la suscripción y qué se va pronto, con los productos que ya da catalog.gamepass.com.

---

6. Diario de partidas que se escribe solo

PRIORIDAD ALTA · EN CURSO · Funcionalidad · propuesta el 21/10/2026

Qué es

Guardar en cada sincronización cuánto ha subido el tiempo de cada juego para tener, sin apuntar nada, un diario de partidas: calendario de días jugados, horas por semana y cuándo empezaste y dejaste cada juego.

Por qué

El diario (Journal) es la función estrella de Backloggd, pero allí cada partida se apunta a mano. NextPlay ya recibe cada noche los minutos y la última partida de Steam, pero solo guarda el total y lo sobrescribe (ImportedTitle.minutes), así que cada noche se pierde un día de historial que después no se puede reconstruir. Además daría las fechas de inicio y de fin que les faltan a «Estadísticas del perfil y resumen anual» y a «Retos del año y control del pendiente».

Pasos previstos

1. Modelo de sesión (usuario, juego, plataforma, día y minutos). En save_titles, antes de sobrescribir los minutos de ImportedTitle, apuntar la diferencia si han subido, con el día de la última partida.
2. Primera y última partida de cada juego de la biblioteca, sacadas de las sesiones.
3. Calendario de actividad (mapa de calor) en el perfil y línea temporal en la ficha: «empezado el 3 de marzo · 14 sesiones · la última hace 2 días».
4. Estados sugeridos solos: un juego en «Jugando» que lleva semanas sin sesiones propone pasar a «Jugado» o «Abandonado», y uno que vuelve a tener sesiones, a «Jugando».
5. PlayStation y Xbox también dejan sus sesiones en cada sincronización a mano, aunque más espaciadas.
6. Respetar el perfil privado y dejar ocultar el diario aunque el perfil sea público.

Notas

Conviene hacer ya el paso 1 aunque el resto espere: el historial que no se guarda no se recupera. Steam solo da el total y la última partida, así que una «sesión» es lo jugado entre dos sincronizaciones. Si algún día se guarda cifrado el token de refresco de Microsoft (ahora no se guarda a propósito), las sesiones de Xbox también serían diarias. Backloggd tiene en su hoja de ruta cambiar los estados según la actividad («Auto Log Organizing») y no lo ha hecho. 25/10/2026: con el feed de actividad, las sesiones darían actividad automática y verificada («Ha jugado 9 h a Elden

Ring esta semana», agrupada por semana), algo que en Backloggd hay que apuntar a mano. 26/10/2026: hecho el paso 1 con los retos (aún sin commit): library.models.PlaySession (usuario, juego, plataforma, día y minutos, sumados si son del mismo día) que apunta accounts.matching.update_library en las importaciones cuando sube el tiempo de un juego que ya tenía horas de esa plataforma, con el día de su última partida; lo de la primera importación y las correcciones del emparejamiento no cuentan. Ya lo usan los retos de horas. Falta lo demás: primera y última partida, calendario, línea temporal en la ficha, estados sugeridos y ocultar el diario. 26/10/2026: paso 1 con commit en la rama retos.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Paso 1 hecho (PlaySession), con solo 8 sesiones porque nightly no corre. Falta todo lo visible. Hay un problema de diseño que conviene arreglar antes de pintarlo: toda la diferencia se apunta en el día de la última partida, así que una sincronización de PlayStation tras dos meses pone 40 h en un solo día.

Cómo lo haría

1. Añadir PlaySession.since: el día de la partida anterior, que update_library ya tiene en la entrada antes de actualizarla. Si coincide con day, la sesión es exacta; si no, es «entre estas fechas». El calendario la pinta distinta y los retos de horas la reparten entre esos días en vez de contarla toda en el último.
2. library/diary.py: calendar(user, year) agrupado por día con una consulta (el índice session_user_day ya existe) y timeline(user, game) para la ficha: primera partida (ImportedTitle.first_played en PlayStation; si no, la primera sesión), última, número de sesiones y horas por mes.
3. Mapa de calor en una pestaña ?vista=diario del perfil: SVG generado en el servidor como la gráfica de games/history.py, una celda por día con su título, sin librerías.
4. En la ficha, dentro de library/_widget.html: «Empezado el 3 de marzo · 14 sesiones · la última hace 2 días».
5. Estados sugeridos en nightly: un juego en Jugando sin sesiones en 30 días crea un aviso con dos botones, y uno que vuelve a tener sesiones propone Jugando. Nunca cambia solo lo que puso el jugador.
6. Profile.diary_visible, respetando visible_to, y las horas de la semana en el feed (Activity.Kind.PLAYTIME, agrupadas) con su interruptor.

Para profundizar

- Reconstruir el pasado con los logros: cada fecha de logro es un día en que seguro se jugó. En tester_steam hay logros de todos los años desde 2022, así que el calendario no tiene por qué empezar el 25/10/2026. Se pintan aparte como «día con logro».
- Sesiones de verdad en PlayStation: según psn-api, la lista de juegos trae playCount, que hoy no se lee; restando dos sincronizaciones se sabe cuántas veces se abrió. Comprobarlo en la próxima sincronización real.
- Sesión con nota: convertir una sesión automática en una entrada con texto e imagen («Hoy he llegado al jefe final»), publicable como Post. Automático de base y personal encima, cuando en otros trackers todo es a mano.
- Ritmo: horas por semana de cada juego, «a este ritmo lo terminas en 3 semanas» con la duración de IGDB, rachas de días seguidos y qué días de la semana juegas más.

7. Logros uno a uno: cuáles tienes, cuándo los conseguiste y lo raros que son

PRIORIDAD ALTA · EN CURSO · Funcionalidad · propuesta el 29/10/2026

Qué es

Guardar cada logro de Steam (y después los trofeos de PlayStation y los logros de Xbox) con su nombre, descripción, icono, la fecha en que se consiguió y el porcentaje mundial de jugadores que lo tienen, en vez de solo «32 de 50».

Por qué

Hoy de cada juego solo se apunta cuántos logros hay y cuántos se tienen (ImportedTitle). GetPlayerAchievements, que ya se pide juego a juego, devuelve la lista entera (qué logro, si se tiene y cuándo), pero steam.player_achievements la cuenta y la tira; con l=spanish trae además el nombre y la descripción sin ninguna petición más. Con la lista se puede enseñar en la ficha qué falta, en el perfil los últimos logros y el más raro, y la fecha de verdad en que se terminó un juego, que es lo que necesita «¿Cuánta gente lo termina?» para proponer marcarlo como Completado.

Pasos previstos

1. Modelos: Achievement por título externo (clave de la API, nombre, descripción, icono, si es oculto y porcentaje mundial), compartido por todos los usuarios, y UserAchievement (título importado, logro y fecha en que se consiguió). Los contadores de ImportedTitle se quedan como resumen para las portadas y el perfil.
2. Steam: pedir GetPlayerAchievements con l=spanish y guardar la lista en vez de contarla; los iconos y si es oculto, de GetSchemaForGame una vez por juego y no por usuario; el porcentaje mundial, de GetGlobalAchievementPercentagesForApp en la tarea nocturna (la misma lectura que «¿Cuánta gente lo termina?»).
3. Ficha: desplegar «Tus logros» con la lista: los conseguidos con su fecha y los que faltan en gris, ordenables por rareza; los ocultos sin desvelar hasta tenerlos.
4. Perfil: los últimos logros conseguidos y el más raro de cada usuario; el «Diario de partidas que se escribe solo» podría contar los logros de cada sesión.
5. PlayStation y Xbox, después: una petición por juego para su lista (trofeos y logros con fecha y rareza), solo de los juegos que han cambiado desde la última lectura.

Notas

Son unas decenas de filas por juego y usuario: nada para la base de datos. La petición de cada usuario y juego es la misma que ya se hace; GetSchemaForGame y los porcentajes son una por juego y valen para todos los usuarios. Ojo con los logros ocultos para no destripar nada. 02/11/2026: fase 1 (Steam) hecha en la rama logros-uno-a-uno, sin commit. Modelos accounts.Achievement (los de cada juego, compartidos: nombre y descripción en castellano, iconos, si es oculto, rareza y orden) y UnlockedAchievement (los que tiene cada uno, con su fecha), y ExternalTitle.achievements_read_at. steam.player_achievements devuelve ya la lista con la fecha de cada logro, sin l=spanish: los textos salen del esquema, una vez por juego. steam.achievement_schema y steam.global_percentages leen los del juego la primera vez que alguien lo importa y los vuelven a leer, cuando se juega, si tienen más de 30 días (ACHIEVEMENTS_INFO_DAYS) o si un jugador trae uno nuevo; si Steam no da el esquema, los de ese jugador esperan a la lectura siguiente. Lo que guarda va en accounts.achievements (save_info y save_unlocked), para que lo usen después PlayStation y Xbox. La migración vuelve a marcar para leer los de Steam que ya se habían contado. Ficha: los logros salen del widget de la biblioteca y pasan a su propia tarjeta, «Tus logros», en la columna de la carátula debajo de la valoración de la comunidad (idea del usuario, 02/11/2026: «en un desplegable como la tarjeta de las valoraciones»); en el móvil baja al final de la ficha. Lleva lo que llevas en cada plataforma y, con «Ver tus logros de Steam», despliega con HTMX los que tienes (con su fecha) y los que te faltan, por fecha, por rareza o en el orden del juego, 12 a la vista y el resto plegado; los ocultos que te faltan van sin icono y con el nombre tapado. Perfil: sección «Logros» del escaparate con el más raro y los 6 últimos, con los ocultos tapados para los demás. Comprobado con Steam: la descripción de los ocultos no la da ninguna llamada; los iconos en la dirección que da el esquema (steamcommunity/public/images) dan 404 en los logros nuevos, así que se piden a shared.akamai.steamstatic.com/community_assets, que los tiene todos y responde en unos 50 ms; el porcentaje llega como texto. Con tester_steam, ya con su segunda cuenta de Steam: 144 juegos, 9583 logros (2229 ocultos) y 2612

conseguidos, solo 1 sin fecha, con logros en 7, 35, 48, 41 y 48 juegos de 2022 a 2026; 13 logros de 4 juegos no salen en el esquema de su juego y se quedan con su clave por nombre. Leerlo todo tarda 54 s (128 juegos con una cuenta) y una sincronización a mano de 40 juegos, 9,8 s con dos cuentas, lo mismo que antes, o 16,1 s si nadie había importado esos juegos. Logros de ejemplo uno a uno en la cuenta tester (seed_test_profile) y 9 tests nuevos (469 en total).

REVISIÓN DEL 02/11/2026

Cómo está hoy

Fase 1 hecha en la rama logros-uno-a-uno, solo de Steam: cada logro se guarda con su fecha, sus textos en castellano y su rareza, y se ve en la ficha («Ver tus logros de Steam») y en la sección «Logros» del perfil. Falta usar las fechas en lo que ya existe, la actividad y PlayStation y Xbox.

Cómo lo haría

1. Usar las fechas en lo que ya existe: «Jugados en 2025» y los años pasados en las tier lists, diciendo «según tus logros»; los días con logro en el calendario del diario; y el 100 % o el último logro de la historia como pista para proponer Completado con su fecha.
2. Actividad: «Ha conseguido 5 logros en Hades», agrupada por juego y día, y en el feed solo los raros y el 100 %, para no llenarlo.
3. PlayStation: guardar el npCommunicationId al sincronizar y pedir los trofeos de cada lista que haya cambiado, con su fecha y su rareza, guardados con save_info y save_unlocked.
4. Xbox: los logros de cada juego (achievements.xboxlive.com) cuando la importación se pueda probar con una cuenta real.
5. Los 13 logros que no salen en el esquema de su juego: probar si GetPlayerAchievements con l=spanish les da nombre y, si es así, pedirlo solo para esos juegos.

Para profundizar

- Historia retroactiva: de los logros salen la primera partida aproximada de Steam (que Steam no da) y los días en que seguro se jugó. Comprobado: 36 juegos con logros en 2025, de los que 14 se perderían con la última partida.
- La prueba de que lo has terminado: el logro final, con su fecha, propone Completado con el completed_at correcto. Con 1 Completado de 348 juegos en tester_steam, es lo que llena esa columna y da sentido a retos y estadísticas.
- Caza de logros: juegos en los que te falta poco, con lo raros que son. En tester_steam hay 15 títulos al 80 % o más sin llegar al 100, como Bloodborne con 38 de 40. Es otro pendiente, más corto y muy agradecido.
- Rareza como puntuación: puntos según el porcentaje mundial y comparar con la gente que sigues. En tester_steam, de 2612 logros de Steam, 163 son de menos del 5 % y 5 de menos del 1 %; el más raro, del 0,3 %.

---

8. La gente que sigues en la ficha y en los perfiles

PRIORIDAD ALTA · PENDIENTE · Funcionalidad · propuesta el 25/10/2026

Qué es

Que seguir a alguien sirva en toda la web: en la ficha, qué han hecho con el juego las personas que sigues (estado, nota, horas verificadas y su reseña primero); en un perfil ajeno, los juegos que tenéis en común y lo compatibles que son vuestros gustos; y sugerencias de a quién seguir según esa compatibilidad.

Por qué

En un tracker el feed no basta: seguir a alguien vale la pena cuando su opinión aparece justo al decidir qué jugar o qué comprar. NextPlay sabe qué tiene cada uno, con qué nota y con cuántas horas verificadas de Steam, PlayStation y Xbox (en Backloggd todo se apunta a mano), así que puede decir en la ficha «Laura lo terminó en 62 h y le puso 4,5».

Pasos previstos

1. Recuadro «Gente que sigues» en la ficha: quién lo tiene, en qué estado, con qué nota y cuántas horas verificadas, y la media de sus notas junto a la de la comunidad.
2. En las reseñas de la ficha, primero las de la gente que sigues.
3. En un perfil ajeno, «En común»: los juegos que tenéis los dos y la compatibilidad de gustos (lo parecidas que son vuestras notas en los juegos que habéis valorado los dos; solo a partir de unos 5 en común).
4. «A quién seguir» en la columna derecha de la portada: perfiles públicos con más compatibilidad y juegos en común. Si un día hay muchos usuarios, se precalcula en el comando nightly.
5. Aviso cuando alguien que sigues reseña un juego que tienes en «Lo quiero» o en Pendiente (uno por juego, sin repetir).
6. Respetar los perfiles privados y, cuando existan, los bloqueos.

Notas

Depende de «Seguir a otros jugadores y feed de actividad». Va antes que «¿A qué jugamos?» y «Encontrar a tus amigos de Steam y Xbox en NextPlay», que se apoyan en lo mismo.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Seguir ya funciona y suggested_people ya cuenta juegos en común con subconsultas. Dato clave: solo 2 de los 348 juegos de tester_steam y 7 de los 259 de tester tienen nota, así que una compatibilidad basada solo en notas saldría vacía casi siempre.

Cómo lo haría

1. social/affinity.py, con friends_on_game(viewer, game): LibraryEntry de la gente que sigue, con perfil visible, y la media de sus notas. Recuadro en games/_community.html, que ya se actualiza por HTMX al cambiar el estado o la nota.
2. En library.views.review_context, primero las reseñas de la gente que sigues: anotar con Exists y ordenar por eso antes que por la fecha, con la etiqueta «Sigues a Laura».
3. «En común» como ?vista=comun en un perfil ajeno: los juegos de los dos, en portadas, con las horas de cada uno.
4. Compatibilidad con varias señales: las notas si hay 5 o más en común; si no, horas en juegos comunes y géneros más jugados. Diciendo con qué se ha calculado («coincidís en 42 juegos y en rol y terror») en vez de un porcentaje que aparenta más de lo que sabe.
5. «A quién seguir» en la columna derecha de la portada, usando suggested_people también cuando ya sigues a alguien, ordenado por esa compatibilidad.
6. Aviso cuando alguien que sigues reseña algo de tu «Lo quiero» o tu Pendiente, uno por juego, y respetar los bloqueos cuando existan.

Para profundizar

- «Lo que ha jugado tu gente y tú no»: recomendación con nombres («tres personas que sigues lo terminaron, con un 4,5 de media»). Es la primera versión del filtrado colaborativo, sin algoritmo.
- Discrepancias: «Le puso 5 y tú 2» como tarjeta en el perfil común; invita a comentar más que cualquier coincidencia.
- Junto al precio: en la lista de deseos y en «Dónde comprarlo», «Dos personas que sigues lo tienen y le dan un 4,5». Es justo el momento en el que esa opinión decide.

- Comparar el avance: «Tú 12 h, Laura 62 h». Con el diario y los logros se puede saber si vais por el mismo sitio y plegar lo que os destriparía.

9. Cabecera y color automáticos según lo que juegas

PRIORIDAD ALTA · PENDIENTE · Funcionalidad · propuesta el 01/11/2026

Qué es

Una opción más en Editar perfil: que la cabecera sea siempre el arte del juego al que más estás jugando y que el color del perfil salga de esa imagen, sin tocar nada.

Por qué

El perfil ya es muy personalizable, pero todo es a mano y se queda viejo. NextPlay sabe a qué juegas (última partida y sesiones), ya tiene el arte de cada juego (games.media) y ya calcula el color del texto sobre cualquier fondo (text_on). Un perfil que cambia solo con lo que juegas es difícil de tener donde todo se apunta a mano, y va con tu objetivo de un perfil muy personalizable.

Pasos previstos

1. Profile.BannerMode.PLAYING: banner_url elige el juego en Jugando con la última partida más reciente, con su arte de games.media; si no hay ninguno, se queda el último.
2. Color: dominant_color en accounts/images.py con Pillow (quantize sobre una miniatura), guardado por juego para no recalcularlo y descartando los colores demasiado oscuros o grises; text_on ya da el contraste del texto.
3. «Usar el color de la cabecera», también como botón de una sola vez para quien tiene una imagen propia; la vista previa en directo lo enseña antes de guardar.
4. Se calcula al sincronizar, sin peticiones de más al abrir el perfil.

Notas

Sale de la revisión del 01/11/2026.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. Profile.banner_game, banner_art, text_on y la vista previa en directo ya existen.

Para profundizar

- Escaparate que cambia solo: secciones automáticas como «Este mes he jugado a…» (PlaySession) o «Mis logros más raros», que se actualizan sin editar nada.
- Vitrina de logros: elegir hasta 5 logros o trofeos para enseñar, como los juegos favoritos, con su rareza (con «Logros uno a uno»).
- Un aspecto por juego: guardar colores y fondo para cada juego y que se activen al jugarlo.

10. Rediseño de toda la web con aspecto de red social

PRIORIDAD ALTA · EN CURSO · Diseño · propuesta el 19/10/2026

Qué es

Llevar a todas las páginas el estilo de las maquetas del usuario: cabecera con Inicio / Descubrir / Videojuegos, tres columnas en escritorio (tu perfil y menú, contenido, juegos en tendencia) y tarjetas oscuras con acentos violeta.

Por qué

Las páginas actuales son funcionales pero sencillas; el usuario quiere que NextPlay se vea como una red social de jugadores, empezando por Editar perfil, que ya se ha rediseñado.

Pasos previstos

1. Editar perfil: índice lateral, vista previa del perfil, interruptores, tiendas en fichas y cuentas en tarjetas por plataforma.
2. Cabecera nueva y estructura de tres columnas reutilizable en base.html.
3. Portada y catálogo, ficha del juego, perfil, lista de deseos, avisos y pantallas de acceso.
4. Versión móvil con barra de navegación inferior.
5. Solo se muestran datos reales: los huecos de publicaciones, seguidores o guardados se rellenan cuando existan esas funciones.

Notas

Propuesta por el usuario con maquetas de móvil y escritorio. Se hace antes que las publicaciones para que estas nazcan ya con el diseño nuevo. 25/10/2026: con «Seguir a otros jugadores y feed de actividad» ya se rellenan los huecos de seguidores (perfil y resumen de la portada) y la portada tiene las pestañas Siguiendo y Descubrir de las maquetas (ramas sin fusionar). 27/10/2026: repaso con el método anti-ui-slop en la rama pulido-del-front: un color por estado de la biblioteca en todas partes, aviso cuando falla un cambio con HTMX y atenuado mientras espera, en el móvil ya nada se sale de lado (la barra inferior se ensanchaba) y en la ficha va antes el estado que las cajas secundarias, el mejor precio arriba de «Dónde comprarlo», vacíos con botón (catálogo y lista de deseos), retos sin «primero» con todos a cero y textos con tokens de color.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sus cinco pasos están prácticamente cubiertos: cabecera y barra inferior en base.html, tokens y contrato de diseño en app.css y los repasos del 27 y 28/10. La excepción son las tres columnas: layout-3 y «En tendencia» solo están en la portada. Lo que queda ya tiene sus propias propuestas: «Inicio en el móvil», «Primer uso», «Formularios que avisan», «Detalles sueltos» y «Páginas de error». Una propuesta «en curso» sin final claro esconde el avance.

Cómo lo haría

1. Decidir las tres columnas fuera de la portada. Yo no las llevaría a la ficha ni al perfil, que necesitan el ancho (carrusel, portadas, fondo), y las probaría en retos y en la lista de deseos solo si la columna derecha aporta algo allí.
2. Comparar cada página con tus maquetas en un pase con capturas a 1280 y 390 px, y apuntar lo que no cuadre en la propuesta pequeña que le toque.
3. Con eso, pasar esta a «hecha» y que lo pendiente siga en las pequeñas.

Para profundizar

- Una página /dev/estilos/, solo con DEBUG, con todos los componentes y estados (botones, tarjetas, vacíos, errores, portadas): el contrato de app.css se puede ver y las regresiones saltan a la vista.
- El color del perfil para toda tu sesión, no solo en tu perfil: --violet ya se sustituye por perfil, y sería una preferencia más de personalización con una línea en base.html.
- Accesibilidad en lo que lleva JavaScript: arrastre, visor y desplegables con teclado y lector de pantalla, foco visible y prefers-reduced-motion.
- Portadas con el tamaño justo: igdb_image_url usa t_cover_big para todo. Con srcset y t_cover_small en listas y feed, el móvil baja bastante menos; pesa más que cualquier otra optimización de imágenes.

11. Publicaciones con imágenes asociadas a un juego

PRIORIDAD ALTA · EN CURSO · Funcionalidad · propuesta el 19/10/2026

Qué es

Que el usuario publique además de reseñar: texto con hasta 4 imágenes, asociado opcionalmente a un juego, con marca de spoiler y visibilidad, y que salga en la portada, en la ficha del juego (pestaña Comunidad) y en su perfil.

Por qué

Ahora solo se puede reseñar un juego de la biblioteca; las publicaciones libres (capturas, momentos, opiniones) son lo que da vida a una red social y lo que muestran las maquetas.

Pasos previstos

1. Modelo Post (autor, texto de hasta 2000 caracteres, juego opcional, spoiler, visibilidad, fecha) y PostImage con Pillow, límite de tamaño y de 4 imágenes; MEDIA_ROOT en local y almacenamiento aparte en producción.
2. Caja «¿Qué estás jugando?» en la portada y página Crear publicación con buscador de juego.
3. Publicaciones en la portada, en la pestaña Comunidad de la ficha y en la pestaña Publicaciones del perfil, respetando perfiles privados y spoilers.
4. Me gusta y comentarios (reutilizable para las reseñas) y guardar publicaciones.
5. Borrar las propias y denunciar para el admin; tests de visibilidad y de subida de imágenes.

Notas

Propuesta por el usuario. Con «Seguir a otros jugadores» la portada tendrá las pestañas Siguiendo y Descubrir de la maqueta. 25/10/2026: el usuario pide quitar «Sigues jugando» de la portada y poner en su sitio la caja para publicar. Hecho en la rama feed-en-la-portada (sin commit; la fase 3 del feed está guardada aparte como árbol de git para poder hacer su commit por separado): modelos Post (hasta 2000 caracteres con el formato de las reseñas, juego opcional y spoiler), PostImage (hasta 4, reutilizando AttachedImage y accounts.images, con su miniatura) y PostLike; caja en la portada con los juegos que estás jugando como fichas para elegir de cuál hablas y vista previa de las imágenes (el navegador reduce las muy grandes); /publicar/ con ?juego= desde el botón «Publicar sobre el juego» de la ficha; tarjeta en el feed y en la actividad del perfil (actividad de tipo publicación, que se borra con ella); página propia con «me gusta» (aviso en la campana una vez por persona), «Copiar enlace», «Borrar» y «Denunciar» (Report genérico; el aviso a los administradores ya no necesita juego); admin con imágenes y denuncias pendientes; 9 tests. Falta: comentarios en las publicaciones (los de las reseñas son solo de reseñas), la pestaña Comunidad de la ficha, una pestaña Publicaciones en el perfil, marcar como spoiler cada imagen y guardar publicaciones. 25/10/2026: hechos los comentarios en las publicaciones (modelo PostComment; la plantilla library/_comments.html y library.views.comment_thread sirven ahora para las reseñas y para las publicaciones), con aviso en la campana, borrar (quien lo escribe y el autor de la publicación), denunciar, contador en el feed y en su página, admin y 5 tests. Falta: la pestaña Comunidad de la ficha, una pestaña Publicaciones en el perfil, marcar como spoiler cada imagen y guardar publicaciones.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Lo principal está hecho y fusionado: Post, imágenes, «me gusta», comentarios, denuncias y feed. Faltan cuatro cosas, confirmadas en el código: la sección en la ficha, la pestaña en el perfil, el spoiler por imagen (PostImage lo hereda de AttachedImage, pero PostForm no lo ofrece y social/_post_images.html no lo mira) y guardar publicaciones.

Cómo lo haría

1. Perfil: ?vista=publicaciones con su contador en las pestañas, reutilizando el cursor de social.views._batch.
2. Ficha: sección «De la comunidad» que se carga con HTMX al llegar a ella (hx-trigger=revealed), con las publicaciones del juego y un filtro «Capturas», reutilizando social/_post_body.html.

3. Spoiler por imagen: una casilla por imagen en la caja de publicar, como en library/review_form.html, y lo mismo en social/_post_images.html que en library/_review_images.html.
4. Guardar: modelo SavedPost y página «Guardados» en el menú de la cuenta. Lo dejaría para el final: es lo que menos aporta hoy, con 2 publicaciones en la base de datos.
5. Pasar a «hecha».

Para profundizar

- Capturas de la comunidad en el carrusel de la ficha (games/_media.html), con un filtro propio: las de publicaciones y reseñas que no son spoiler. Estaba apuntado en el rediseño de la ficha.
- Traer capturas de Steam: IPublishedFileService/GetUserFiles con filetype=4 devuelve las públicas de un usuario (comprobado, responde). «Publicar esta captura» evita descargarla y volver a subirla.
- Hitos que proponen publicar: al llegar a 100 h, al completarlo o al conseguir un platino, ofrecer «¿Lo cuentas?» con el juego ya puesto. Salen de datos verificados.
- Guardar las horas al publicar, como Review.snapshot, para plegar lo que va por delante de quien lee con la regla de AHEAD_HOURS. Y vídeos por enlace (YouTube, Twitch) con miniatura y carga al pulsar, sin alojarlos.

---

12. Ficha del juego más completa

PRIORIDAD ALTA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Traer más datos de IGDB a la ficha: tráiler, capturas, estudio y editora, edad PEGI, modos de juego (cooperativo, online, pantalla partida), DLC y la saga con tu progreso («has jugado 3 de 9»).

Por qué

La ficha solo tiene carátula, resumen, géneros y plataformas; IGDB da todo lo demás en la misma consulta y varias ideas (buscar grupo, «qué juego esta noche») necesitan los modos de juego.

Pasos previstos

1. Ampliar GAME_FIELDS con videos, screenshots, involved_companies, age_ratings, game_modes, multiplayer_modes, dlcs, expansions, franchises y collections.
2. Modelos para compañías y sagas, y campos para PEGI y modos de juego.
3. Tráiler de YouTube que solo se carga al pulsar (para no meter la página de YouTube en cada visita) y galería de capturas.
4. Páginas de estudio y de saga con tu progreso en ella.
5. Volver a importar los juegos existentes.
6. Más campos que IGDB ya da (comprobado con The Witcher 3): storyline, themes, player_perspectives, keywords, game_engines, websites, alternative_names, release_dates por plataforma y región, game_status, game_type, ports, remakes, standalone_expansions, bundles y rating (la nota de los usuarios de IGDB).
7. Usar un arte o una captura apaisada de IGDB como fondo de la cabecera de la ficha en vez de la portada desenfocada (Backloggd pone una captura arriba).
8. Enlace «Ver en IGDB» con el campo url, como el «More on IGDB» de Backloggd.

Notas

21/10/2026: comparada con la ficha de Backloggd, que también sale de IGDB: enseña estudio y editora, una captura arriba, resumen, fecha, géneros, plataformas y pestañas de DLC, actualizaciones, ediciones, saga y mods. De The Witcher 3, IGDB da 16 artes, 12 capturas, 17 vídeos, 7 clasificaciones de edad, 48 idiomas, 16 webs y 82 palabras clave, así que la ficha puede quedar más completa que la suya. Pedirlo todo en cada importación hace las respuestas mucho más grandes: mejor traer lo extra al abrir la ficha (con caché) y por la noche para los más populares.
22/10/2026: el tráiler, la galería de capturas y el arte de la cabecera pasan a «Rediseño de la ficha: cabecera con arte y carrusel de imágenes y vídeos».

REVISIÓN DEL 01/11/2026

Cómo está hoy

Lo visual ya está (cabecera y carrusel, games/media.py). Falta casi todo el dato. Medido hoy en IGDB sobre los 2206 juegos del catálogo: compañías en 2191, clasificación por edades en 2102, modos de juego en 2193, modos multijugador en 838, colecciones en 1776, DLC en 471, expansiones en 270, remasters en 278, remakes en 152 y webs en 2190.

Cómo lo haría

1. Separar lo que se filtra de lo que solo se lee. Se filtra: game_type, game_status y modos, en campos normales o M2M rellenos al importar. Solo se lee: un JSON Game.details con la misma receta que Game.media (se pide con HTMX al abrir la ficha si falta o tiene más de 30 días, y de noche para los populares).
2. IGDBClient.game_details(igdb_id), una sola consulta con los campos expandidos: compañías con desarrolladora y editora, edades (comprobado: age_ratings.organization.name y rating_category.rating dan «PEGI» y «18»), modos, colecciones, franquicias, DLC, remakes, remasters, ports, webs, storyline, temas, perspectivas y motor.
3. Bloque «Ficha técnica» junto a los idiomas, con el mismo <details>; PEGI con su icono; estudio y editora enlazados al catálogo.
4. Páginas de saga solo cuando haga falta: /sagas/<slug>/ con los juegos en orden de salida y tu progreso, que lists.views.list_progress ya sabe calcular.
5. Comando fill_game_fields para el catálogo existente, por lotes de 500 (5 peticiones en total), como localize_games --sin-tiendas.

Para profundizar

- «¿Por dónde empiezo?»: con colecciones y remakes, el orden recomendado de una saga y el aviso «hay un remake de 2023; este es el original».
- «Si te gustó»: similar_games está en 2196 de 2206 juegos. Un bloque con los que ya tienes en tu pendiente primero convierte la ficha en una decisión de qué jugar.
- Accesibilidad: las categorías de Steam incluyen dificultad ajustable, opciones de subtítulos, alternativas de color o jugable sin restricciones de tiempo (comprobado, en español). Se piden en la misma llamada de localize_games con filters=basic,categories, sin peticiones de más.
- Estado real antes de comprar: game_status marca 37 juegos del catálogo como retirados de la venta, 24 sin servidores y 16 en acceso anticipado. «Ya no se vende» o «Servidores cerrados» ahorran disgustos.

---

13. Catálogo con filtros y orden al nivel de Backloggd (y más)

PRIORIDAD ALTA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Que el catálogo se pueda ordenar y filtrar como en Backloggd (año, si ya ha salido, tipo de juego, nota, juegos de nicho…) y además por cosas que solo tiene NextPlay: precio, rebajas, idioma y lo que tienes en tu biblioteca.

Por qué

Backloggd deja ordenar por popularidad, título, tendencia, fecha de salida, nota, tiempo medio de juego y tiempo medio para terminar (en los dos sentidos) y filtrar por tipo (juego principal, DLC, remake, remaster, port, mod…), año, lanzados o próximos, género, plataforma, nota y «popular o de nicho». El catálogo de NextPlay solo filtra por texto, género y plataforma y siempre ordena por popularidad.

Pasos previstos

1. Guardar de IGDB game_type y game_status (ver «Quitar DLC y actualizaciones de las búsquedas en IGDB») para filtrar por tipo y por lanzados o próximos.
2. Filtros nuevos: año o década, lanzado o próximo, tipo de juego, nota mínima (de la crítica o de la comunidad) y popular o de nicho (con la popularidad que ya se guarda).
3. Órdenes nuevos, en los dos sentidos: título, fecha de salida, nota, tendencia (lo más añadido en los últimos 30 días, como en Inicio) y duración (cuando esté «Duración estimada de los juegos»).
4. Filtros que Backloggd no puede tener: precio máximo, rebajado ahora, en su mínimo histórico, doblado al castellano y «no lo tengo» o «está en mi pendiente».
5. Filtros con HTMX que cambian la URL, para poder compartir una búsqueda («RPG de 2015 por menos de 10 €»).

Notas

Los filtros de idioma y de Game Pass dependen de «Fichas en castellano» y de «Saber qué juegos están en Game Pass».

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sigue con texto, género, plataforma e idioma, y orden fijo por popularidad. Ojo: el catálogo local son 2206 juegos y filtros finos saldrán casi vacíos aunque IGDB tenga cientos, así que ampliar el catálogo va en el mismo trabajo.

Cómo lo haría

1. games/filters.py con un formulario que valida todos los parámetros y devuelve el queryset, con un test por filtro: la vista catalog ya ha crecido y esto la vacía.
2. Primero lo que se puede hoy: año o década, nota de la crítica, popular o de nicho (popularity), «no lo tengo» y «en mi pendiente» (Exists sobre LibraryEntry). Precios: Min de prices__price filtrado por las tiendas de Profile.shops, rebajado (cut mayor que 0) y en su mínimo (historical_low).
3. Tipo y estado cuando esté «Quitar DLC y actualizaciones de las búsquedas en IGDB»; duración cuando esté «Duración estimada de los juegos».
4. Órdenes con un diccionario como PROFILE_SORTS, pero de expresiones SQL. Tendencia: las altas de 30 días de trending_games y, con pocos usuarios, popularity_primitives de IGDB.
5. HTMX con hx-get, hx-push-url y la rejilla como destino: la URL se puede compartir y, sin JavaScript, sigue siendo un formulario GET.
6. Ampliar el catálogo por tandas nocturnas con import_igdb (por ejemplo, hasta 20.000) y medir cómo responde cada filtro antes de seguir.

Para profundizar

- Búsquedas guardadas con aviso: guardar «RPG de 2015 por menos de 10 €, doblado» y recibir un aviso cuando aparezca alguno. Encaja con wishlist.check_alerts: una lista de deseos por condiciones y no por juegos.
- Tendencia real desde el primer día: popularity_primitives da datos de Steam (pico de jugadores, más vendidos, más deseados) y de Twitch (horas vistas), comprobado. Sirve hasta que haya comunidad.
- Temas y perspectivas como filtros: themes está en 2180 juegos y player_perspectives en 2178. «Terror en primera persona» dice más que los géneros de IGDB.
- La misma vista para decidir qué jugar: «de tu pendiente, menos de 10 h, en castellano y jugable en Deck». Es la pregunta que da nombre a NextPlay.

14. Que ningún juego se quede sin precio: pedirlo al importar o al abrir la ficha, y el de Steam a Steam

PRIORIDAD MEDIA · EN CURSO · Funcionalidad · propuesta el 02/11/2026

Qué es

Que los juegos nuevos tengan precio sin esperar a la tarea de cada noche, que la ficha distinga «aún no se han buscado» de «no lo vende nadie» y que el precio de Steam salga de la propia Steam, para que no falte cuando IsThereAnyDeal lo apunta en otra edición ni cuando el juego se ha vuelto gratis.

Por qué

Pregunta del usuario (02/11/2026): «¿por qué no todos los juegos de Steam tienen precio?». Medido con tester_steam: de sus 284 juegos de Steam, 146 tienen precio de Steam (10 de ellos, gratis) y 138 no. 123 de esos no tienen ningún precio porque nunca se han pedido: los precios solo llegan con update_prices, que va en la tarea de cada noche y no ha pasado por todo el catálogo desde el 16/10, y nada los pide al importar salvo la lista de deseos (wishlist.fetch_missing_prices). En todo el catálogo son 912 de 2235, y su ficha dice «No se han encontrado precios en las tiendas», cuando no se han buscado. Los otros 15, comprobados en la tienda de Steam (appdetails con cc=es): 8 Steam sí los vende, pero IsThereAnyDeal apunta su precio en la edición que vende Steam y no en el juego que buscamos por su app (Resident Evil 5 Gold Edition a 4,99 €, Resident Evil 6 Complete, Rise of the Tomb Raider: 20 Year Celebration, Dying Light Essentials Edition, BioShock Remastered, BioShock 2 Remastered, ARK: Survival Evolved y Call of Duty: Black Ops III); 4 se han vuelto gratis y no se ve (Command & Conquer 3, Generals, Zero Hour y Red Alert 3); y 3 ya no se venden así (Yakuza 0, sustituido por el Director's Cut; GTA V Legacy, que solo va con Enhanced; y Black Desert, cuya app ya no existe).

Pasos previstos

1. Ya, sin tocar código: lanzar update_prices a mano para los 912 juegos que nunca han tenido precios, y de paso poner al día los del 16/10.
2. Precios que faltan sin esperar a la noche: después de cada importación (Steam, PlayStation y Xbox), wishlist.fetch_missing_prices con los juegos nuevos; y en la ficha, si el juego nunca los ha tenido, pedirlos con HTMX al abrirla, como las imágenes, con «Buscando precios…».
3. Que la ficha distinga «aún no hemos buscado sus precios» de «no lo vende ninguna tienda».
4. El precio de Steam, de Steam: appdetails con filters=price_overview y cc=es acepta muchas apps en una petición y da el precio, el normal y la rebaja de cada una tal como la vende Steam, aunque sea con otra edición. Se guarda como la tienda Steam en lugar del de IsThereAnyDeal, que se queda para las demás tiendas y el mínimo histórico.
5. Gratis y retirados: si Steam lo da gratis, «Gratis en Steam»; si la app ya no se vende, «Steam ya no lo vende» con el enlace a su página, en vez de no decir nada.

Notas

appdetails ya se usa en games/spanish.py (localize_games), con pausas entre peticiones: no está documentada como la Web API y Steam la corta si se le piden demasiadas seguidas, así que por lotes y de noche. Relacionada con «¿Qué edición compro? Precios de cada edición y remaster», con «Ediciones agrupadas bajo su juego (GOTY, Complete, Definitive…)» y con «Sincronización automática de la biblioteca», que es la que tendría los precios al día cada noche. 02/11/2026: paso 1 hecho, con el visto bueno del usuario: update_prices a mano sobre todo el catálogo, en 816 s. 1481 juegos con los precios pedidos de nuevo y 1452 con algún precio (1347 de PC, 896 de Xbox y 402 de Nintendo); los 754 que siguen sin pedir no tienen ID de Steam ni de GOG con que buscarlos en IsThereAnyDeal (update_prices se salta los que no tienen price_lookups). De los 284 juegos de Steam de tester_steam, 256 tienen ya precio de Steam (11 gratis) y quedan 28: los de otra edición de arriba, más ediciones que Steam solo vende en un paquete (la trilogía Definitive de GTA, Resident Evil Re:Verse y Resistance, los Deathmatch de Half-Life), los Command & Conquer, que ahora son gratis, y algunos retirados. Salieron 6 avisos de la lista de deseos de tester_steam, con su correo solo en la consola. Los pasos 2 y 3 van en una sesión aparte, en su propia rama. 02/11/2026: hecho lo de los IDs de tienda guardados en otra ficha (ver «profundizar»), en la rama ids-de-ediciones-y-ports (PR #31, fusionado el 03/11/2026): igdb.GAME_FIELDS pide también los IDs de los ports y de las ediciones ampliadas (RELATED_STORES) y _store_alternatives los usa después de las ediciones y antes que

los remasters; igdb.IGDBClient.games_by_ids; e import_igdb --sin-ids-de-tienda vuelve a pedir a IGDB los juegos del catálogo que no tienen con qué buscar sus precios. Lanzado sobre la base de datos local: de los 754, 41 tienen ya con qué buscarlos y 35 tienen precio de PC, como Red Dead Redemption (24,99 € en Steam), The Witcher 2 con su Enhanced Edition (2,99 € en GOG), Ghost of Tsushima Director's Cut, Persona 5 Royal o Final Fantasy XV Windows Edition; la ficha dice la edición que se vende. Los 6 que siguen sin precio (Metal Gear Solid 2, Golden Axe, Unreal Tournament 2003, Altered Beast, Heretic y Jazz Jackrabbit 2) no se venden sueltos. 3 tests nuevos. 03/11/2026: pasos 2 y 3 hechos en la rama precios-al-importar-y-en-la-ficha (PR #29, fusionado el 03/11/2026). Al importar de Steam, PlayStation o Xbox, y al traer la lista de deseos de Steam, se piden al momento los precios de los juegos de la biblioteca que nunca se han buscado, con games.prices.fetch_missing, que sustituye a wishlist.fetch_missing_prices: los de PC de todos a la vez, y los de Xbox y Nintendo solo de los que se quedan sin ningún precio, porque esas tiendas se buscan por el nombre juego a juego y alargarían mucho una importación grande (tester_steam tiene 292 juegos en Xbox y 138 en Switch). Si IsThereAnyDeal no responde, la importación vale igual y se vuelven a pedir en la siguiente, al abrir la ficha o por la noche; ni la importación ni la ficha esperan si pide calma (ITADClient con retries=0). sync_libraries no los pide, porque en la tarea de cada noche va update_prices justo detrás. En la ficha, si nunca se han buscado, «Dónde comprarlo» llega por HTMX con «Buscando precios…» y trae también los de Xbox y Nintendo de ese juego. La ficha distingue «Aún no hemos buscado sus precios», «Ninguna tienda de PC, Xbox ni Nintendo lo vende ahora» (con cuándo se buscaron; la PlayStation Store sigue siendo un enlace para buscarlo) y «No se han podido buscar sus precios». update_prices marca ya como buscados los juegos sin ID de Steam ni de GOG, porque no tener forma de buscarlos en PC no es lo mismo que no haberlos buscado: antes se quedaban sin fecha para siempre y la ficha los habría pedido cada vez que se abriera. Probado en una copia de la base de datos local: la importación real de Steam de tester_steam tardó 22 s, 5 de ellos en los precios de los 35 juegos de su biblioteca sin buscar (3 de PC a los que se quitaron los precios para hacerlos pasar por nuevos, que volvieron con los suyos, y 32 de consola); y la ficha de Hollow Knight, sin precios, los trajo al abrirla de PC, Xbox y Nintendo.

REVISIÓN DEL 03/11/2026

Cómo está hoy

Pasos 1, 2 y 3 hechos: el 1 a mano y el 2 y el 3 en el #29, fusionado el 03/11/2026. Los IDs de tienda que IGDB guarda en otra ficha, en el #31, también fusionado el 03/11/2026. Los pasos 4 y 5, el precio de Steam de la propia Steam y los gratis y retirados, sin empezar.

Para profundizar

- Antes de cambiar de fuente: comprobar con unas pocas apps a la vez que filters=price_overview devuelve el precio de cada una y cuántas peticiones seguidas aguanta Steam.
- De los que se venden en otra edición, guardar el nombre del paquete que vende Steam («Resident Evil 5 Gold Edition») y enseñarlo junto al precio, como ya hace prices_edition.
- Que los avisos de la lista de deseos usen ese precio de Steam, y no el de una edición que no es la que se compra.
- IDs de tienda que IGDB guarda en otra ficha. De los 754 juegos sin ID de Steam ni de GOG, 231 salen en PC (comprobado el 02/11/2026). Unos se venden fuera de Steam y de GOG (Battle.net, Riot, Epic, el launcher de Minecraft), pero otros están en Steam o en GOG en otra ficha de IGDB: The Witcher 2 solo como «Enhanced Edition» (tipo Expanded Game) y Red Dead Redemption con su versión de PC aparte (tipo Port). Tomar también los IDs de las ediciones ampliadas y de los ports al importar, y en la pasada a IGDB del catálogo, les daría precio. Hecho el 02/11/2026 (ver las notas). Los recopilatorios (74 de los 754) y los remakes (15) también tienen IDs de tienda, pero no son el mismo juego: si algún día se usan, que se vea bien qué se compra («Se vende dentro de Mega Man Legacy Collection»).
- Por consolas, los 754 (un juego cuenta en todas las suyas): Nintendo 352 (253 solo de Nintendo), PlayStation 347 (136), PC 231 (63), Xbox 190 (12), móvil 55. De los 523 que no salen en PC, 430 solo están en consolas antiguas, sin tienda digital de la que sacar precio. De la generación actual hay 43 exclusivos de PlayStation (The Last of Us, Uncharted 4, Bloodborne…) que no tendrán precio hasta leer la PlayStation Store («Precios de Nintendo eShop y PlayStation Store»).
- Precios de consola al importar de Xbox: ahora en una importación solo se buscan los de Xbox y Nintendo de los juegos que se quedan sin ningún precio. Los de Xbox con ID de IGDB se pueden pedir de 20 en 20 sin buscar por el nombre (xbox.fetch_products): pedir esos al importar de Xbox y dejar las búsquedas por nombre para la ficha y la noche.

15. Arreglos pequeños que han salido al medir: mejor precio, redirección y consultas de más

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 01/11/2026

Qué es

Cuatro retoques de una tarde: que la ficha diga siempre cuál es el mejor precio, que la vuelta de «Lo quiero» use la misma comprobación de seguridad que las demás, evitar una consulta de más por cada autor de reseña o comentario y quitar lo que ya nadie usa.

Por qué

Al medir la web el 01/11/2026, 676 de las 1265 fichas con precios no enseñan «Mejor precio ahora»: Game.price_groups solo lo devuelve cuando la tienda más barata ha quedado plegada, y la plantilla da por hecho que siempre está. Lo demás son restos: library.views._wish_response comprueba next a mano y deja pasar «/\evil.com», que los navegadores tratan como otra web (va por POST con CSRF, así que el riesgo es bajo); ReviewQuerySet.visible_to y comment_thread no cargan user__xbox_account; y sobran 7 clases de app.css y una propiedad, LibraryEntry.steam_hours, que solo usan los tests.

Pasos previstos

1. En games.views.detail, best_price sale de game.cheapest con las tiendas del usuario, y price_groups deja de devolverlo. La plantilla se queda con un solo sitio para el mínimo histórico. Test con la tienda más barata a la vista y plegada.
2. _wish_response usa url_has_allowed_host_and_scheme, como accounts.views._back.
3. Añadir user__xbox_account a ReviewQuerySet.visible_to y a comment_thread, con un test de assertNumQueries.
4. Quitar fact-source, rate-star, rate-stars, review-form, review-stars, settings-form y shop-logo de app.css (steam_hours ya está quitada).

Notas

La misma medición salió bien en casi todo lo demás: en local, ninguna página pasa de 300 ms (salvo la primera visita tras arrancar, 359 ms) ni de 37 consultas; no hay más funciones ni plantillas sin usar; todas las imágenes llevan alt y casi todas, carga diferida; el formato de las reseñas es seguro; las migraciones están al día y los 422 tests pasan en 33 s. 02/11/2026: steam_hours ya no está: la sustituye LibraryEntry.hours_by_platform (ver «Horas de cada plataforma, a la vista»).

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. Comprobado en el código y con la web funcionando.

Para profundizar

- Señalar la más barata también entre las fichas de tienda, no solo en la línea de arriba, y decir si es más barata en una tienda que no has elegido.
- Medidas fijas: un test por página principal con assertNumQueries, para que estas cifras no empeoren sin que nadie se entere.

16. Que las pantallas de cada función se comprueben solas

PRIORIDAD MEDIA · PENDIENTE · Documentación · propuesta el 03/11/2026

Qué es

Un test que pare las funciones nuevas sin su párrafo «Pantallas:» y un aviso en el PDF de funciones cuando el código usa una función en una pantalla que su docstring no dice, para que el mapa de pantallas no se quede viejo.

Por qué

Desde el 03/11/2026 las 695 funciones y los 14 bloques de JavaScript dicen a qué pantallas afectan, con los nombres de docs/pantallas.json, y el PDF de funciones arma con eso el mapa de pantallas. Pero lo escribe quien cambia el código: si una función empieza a usarse en otra pantalla, nada avisa, y el PDF solo marca en rojo lo que falta o lo que nombra una pantalla que no existe.

Pasos previstos

1. docs/pantallas.py con el análisis que sirvió para escribirlas: las URL de cada urls.py, las plantillas que pinta cada vista, los {% include %} y los {% url %} de cada plantilla y las llamadas entre funciones. De ahí sale, para cada función, en qué pantallas se ejecuta.
2. Un test que falle si una función no termina su docstring con «Pantallas:» o nombra una pantalla que no está en el catálogo, para que no llegue a main.
3. En el PDF, una sección «Pantallas que el código ve y la docstring no dice», solo como aviso: el análisis no sabe dónde sale lo que se guarda (lo que importa Steam se ve en el Perfil), así que propone, no decide.

Notas

Sale de reestructurar la documentación para que cada función diga qué hace y a qué pantallas afecta (03/11/2026).

REVISIÓN DEL 03/11/2026

Cómo está hoy

Propuesta nueva. El análisis existe, pero fuera del repositorio: acierta con las vistas, lo que llaman y lo que usan las plantillas, y falla con los métodos que se llaman igual en varios modelos (get_absolute_url sale en todas las pantallas) y con lo que no se llama por su nombre (las señales y los métodos de los formularios).

Para profundizar

- Qué pantallas mirar en cada rama: con el diff, las pantallas de las funciones y plantillas que cambian. Es la lista de lo que revisar en el servidor antes de fusionar.
- Las mismas pantallas en las plantillas: un comentario de cabecera en cada trozo (_algo.html) con dónde sale, que el análisis ya sabe calcular.

17. Intercambio de pendientes: elige un juego del pendiente de un amigo

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 01/11/2026

Qué es

Un reto entre dos personas: cada una elige un juego del pendiente de la otra y las dos tienen que terminarlo. Se cuenta solo, con lo que ya miden los retos.

Por qué

El pendiente es lo que más crece con las importaciones (120 juegos en tester_steam, con una mediana de 17 h según IGDB y 22 de 10 h o menos), y lo difícil es decidir. Que te lo elija alguien que te conoce quita esa decisión, da conversación y solo se puede hacer con bibliotecas importadas de verdad.

Pasos previstos

1. Challenge de dos personas: se propone a quien te sigue y acepta, como las invitaciones de ListCollaborator. Cada una elige entre los juegos en Pendiente de la otra, con filtros por duración y por castellano.
2. El juego elegido se queda oculto hasta que han elegido las dos, para que sea justo.
3. Progreso con challenges.progress, un juego por persona, y al feed al empezar y al terminar.
4. Al terminar, cada una reseña el que le tocó, con un enlace directo al editor.

Notas

Sale de la revisión del 01/11/2026. Se apoya en la fase 1 de retos, ya hecha.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. Follow y ListCollaborator dan el patrón de la invitación, y Challenge.Kind.GAMES, el de contar el progreso.

Para profundizar

- Rueda de grupo: con más gente, cada uno elige para el siguiente, en cadena, con una tanda al mes.
- «Elígeme algo»: pedir a tus seguidores que voten qué juego de tu pendiente jugar ahora, con los más cortos como opciones.
- Apuesta simbólica: quien no lo termina a tiempo reseña el juego que elija el otro.

---

18. Biblioteca del perfil por tandas en vez de entera

PRIORIDAD MEDIA · PENDIENTE · Diseño · propuesta el 27/10/2026

Qué es

La pestaña Biblioteca del perfil enseña ahora todos los juegos de una vez: con una importación de Steam de 259 juegos la página mide unos 16.000 píxeles. Enseñar los primeros 48 y un «Cargar más» con HTMX (y sin JavaScript, con ?pagina=), como ya hace la pestaña Actividad.

Por qué

Al repasar el front con anti-ui-slop se vio que quien importa una biblioteca grande recibe cientos de portadas en una sola página: tarda en cargar, el pie no se alcanza y en el móvil es interminable. Las importaciones de Steam, PlayStation y Xbox hacen que lo normal sean cientos de juegos.

Pasos previstos

1. accounts.views.profile: cortar la lista ya filtrada y ordenada en tandas de 48 (el recuento de las pestañas sigue siendo el total).
2. Botón «Cargar más» al final de la rejilla que pide la siguiente tanda con HTMX y la añade debajo, con el mismo patrón que social/_activities.html.
3. Mantener el estado, el idioma y el orden elegidos al pedir cada tanda.

Notas

La vista previa de Editar perfil ya enseña solo los primeros (PREVIEW_LIBRARY), así que no le afecta.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sigue igual, pero en la revisión anterior exageré. Medido con tester_steam (348 juegos): 289 ms en el servidor y 23 consultas, 207 KB de HTML (23 KB comprimida) y casi todas sus 374 imágenes con carga diferida. No es urgente: lo que se nota hoy es lo larga que es la página, sobre todo en el móvil. Las consultas importarán cuando haya bibliotecas de miles de juegos.

Cómo lo haría

1. Tandas de 48 y «Cargar más» con hx-swap=beforeend y el mismo patrón que social/_activities.html; sin JavaScript, ?pagina=. Con hx-push-url, volver atrás no pierde el sitio. Esto solo ya resuelve lo que se nota hoy.
2. Después, cuando haga falta: recuentos de las pestañas con una consulta agregada por estado y las cifras de la cabecera con agregados aparte, sin cargar las entradas.
3. Orden y filtros en SQL. Para ordenar por horas basta el primer paso de «Horas por plataforma sin una columna por tienda» (total_minutes).
4. Pedir reseñas y listas solo en su pestaña; los contadores, con count.
5. Un test con assertNumQueries y 300 juegos, para que no crezca sin que nadie se entere.

Para profundizar

- Buscar en tu biblioteca: un campo en la barra con HTMX (search_filter ya existe). Con cientos de juegos importados es imprescindible.
- Selección múltiple: marcar varios y pasarlos a un estado, a una lista o a un reto de una vez. Es lo que hace falta para ordenar una importación de 300 juegos.
- Vista en filas, con horas, última partida y logros, ordenable por columnas, además de las portadas.
- Agrupar por año de última partida o por plataforma: una biblioteca de cientos se recorre mucho mejor por tramos que en una sola rejilla.

---

19. Horas por plataforma sin una columna por tienda

PRIORIDAD MEDIA · PENDIENTE · Infraestructura · propuesta el 01/11/2026

Qué es

Guardar en cada juego de la biblioteca las horas totales y la última partida en columnas propias, y las de cada plataforma en un solo campo, para poder ordenar y filtrar en la base de datos y añadir plataformas sin migraciones.

Por qué

LibraryEntry tiene dos columnas por plataforma (steam_, psn_ y xbox_playtime_minutes y last_played) y las horas totales son una propiedad de Python: por eso _profile_context carga toda la biblioteca para ordenar y Game.community_rating repite las tres columnas. Importar Switch, Epic, GOG o Battle.net añadiría dos columnas por tienda y tocaría unas 90 referencias entre código y tests.

Pasos previstos

1. Añadir total_minutes y last_played, con índice, que mantiene update_import y rellena una migración. Solo con este paso ya mejoran «Biblioteca del perfil por tandas en vez de entera», el catálogo y las estadísticas.
2. Añadir minutes_by_platform y last_played_by_platform (JSON), y leer de ahí en playtime_sources, steam_hours, Review.snapshot, Game.community_rating y lists.views.
3. Quitar las seis columnas en otro PR, cuando nada las lea. HOURS_SOURCES pasa a ser la única lista de plataformas, con su nombre y su logo.
4. Un test con una plataforma inventada: si pasa sin tocar nada más, el trabajo está hecho.

Notas

Sale de la revisión del 01/11/2026. El primer paso se puede hacer ya, por separado. Va antes de «Importar Nintendo Switch con capturas de la app Nintendo Store» y de «Importar Epic, GOG, EA y Battle.net con Playnite, y CSV de otros trackers».

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. update_library ya recibe la plataforma como texto y compone el nombre de la columna, así que es casi genérica; lo que no lo es son los modelos.

Para profundizar

- LibraryEntry como resumen recalculable: ImportedTitle ya tiene los minutos por título, de modo que un comando recalc_hours puede rehacer las horas de cualquiera tras una corrección o un error.
- Horas declaradas: dejar apuntarlas a mano para juegos sin plataforma (físico o retro), marcadas como no verificadas y fuera de la media «con horas jugadas».
- Horas por dispositivo: deck_minutes ya existe y PlayStation separa PS4 y PS5. Con el campo genérico, «lo jugaste sobre todo en Deck» sale para cualquier plataforma.

---

20. Primer uso: traer tus juegos nada más crear la cuenta

PRIORIDAD MEDIA · PENDIENTE · Diseño · propuesta el 28/10/2026

Qué es

Al registrarse, NextPlay manda al catálogo con «Añade tus primeros juegos a la biblioteca», y ni el inicio ni la biblioteca vacía del perfil ofrecen traerlos de Steam, PlayStation o Xbox, que es lo que trae las horas. Mientras la biblioteca esté vacía, el inicio enseñaría arriba una tarjeta «Trae tus juegos» con un botón por plataforma y, debajo, la opción de marcarlos a mano en el catálogo.

Por qué

En el repaso con anti-ui-slop del 28/10/2026 se vio que el primer uso no lleva a lo que distingue a NextPlay: importar la biblioteca con sus horas. accounts.views.signup redirige al catálogo; el inicio (games/home.html) no le dice nada a quien tiene la biblioteca vacía; y el vacío de la biblioteca del perfil solo nombra Steam, en texto y sin botón, aunque ya se pueden vincular PlayStation y Xbox. El botón «Vincular cuentas» solo está en la cabecera del perfil.

Pasos previstos

1. Tarjeta «Trae tus juegos» en el inicio mientras la biblioteca esté vacía: botones de Steam, PlayStation y Xbox que abren el diálogo «Vincular cuentas» (accounts/linking/_dialog.html) y un enlace al catálogo para añadirlos a mano.
2. Los mismos botones en el vacío de la biblioteca del perfil, en .empty-actions, como ya tienen el catálogo y la lista de deseos.
3. Después de registrarse, ir al inicio con esa tarjeta en vez de al catálogo, con un aviso que diga qué hacer primero.
4. Quitar la tarjeta en cuanto haya algún juego en la biblioteca o una plataforma vinculada.

Notas

Al vincular Steam la biblioteca se importa en la misma petición de vuelta y tarda unos segundos; «Sincronizar» e «Importar» tampoco avisan mientras trabajan (ver «Formularios que avisan mientras trabajan»).

REVISIÓN DEL 01/11/2026

Cómo está hoy

Igual: signup manda al catálogo y el vacío de la biblioteca solo nombra Steam. El diálogo de vincular (accounts/linking/_dialog.html) y su data-open-dialog ya sirven para cualquier página; ahora solo se incluyen en el perfil.

Cómo lo haría

1. En games/home.html, si summary.games es 0 y no hay cuentas vinculadas, una tarjeta «Trae tus juegos» antes de la caja de publicar, e incluir el diálogo también en la portada.
2. Un botón por plataforma que abra su panel: Steam va directo a steam_link, PlayStation explica el NPSSO y Xbox sale solo si está configurado. Debajo, «o añádelos a mano» al catálogo.
3. signup redirige al inicio, con un mensaje que diga qué hacer primero.
4. Los mismos botones en el vacío de la biblioteca (.empty-actions). La tarjeta desaparece en cuanto hay un juego o una cuenta vinculada, sin ningún modelo nuevo.
5. Tests de las tres situaciones: recién registrado, con cuenta vinculada y con juegos.

Para profundizar

- Resumen de la primera importación: «1.200 horas, tus 3 más jugados, 12 de tus deseados están en su mínimo y 8 sin emparejar». Los datos ya están en _report_import y _report_wishlist; hoy son un mensaje de una línea, y es el mejor momento para enseñar lo que hace NextPlay.
- Valorar en ráfaga: tras importar, «puntúa tus 10 más jugados» en una sola pantalla. Con 2 notas de 348 juegos, es lo que más desbloquea para la compatibilidad y las recomendaciones.
- Primera tier list en un clic: ofrecer «Jugados en 2026» justo después de importar; pasa de cero a algo tuyo que compartir en un minuto.
- Lista de primeros pasos que se tacha sola (foto, juegos, primera nota, primera lista, seguir a alguien) y desaparece al completarla.

---

21. Inicio en el móvil: la actividad antes que tu resumen

PRIORIDAD MEDIA · PENDIENTE · Diseño · propuesta el 28/10/2026

Qué es

En un móvil de 390 × 844 la primera pantalla del inicio es tu tarjeta (foto, bio y contadores) y la caja de publicar entera, con todos los juegos que estás jugando; las pestañas Siguiendo y Descubrir empiezan debajo de la barra inferior y no se ve ninguna actividad sin desplazarse. En el móvil, quitar esa tarjeta, cuyos datos ya están en la pestaña Perfil, y dejar la caja de publicar en una línea que se abre al tocarla.

Por qué

Es la pantalla que más se abre y lo que se viene a ver es lo que hacen los demás. Lo enseñaron las capturas a 390 px del repaso con anti-ui-slop del 28/10/2026, y va contra la regla 6 del contrato de diseño de app.css: en el móvil, primero lo que más se usa.

Pasos previstos

1. Ocultar .me-card en el inicio por debajo de 760 px (la barra inferior ya lleva al perfil).
2. Caja de publicar plegada en el móvil: solo el campo «¿Qué estás jugando?»; los juegos, «Imágenes», «Es un spoiler» y «Publicar» salen al tocarla o si ya tiene texto, con :focus-within y :has() y sin JavaScript.
3. Comprobar a 360 y 390 px que las pestañas del feed y la primera actividad se ven sin desplazarse.

Notas
En escritorio no cambia nada: las tres columnas siguen igual.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Igual: la tarjeta de resumen y la caja de publicar van enteras antes del feed en cualquier ancho. No necesita nada de servidor.

Cómo lo haría
1. En app.css, por debajo de 760 px, ocultar .me-card: la pestaña Perfil de la barra inferior ya lleva a esos datos.
2. Caja de publicar plegada con :focus-within y :has(textarea:not(:placeholder-shown)), sin JavaScript: solo se ve el campo, y lo demás sale al tocarlo o si ya tiene texto.
3. Comprobar a 360 y 390 px que las pestañas y la primera actividad se ven sin desplazarse, y que en escritorio no cambia nada.

Para profundizar
- Pestañas fijas al desplazarse (position: sticky), para cambiar de Siguiendo a Descubrir sin volver arriba.
- Instalable: manifest.json, iconos y theme-color para abrirla desde la pantalla de inicio. Sin service worker complicado; la barra inferior ya parece una app.
- Compartir nativo: navigator.share en «Copiar enlace» y, ya instalada en Android, que aparezca en el menú de compartir del móvil para mandar una captura directamente a «Publicar» (Web Share Target).
- Gesto para cambiar de pestaña deslizando, con scroll-snap y sin librerías, como ya hace el carrusel.

---

22. Formularios que avisan mientras trabajan

PRIORIDAD MEDIA · PENDIENTE · Diseño · propuesta el 28/10/2026

Qué es
Lo que va por HTMX ya se atenúa mientras espera, pero los formularios normales no dan ninguna señal: al pulsar «Sincronizar» en Steam, «Publicar» con imágenes o «Guardar cambios» en Editar perfil, la página se queda quieta hasta que responde el servidor y un segundo toque lo envía dos veces. Al enviarlos, desactivar el botón y poner en él lo que está pasando: «Importando tus juegos…», «Publicando…», «Guardando…».

Por qué
La regla 4 del contrato de diseño de app.css dice que toda petición al servidor enseña que está esperando, y hoy solo se cumple con HTMX (lo vio el repaso con anti-ui-slop del 28/10/2026). La importación de Steam se hace dentro de la petición (accounts.views._import_steam: primero Steam y luego IGDB) y con bibliotecas grandes tarda; una publicación con imágenes se puede duplicar.

Pasos previstos
1. Un script en base.html: al enviar un formulario sin hx-post, ponerle aria-busy, desactivar su botón y, si el botón tiene data-busy, cambiar su texto por ese.
2. Volver a activarlo en pageshow, para cuando se vuelve atrás y el navegador recupera la página de su caché.
3. data-busy en los botones de Steam, PlayStation y Xbox, en Publicar, en Guardar cambios y en los de crear y editar listas y retos.

Notas

Los filtros que se envían solos al cambiar un desplegable usan this.form.submit(), que no lanza el evento submit: si se quiere que también avisen, cambiarlo por requestSubmit().

REVISIÓN DEL 01/11/2026

Cómo está hoy

Igual: ninguna plantilla usa aria-busy y siete desplegables siguen con this.form.submit() (3 en el catálogo, 2 en el perfil y 2 en la lista de deseos). Hay una trampa en su plan: _review_editor decide con «borrador» in request.POST, y un botón desactivado en el evento submit no se envía. Hecho tal cual, «Guardar borrador» publicaría la reseña.

Cómo lo haría

1. Un script en base.html, junto al de los avisos, por delegación: en submit, si el formulario no lleva hx-*, poner aria-busy y cambiar el texto del botón (event.submitter) por su data-busy. Desactivarlo después con setTimeout(…, 0), para que el navegador ya haya recogido su nombre, e impedir un segundo envío con una marca en el formulario.
2. En pageshow con event.persisted, quitar todo eso: es lo que pasa al volver atrás.
3. data-busy en los botones de Steam, PlayStation y Xbox, en Publicar, en Guardar cambios y en los de crear y editar listas y retos.
4. Cambiar los siete this.form.submit() por requestSubmit(), que sí lanza el evento.
5. Probarlo en el navegador con el editor de reseñas, comprobando que el borrador sigue siendo borrador.

Para profundizar

- Evitar duplicados también en el servidor: un botón desactivado no protege de dos pestañas ni de un reintento con mala red. Un token de un solo uso en Publicar y en crear listas lo cierra.
- Progreso de verdad en las importaciones: con «Tareas en segundo plano para lo que tarda», «Leyendo Steam · Emparejando 267 juegos · Logros 40 de 133».
- Barra de subida en publicaciones y reseñas con fotos grandes, con el evento htmx:xhr:progress o XMLHttpRequest.
- Respuesta optimista en lo pequeño (me gusta, estado, favorito): cambiar al momento y deshacer con el aviso que ya existe si falla.

---

23. Listas colaborativas: invitar solo a quien te sigue y avisar al aceptar

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 26/10/2026

Qué es

Tres arreglos de las listas que se vieron antes de subirlas: que solo se pueda invitar a editar una lista a quien te sigue, avisar al dueño cuando alguien acepta y que un color de nivel mal guardado no rompa la página de una tier list.

Por qué

Ahora se puede invitar a cualquiera y, creando varias listas, mandarle muchos avisos sin que haya pedido nada; el dueño no se entera de que alguien ha aceptado; y accounts.models.text_on falla con un color que no sea «#rrggbb», cosa que solo puede pasar si se edita a mano desde el admin.

Pasos previstos

1. En lists.views.invite, solo gente que sigue al dueño (seguir ya es una forma de decir que quiere saber de él); en el editor, sugerir sus seguidores en vez de la gente a la que sigue.
2. Aviso al dueño cuando alguien acepta la invitación («Luis ya puede editar tu lista…»), con su foto.

3. En GameList.tier_list, si un color no es «#rrggbb», usar uno de los de serie en vez de fallar.

Notas

Salen de la revisión de la fase 3 de «Listas personalizadas de juegos» (26/10/2026); el usuario prefirió seguir con los retos y dejarlos para después.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Confirmado en el código: lists.views.invite acepta a cualquiera, accept no avisa al dueño y GameList.tier_list pasa cada color por text_on sin comprobarlo.

Cómo lo haría

1. En invite, comprobar Follow de invitee hacia request.user, con el error «Solo puedes invitar a quien te sigue»; collaborators_context sugiere owner.followers en vez de owner.following.
2. En accept, un aviso al dueño con actor y un Notification.Kind nuevo (con su migración de choices, como la 0019).
3. En GameList.tier_list, validar con accounts.models.HEX_COLOR y usar el de DEFAULT_TIERS en su misma posición si no cuadra. Validarlo también al guardar en el admin.
4. Tests de los tres casos. Ajustar los tests y seed_test_profile que hoy invitan a gente que no sigue al dueño.

Para profundizar

- Invitar por enlace: un enlace con token para un amigo que aún no te sigue, en vez de abrir la invitación a cualquiera.
- Lista por votos: cada colaborador ordena por su cuenta y la lista enseña la media («la tier list del grupo») y dónde discrepáis. Encaja con grupos de amigos y con «¿A qué jugamos?». Para las tier lists abiertas, va en «Tier lists de la comunidad, con su propia pestaña como Retos».
- Historial de cambios: quién añadió, movió o quitó cada juego; added_by ya es la mitad.
- Permisos por persona: puede añadir o también ordenar y quitar, para listas grandes en las que no todos deben reordenar.

---

24. Avisar al salir de Editar perfil con cambios sin guardar

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 22/10/2026

Qué es

Si se han tocado campos de Editar perfil y no se ha pulsado «Guardar cambios», preguntar antes de salir de la página (al pulsar un enlace, cerrar la pestaña o recargar) y señalar en la barra de guardar que hay cambios pendientes.

Por qué

Con la vista previa en escritorio y en móvil apetece probar colores, cabeceras y fondos; es fácil irse a otra página creyendo que ya está guardado y perderlo todo, sobre todo en el móvil, donde la barra inferior está siempre a mano.

Pasos previstos

1. Marcar el formulario como cambiado en cuanto llegue un evento input o change, y desmarcarlo al enviarlo.
2. Evento beforeunload para el aviso del navegador al cerrar o recargar con cambios.
3. Texto «Cambios sin guardar» junto a «Guardar cambios», con el color del perfil.
4. Botón «Descartar» que vuelva a lo guardado (recargar la página) y limpie la vista previa.

Notas
Va justo después de la vista previa en escritorio y móvil (22/10/2026), que pinta los cambios sin guardarlos.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Igual: edit_profile.html no tiene beforeunload, aunque su script ya escucha cada cambio para pedir la vista previa (profile_preview).

Cómo lo haría
1. Aprovechar ese mismo script: guardar el FormData inicial y comparar en cada cambio, para que volver al valor original no cuente como cambio.
2. beforeunload solo si hay cambios; quitarlo en el submit del formulario principal. Comprobar que los otros formularios de la página (tiendas, cuentas vinculadas) no disparan el aviso en falso, y que sí avisa si se pulsa uno de ellos con cambios pendientes.
3. «Cambios sin guardar» junto a «Guardar cambios», con el color del perfil, y «Descartar», que recarga sin preguntar.

Para profundizar
- Borrador automático: guardar en localStorage lo que hay en el formulario (sin las imágenes) y ofrecer «Recuperar cambios» al volver. En el móvil protege más que cualquier aviso.
- Generalizarlo con un atributo data-dirty-guard en el <form> y el script en base.html, para la caja de publicar, los comentarios largos y el editor de reseñas.
- Aspectos guardados: guardar la combinación de color, cabecera y fondo con un nombre y cambiar entre varias. Va con el objetivo de un perfil muy personalizable, y da de paso un «Volver a como estaba».
- Copiar el aspecto de otro perfil si su dueño lo permite: otra forma de personalizar para quien no quiere empezar de cero.

---

25. Quitar DLC y actualizaciones de las búsquedas en IGDB
PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 18/10/2026

Qué es
Que al buscar en IGDB (desde el catálogo o al emparejar un juego importado) solo entren juegos completos, remakes, remasters y ediciones ampliadas, no DLC, trajes ni «Title Update».

Por qué
Buscando «Monster Hunter Wilds» para emparejar un título de Steam entraron al catálogo una docena de trajes y actualizaciones del juego. Ensucian el catálogo, la búsqueda y las recomendaciones futuras.

Pasos previstos
1. Pedir a IGDB el campo game_type (antes category) en las búsquedas.
2. Filtrar en la consulta los tipos de juego principal, remake, remaster, juego ampliado y port.
3. Comando para borrar del catálogo los DLC ya importados que no estén en ninguna biblioteca.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Medido hoy en IGDB sobre los 2206 juegos: 20 DLC, 5 packs y 6 actualizaciones (31 que claramente sobran), más 3 episodios, 3 temporadas y 2 mods por revisar. También hay 40 expansiones y 35 bundles, que no conviene quitar a ciegas. No es tan grave como parecía, pero crece con cada búsqueda. Game no guarda game_type.

Cómo lo haría

1. Game.game_type y Game.game_status (enteros con índice), en GAME_FIELDS y rellenos con fill_game_fields. Los valores los da IGDB (comprobado): 1 DLC, 13 pack, 14 actualización.
2. Filtro en las consultas de búsqueda (search, top_games, games_by_names): excluir 1, 13 y 14. No tocar games_by_steam_ids: lo que alguien tiene de verdad se tiene que poder emparejar.
3. No quitar bundles: IGDB marca así algunas reediciones sin version_parent (GTA V Enhanced), que la gente tiene y juega. Comprobado: 17 de los 348 juegos de tester_steam son bundles, como Mass Effect Legendary Edition o BioShock: The Collection, y solo 1 es un DLC.
4. Catálogo: ocultar por defecto lo que no es juego completo, con el filtro «tipo»; las fichas siguen funcionando por su URL.
5. Comando prune_catalog --dry-run: borra los DLC que no están en ninguna biblioteca, lista, lista de deseos ni reseña, y enseña la lista antes de borrar.

Para profundizar

- Enseñar los DLC donde toca: en la ficha del juego base (dlcs en 471 juegos y expansions en 270), con su precio cuando lo haya. El problema no es que existan, sino que salgan como juegos.
- Etiqueta de tipo en las tarjetas («Remake», «Expansión», «Remaster»), para distinguir los dos Resident Evil 4 de un vistazo.
- Limpiar también lo importado: ocultar solos los títulos sin emparejar que acaban en Soundtrack, SDK, Dedicated Server o Demo. Hoy se ocultan a mano uno a uno.

---

26. Contadores y reparto de valoraciones en la ficha

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Enseñar en la ficha de cada juego cuánta gente de NextPlay lo ha jugado, lo está jugando, lo tiene pendiente, lo quiere y lo ha valorado, con una gráfica de barras del reparto de las notas, como hace Backloggd.

Por qué

Backloggd pone en cada ficha las partidas, los que lo juegan, los pendientes, los deseados, las valoraciones, las listas, las reseñas y los me gusta, más un histograma de notas. NextPlay ya tiene esos datos (estados de la biblioteca, lista de deseos y valoraciones) pero solo enseña la media.

Pasos previstos

1. Una consulta agrupada por estado sobre LibraryEntry del juego y el número de WishlistItem.
2. Histograma de valoraciones de 1 a 5 (o de 0,5 a 5 cuando estén las medias estrellas de «Reseñas completas») en el recuadro de la comunidad.
3. Cada contador enlaza a quién lo juega o lo quiere, respetando los perfiles privados.
4. Guardar los contadores unos minutos en caché en los juegos muy populares.

Notas

Con pocos usuarios las cifras serán pequeñas; se puede enseñar el histograma solo a partir de un mínimo de valoraciones.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Igual. Su nota de 1 a 5 ya es de 1 a 10 medias estrellas, y Game.community_rating agrega ya la media, así que se amplía ahí. Atención a la caché de su paso 4: settings no define CACHES, y sin eso es memoria por proceso.

Cómo lo haría

1. Game.community_stats(): un solo aggregate con Count por estado (filter=Q(status=…)), el histograma con values(«rating») y annotate, y el número de WishlistItem aparte.
2. Histograma de 10 barras en games/_community.html, que ya se refresca por HTMX al votar. Solo a partir de 5 valoraciones.
3. Cada contador enlaza a una página de jugadores paginada, con Profile.visible_filter, como social.views._people. La lista de deseos solo da el número, nunca quién: es privada.
4. Sin caché de momento: con índices y pocos usuarios no hace falta. Cuando haga falta, cache.get_or_set con 5 minutos y una caché compartida en settings.

Para profundizar

- Dos histogramas: todos y solo quienes tienen horas. verified_average ya existe, y la diferencia entre ambos dice más que la media.
- Nota según el final: quien lo terminó frente a quien lo abandonó, y qué porcentaje lo abandona. Es lo más útil antes de empezar un juego largo.
- Mientras haya pocos usuarios, dar datos de fuera: la nota de los usuarios de IGDB (rating, en 2148 de 2206 juegos) y el pico de jugadores. La API de reseñas de Steam (appreviews) me ha devuelto una página HTML en todas las pruebas, así que no cuento con ella.
- Evolución: gráfica de cuánta gente lo juega cada mes con PlaySession, donde se notan los lanzamientos y las grandes actualizaciones.

---

27. Ediciones agrupadas bajo su juego (GOTY, Complete, Definitive…)

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Que las ediciones de un juego no salgan como juegos sueltos sino dentro de la ficha del juego principal, con un selector de edición para las estadísticas y las reseñas, como hace Backloggd.

Por qué

En la biblioteca de la cuenta de prueba sale «Batman: Arkham Asylum - Game of the Year Edition» como un juego aparte, porque Steam vende esa edición con su propio ID; en IGDB es una edición (version_parent) de «Batman: Arkham Asylum», así que sus valoraciones y reseñas no se juntan con las del juego. Backloggd las agrupa («Game Editions Combining») y deja ver las estadísticas de todas las versiones o de una.

Pasos previstos

1. Guardar en Game el juego del que es edición (version_parent de IGDB) y el nombre de la edición (version_title).
2. Al importar de Steam, PlayStation o Xbox, apuntar las horas de una edición en la entrada del juego principal, guardando en qué edición se jugó.
3. En la ficha del juego principal, la lista de ediciones con su precio (ver «¿Qué edición compro?») y un selector «todas las versiones o una edición» para los contadores y las reseñas.
4. Redirigir la ficha de una edición a la del juego principal con esa edición elegida.
5. Comando para fusionar las entradas de biblioteca que ya están repetidas por ediciones.

Notas

Los remasters y remakes son juegos distintos en IGDB (BioShock Remastered es un remaster con parent_game BioShock) y deben seguir separados: solo se juntan las ediciones. Ojo: IGDB marca algunas reediciones como bundle sin version_parent (GTA V Enhanced), que seguirán necesitando el ID manual.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Medido: solo 4 de los 2206 juegos tienen version_parent en IGDB, porque top_games, search y games_by_names ya excluyen las ediciones. Pero mirar solo eso se queda corto: en la biblioteca de tester_steam hay 3 con version_parent y otros 19 son bundles o ediciones ampliadas que IGDB no enlaza así (Ghost of Tsushima: Director's Cut, Final Fantasy VII Remake Intergrade, Mass Effect Legendary Edition…), más del 6 % de sus 348 juegos. Son notas y reseñas repartidas entre dos fichas.

Cómo lo haría

1. Game.parent (FK a sí mismo) y Game.edition_name, rellenos con fill_game_fields a partir de version_parent y parent_game (comprobar cuántas ediciones ampliadas cubre este último); lo demás, a mano en el admin.
2. En accounts.matching.update_library, si el juego de un título es una edición, sumar sus horas en el padre. ExternalTitle sigue apuntando a la edición, para saber con cuál se jugó.
3. Comando merge_editions --dry-run: pasa LibraryEntry, Review, ListItem, WishlistItem, FavoriteGame, PlaySession y Activity al padre, resolviendo los duplicados de las restricciones únicas, en una transacción y con un resumen.
4. games.views.detail redirige de la ficha de la edición a la del padre con ?edicion=.
5. El selector de edición para contadores y reseñas, después de la fusión, que es lo que arregla lo que se ve. Ojo con no juntar colecciones de varios juegos, como BioShock: The Collection.

Para profundizar

- Mi edición: cada uno elige con qué edición y portada se ve el juego en su biblioteca. Va con el perfil personalizable y con «Colección física con escáner de código de barras».
- Los bundles sin padre: un campo «es edición de» en el admin para los 35 del catálogo, con la corrección compartida, como en ExternalTitle.
- Comparar notas entre ediciones: «la Definitive tiene mejor nota que la original», cuando haya datos.

---

28. Próximos lanzamientos: fechas por plataforma y aviso cuando sale

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Guardar las fechas de salida de cada plataforma y región, enseñar en Inicio los próximos lanzamientos de tu lista de deseos y avisarte el día que sale un juego que quieres, con su precio de salida.

Por qué

Backloggd tiene en su portada «Coming Soon» con los juegos deseados y un filtro de próximos lanzamientos. NextPlay solo guarda first_release_date, así que no sabe cuándo sale un juego en cada consola ni si está en acceso anticipado o cancelado.

Pasos previstos

1. Importar release_dates de IGDB (fecha, plataforma, región y estado: alfa, beta, acceso anticipado o lanzamiento) y game_status (cancelado, retirado…).
2. Ficha: fechas de salida por plataforma, con la de Europa primero, y etiquetas «Acceso anticipado» o «Cancelado».
3. Inicio: «Próximamente» con los juegos de tu lista de deseos que salen en los próximos 90 días y los más esperados según IGDB (hypes).
4. Aviso en la campana el día que sale un juego de tu lista, con su precio en tus tiendas (aprovechando los avisos de precio que ya existen).
5. Actualizar cada noche las fechas de los juegos que aún no han salido.

Notas
IGDB cambia a menudo las fechas de los juegos que no han salido: por eso hay que refrescarlas cada noche.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Solo existe first_release_date, y hay 4 juegos con fecha futura en todo el catálogo: se llena con total_rating_count, y lo que no ha salido casi no tiene votos. Hace falta traerlos aposta. En IGDB hay 1259 juegos futuros con algo de expectación y 340 con 5 o más «hypes» (medido hoy).

Cómo lo haría
1. Comando import_upcoming en nightly: los de first_release_date futura y hypes de 5 o más, y los más deseados de Steam con popularity_primitives de tipo 10 (comprobado).
2. Modelo ReleaseDate (juego, plataforma, región, fecha, texto como «Q1 2027» y estado). Región 1 es Europa y 8, todo el mundo; las dos sirven, con Europa primero.
3. Ficha: fechas por plataforma, con las etiquetas «Acceso anticipado» (estado 3) o «Cancelado» (game_status 6).
4. Inicio: «Próximamente», en la columna derecha, con tus deseados de los próximos 90 días y los más esperados.
5. update_releases cada noche solo para lo que no ha salido, con aviso el día de salida y su precio en tus tiendas.

Para profundizar
- Avisar de los retrasos: al refrescar cada noche se ve si la fecha cambia («X se retrasa a marzo»). Es lo primero que quiere saber quien lo espera.
- Calendario .ics: una dirección privada con token para suscribirse desde Google o Apple y ver tus lanzamientos en tu agenda. Barato: es texto plano.
- Secuelas: si has terminado el anterior de una saga (collections), avisar de la siguiente aunque no esté en tu lista de deseos.
- Precio de salida con perspectiva: «sale a 69,99 €; los anteriores de esta editora bajaron un 40 % en 6 meses», con el historial de ITAD.

---

29. Trofeos de PlayStation emparejados por el ID del juego, no por el nombre

PRIORIDAD MEDIA · EN CURSO · Funcionalidad · propuesta el 29/10/2026

Qué es
Leer los trofeos de cada juego de PlayStation a partir de su ID (CUSA… o PPSA…) en vez de buscar su lista de trofeos por el nombre, para que no se queden sin trofeos los juegos cuyo nombre no coincide con el de su lista.

Por qué
En la primera sincronización de verdad (tester_steam, 29/10/2026) se leyeron los trofeos de 129 de los 176 juegos: 47 se quedaron sin ellos, y entre ellos están los más jugados (Call of Duty: Modern Warfare, 530 h; Rainbow Six Siege, 383 h; Apex Legends, 134 h). Fallan las colecciones, que tienen una lista de trofeos por juego (Uncharted, Ezio Collection, BioShock, Trilogía Mafia, Hasbro), los nombres que no coinciden (el apóstrofo curvo de «Tom Clancy’s», «Genshin Impact 6.º aniversario», «Ghostrunner2», años o regiones entre paréntesis) y otros como Titanfall 2, For Honor o The Last of Us Part II, que parecen llamarse igual que su lista; como no se guarda lo que manda Sony, no se puede saber por qué. PlayStation tiene una petición que da las listas de trofeos de un juego a partir de su ID, que ya se guarda en cada título importado, así que no hace falta adivinarlo por el nombre.

Pasos previstos

1. Al sincronizar, pedir /api/trophy/v1/users/me/titles/trophyTitles?npTitleIds=… con los ID de los juegos importados, de 5 en 5 (unas 36 peticiones para 176 juegos), y apuntar en cada uno sus trofeos conseguidos y totales.
2. Las colecciones con varias listas suman las suyas y cuentan como platino si tienen alguno.
3. Dejar el emparejado por nombre solo para lo que no salga por ID, igualando también los apóstrofos curvos y quitando el año o la región entre paréntesis.
4. Guardar el npCommunicationId de cada lista en el título importado: sirve para pedir los trofeos uno a uno (propuesta «Logros uno a uno») y para no volver a leer los juegos que no han cambiado (lastUpdatedDateTime).
5. Tests con respuestas simuladas de una colección, de un juego con versión de PS4 y de PS5 y de un nombre que no coincide.

Notas

Los lanzadores como «Call of Duty» (el HQ) no tienen trofeos propios y seguirán sin ellos. Aparte, 8 juegos ya tienen trofeos pero no se ven porque no están emparejados con el catálogo (entre ellos el platino de Sniper Ghost Warrior 3): eso se arregla emparejándolos a mano en «Sin emparejar». La petición la documenta psn-api (getUserTrophiesForSpecificTitle): admite como mucho 5 ID a la vez y solo responde de juegos que la cuenta ha jugado, que son justo los que se importan. 29/10/2026: en curso en la rama trofeos-por-id, a petición del usuario. Los trofeos se piden por el ID de cada juego (de 5 en 5 y 6 peticiones a la vez; si Sony rechaza una petición por un ID que no conoce, sus juegos se piden de uno en uno) y las colecciones suman sus listas. El emparejado por nombre queda de reserva, sin signos ni espacios (apóstrofos curvos, «Ghostrunner2») y quitando el año o la región del final solo si hace falta. El npCommunicationId aún no se guarda. 4 tests nuevos (422 en total, todos bien). Falta una sincronización de verdad para ver cuántos de los 47 salen. 01/11/2026: con el emparejado nuevo ya se ven los trofeos de los juegos que estaban sin emparejar, entre ellos el platino de Sniper Ghost Warrior 3. 02/11/2026: con commit y fusionada con main; sigue en curso porque falta medirlo con una sincronización real y guardar el npCommunicationId.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Programado en esta rama, sin commit (accounts/psn.py, accounts/tests.py y README). Lo único que falta es medirlo con una sincronización real, para ver cuántos de los 47 salen. Solo se puede hacer con un NPSSO tuyo. El npCommunicationId aún no se guarda.

Cómo lo haría

1. Tú pegas un NPSSO en tester_steam y yo comparo antes y después: juegos con trofeos y cuáles de los 47 siguen sin ellos.
2. Para los que sigan sin salir, ver en el log qué responde Sony, sin datos de la cuenta: si es otro ID (versión regional, PS4 o PS5) o si no tienen lista.
3. Guardar los npCommunicationId en ImportedTitle (lista JSON, porque una colección tiene varios), con su migración.
4. Commit en su rama, PR, servidor para que lo mires y fusionar, como siempre. Lo demás sigue en «Logros uno a uno».

Para profundizar

- No releer lo que no ha cambiado: lastUpdatedDateTime de cada lista permite saltarse juegos. Menos peticiones y menos riesgo de que Sony corte.
- Usar los trofeos para emparejar: quedan 7 títulos de PlayStation sin juego tras «Emparejar los juegos por nombre sin contar signos: recuperar horas perdidas», y el nombre de su lista de trofeos es otra pista, a veces más limpia que la de la tienda.
- PS3 y Vita: trophy_titles también trae esos juegos, que la lista de juegos no da porque solo pide PS4 y PS5. Importarlos como títulos sin horas pero con trofeos y fechas rescata historia que hoy se pierde.

- Guardar los grados (bronce, plata, oro, platino): dan una puntuación tipo PSN para el perfil y distinguen un 100 % de un platino.

30. ¿Cuánta gente lo termina? Tasa de finalización con los logros de Steam

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Mostrar en la ficha qué parte de los jugadores llega al final del juego («lo termina el 31 % de quienes lo empiezan»), sacado del porcentaje mundial de jugadores de Steam que tienen el logro de los créditos.

Por qué

Backloggd solo sabe lo que apunta su gente. Valve publica sin clave el porcentaje de jugadores que tiene cada logro, así que con el steam_app_id que ya guarda NextPlay se puede dar un dato real de millones de jugadores que funciona desde el primer día, aunque haya pocos usuarios. Es lo que más pesa antes de empezar un juego largo y combina con «Duración estimada de los juegos».

Pasos previstos

1. Leer GetGlobalAchievementPercentagesForApp (sin clave) y GetSchemaForGame con l=spanish (con la STEAM_API_KEY que ya hay) de los juegos con ID de Steam, en la tarea nocturna y como mucho una vez por semana.
2. Elegir el logro del final: primero por reglas (logros ocultos con palabras como «créditos», «final» o «complete the story») y después que un administrador o la comunidad lo confirme o lo corrija, igual que con los juegos importados (emparejado por, bloqueado).
3. Dividir entre el porcentaje del primer logro de la historia para dar «de los que lo empiezan, lo termina el X %», porque no todos los que lo tienen llegan a abrirlo.
4. En la ficha: «Lo termina el 31 % · el logro más difícil lo tiene el 4 %», junto a la nota y la duración. En el catálogo y el pendiente, poder ordenar por los que más gente termina.
5. Con los logros del usuario (propuesta «Importar logros y trofeos»), proponer marcarlo como Completado cuando consigue el logro del final, con su fecha.

Notas

Los juegos sin final (multijugador, sandbox) o sin logro de créditos se quedan sin dato. Solo cuenta a jugadores de Steam, pero es una muestra enorme.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar, con las APIs comprobadas: los porcentajes salen sin clave y el esquema, en español (Hades: 49 logros, del 81,8 % al 5,9 %). Lo difícil es saber cuál es el logro del final, y las reglas por palabras no bastan: probadas con los 100 juegos más populares con ID de Steam, de 76 con logros solo 40 dan algún candidato y solo 16 dan uno solo, con muchos falsos (en Hades, «To Charon's Credit» por la palabra credit). GTA V, Elden Ring o Skyrim no dan ninguno.

Cómo lo haría

1. Comando fetch_completion en nightly, semanal y con pausas: guarda los porcentajes en Achievement (los mismos modelos de «Logros uno a uno») o, si esto va antes, en un JSON en Game.
2. Elegir el logro con un modelo de lenguaje y revisarlo a mano: pasarle a Claude nombre, descripción y porcentaje de cada logro para que marque el del final, con su confianza. Las reglas por palabras solo sirven de pista. Se guarda Game.final_achievement con su origen (auto, usuario o administrador), como ExternalTitle.Source.

3. Revisión en el admin, con los candidatos y confirmar con un clic. Empezar por los 200 más populares: son los que más se miran.
4. Tasa relativa: final entre el logro más común, para responder «de quienes lo empiezan, lo termina el X %».
5. Ficha: una línea junto a la nota y la duración, diciendo que sale de los logros de Steam. Sin dato, no se enseña nada: nada de inventarse un 0 %.

Para profundizar

- Embudo de la historia: los logros ordenados por porcentaje dibujan dónde se deja el juego («la mitad se queda antes del acto 2»). Se ve de un vistazo y es información nueva para quien duda si empezarlo.
- Que corrijan quienes lo han terminado: «¿Este es el logro del final?», con un clic y compartido por todos, como las correcciones del emparejado.
- Comparar y ordenar: «lo termina más gente que el 70 % de los RPG» y, en el pendiente, ordenar por los que más gente termina.
- Reconocimiento personal: al conseguirlo, «solo el 12 % llega hasta aquí», con una tarjeta para compartir.

---

31. Bloquear, silenciar y quitar seguidores

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 25/10/2026

Qué es

Poder bloquear a un usuario (no te sigue, no ve lo tuyo, no comenta ni da «me gusta» a tus reseñas), silenciarlo (deja de salir en tu feed sin que se entere), quitar a alguien de tus seguidores y denunciar un perfil.

Por qué

Con seguidores y comentarios llegan el acoso y el spam. Las denuncias ya existen, pero no hay forma de cortar el contacto con alguien. Hace falta antes de abrir NextPlay al público.

Pasos previstos

1. Modelo Block (quien bloquea y bloqueado); al bloquear se borran los follows en los dos sentidos.
2. El bloqueado no puede seguirte, comentar ni dar «me gusta» a lo tuyo, y no ve tu perfil ni tu actividad; tú tampoco ves la suya.
3. Silenciar: su actividad deja de salir en tu feed y sus avisos no te llegan, sin que lo sepa.
4. «Quitar de mis seguidores» en tu lista de seguidores.
5. Denunciar un perfil con el modelo Report, que ya es genérico.
6. Lista de usuarios bloqueados y silenciados en Editar perfil, para deshacerlo.

Notas

Depende de «Seguir a otros jugadores y feed de actividad». Puede ir junto a su fase 1 si se quiere abrir pronto. 04/11/2026: todos los avisos pasan ya por accounts.notices.notify (ver «Un solo sitio para crear los avisos, con preferencias por tipo»), así que el bloqueo y el silencio solo tienen que mirarse allí para que no lleguen avisos de quien está bloqueado o silenciado.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Report ya es genérico y las reglas de visibilidad están casi centralizadas en Profile.visible_to y visible_filter, salvo lists/models.py y lists/views.py, que repiten user__profile__is_public: hay que llevarlos a la misma regla antes.

Cómo lo haría

1. Modelos Block y Mute en social, y un helper blocked_ids(user) con caché por petición.
2. Ampliar visible_to y visible_filter para excluir a los bloqueados en los dos sentidos, y que las listas usen la misma regla.
3. Una función can_interact(actor, target) en follow, los «me gusta», los comentarios y las invitaciones de reseñas, publicaciones y listas. Con «Un solo sitio para crear los avisos», los avisos se filtran solos.
4. Bloquear borra los Follow en los dos sentidos y saca de las colaboraciones de listas; en los retos sigue participando, pero sale oculto para quien bloquea.
5. Silenciar: excluir en feed_context y en los avisos, sin que se entere.
6. Interfaz: menú en el perfil y en social/_person.html, «Quitar de mis seguidores», denunciar el perfil (una clave más en REPORT_TARGETS) y la lista para deshacerlo en Editar perfil. Tests con una matriz de quién ve qué.

Para profundizar

- Silenciar juegos: no ver en el feed nada de un juego hasta terminarlo («silenciar Elden Ring hasta completarlo»). Propio de una web de juegos, y casi gratis con los estados y las marcas de spoiler.
- Quién puede interactuar: una preferencia por usuario para que solo comenten sus seguidores, o nadie, sin llegar a bloquear.
- Límites contra el spam: comentarios, invitaciones y denuncias por hora, con la caché de Django y sin librerías.
- Moderación ligera: ocultar de forma automática lo que tenga 3 denuncias distintas hasta que un administrador lo revise; el admin de denuncias ya está hecho.

---

32. Perfil solo para seguidores, con solicitudes

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 25/10/2026

Qué es

Un nivel de privacidad entre público y privado: tu perfil, tu biblioteca, tus reseñas y tu actividad solo los ven las personas que aceptas como seguidores.

Por qué

Ahora un perfil privado no lo ve nadie, así que su dueño no puede usar nada social. Con este nivel puede compartirlo con sus amigos sin enseñárselo a todo el mundo.

Pasos previstos

1. Cambiar is_public por un nivel de visibilidad (público, solo seguidores o privado), con una migración que respete lo que tenía cada uno.
2. Follow con estado (pendiente o aceptado): en los perfiles «solo seguidores», seguir manda una solicitud.
3. Aviso de cada solicitud y página para aceptarlas o rechazarlas.
4. Contar a los seguidores aceptados en todas las comprobaciones de visibilidad (perfil, reseñas, comentarios, feed y ficha); si en la fase 1 de seguir se dejaron en un solo sitio, es un cambio pequeño.
5. Tests de visibilidad con los tres niveles.

Notas

Depende de «Seguir a otros jugadores y feed de actividad». Si el usuario lo quiere desde el principio, entra en su fase 1.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Follow no tiene estado. is_public de los perfiles se mira en Profile.visible_to y visible_filter, en lists/models.py y lists/views.py, y en tres plantillas (accounts/profile.html, social/_person.html y lists/detail.html): pocos sitios, pero «solo seguidores» añade una consulta por cada comprobación que hoy es un simple booleano.

Cómo lo haría

1. Profile.visibility (público, solo seguidores o privado), con una migración que respete lo que tiene cada uno, y dejar is_public como propiedad mientras se migra el resto.
2. Follow.accepted_at, relleno al migrar: en un perfil «solo seguidores», seguir crea una solicitud sin aceptar. Todos los contadores y el feed cuentan solo las aceptadas.
3. visible_to mira el Follow aceptado y visible_filter usa Exists con OuterRef, para no duplicar filas al cruzar con seguidores. Guardar el resultado por petición: una página con 20 tarjetas no puede hacer 20 consultas.
4. Aviso de cada solicitud y página para aceptar o rechazar. Al pasar a este nivel, quien ya te seguía se queda, con la opción de revisarlos.
5. Tests con una matriz: tres niveles por cuatro tipos de visitante (dueño, seguidor aceptado, pendiente y desconocido) en perfil, reseñas, feed, listas, retos y ficha.

Para profundizar

- Privacidad por partes: quién ve tus horas, tu diario, tu lista de deseos y tus cuentas de plataforma, por separado. Más útil que todo o nada, y es parte del perfil a medida.
- Lista de deseos para seguidores: hoy es siempre privada. Enseñársela a tus amigos da ideas de regalo sin hacerla pública.
- «Ver como»: un botón en tu perfil para verlo como un desconocido o como un seguidor. La vista previa de Editar perfil ya pinta el perfil con AnonymousUser.
- Aceptar solos a tus amigos de Steam o Xbox, con los datos de «Encontrar a tus amigos de Steam y Xbox en NextPlay».

---

33. ¿A qué jugamos? Juegos en común con tus amigos

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Elegir de 2 a 4 personas y ver los juegos multijugador que tenéis todos (en cualquier plataforma), cuáles son cooperativos y cuánto le cuesta hoy a quien le falte.

Por qué

Es la pregunta de cada grupo de amigos y Backloggd no puede contestarla porque no sabe qué juegos tiene cada uno. NextPlay ya importa las bibliotecas de Steam, PlayStation y Xbox y tiene los precios de cada tienda. Además, el resultado se comparte en el grupo de Discord o de WhatsApp, así que trae gente nueva.

Pasos previstos

1. Guardar también los juegos de Steam que se tienen sin abrir (ImportedTitle con 0 minutos) aunque no entren en la biblioteca, para saber todo lo que tiene cada uno.
2. Importar de IGDB multiplayer_modes (online, local, cooperativo, pantalla partida y jugadores máximos), que ya está en «Ficha del juego más completa».
3. Página «¿A qué jugamos?»: eliges amigos (o escribes sus usuarios) y salen tres grupos: lo tenéis todos, le falta a alguien (con su mejor precio en sus tiendas) y gratuitos.

4. Tener en cuenta la plataforma: si uno lo tiene en PS5 y otro en PC, solo vale si hay juego cruzado (la categoría de Steam de multijugador multiplataforma, cuando se sepa).
5. Enlace para compartir el resultado con la carátula de cada juego.

Notas
Encaja con «Seguir a otros jugadores y feed de actividad» y «Encontrar a tus amigos de Steam y Xbox en NextPlay», pero puede empezar con nombres de usuario. Con perfiles privados, solo si su dueño lo acepta.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Sin empezar. Hay un hueco de datos: ImportedTitle solo guarda lo jugado salvo que se pida «unplayed» (tester_steam tiene 267 juegos en Steam y solo 135 con última partida), y PlayStation y Xbox solo listan lo abierto. Los modos multijugador de IGDB están en 838 de 2206 juegos.

Cómo lo haría
1. En steam.import_library, guardar siempre en save_titles los juegos sin abrir, y aplicar el filtro de horas solo al pasarlos a la biblioteca (update_library): se sabe lo que cada uno tiene sin llenarle el pendiente.
2. Datos del juego: multiplayer_modes de IGDB (cooperativo en línea y local, pantalla partida y jugadores máximos) y las categorías de Steam, en la misma llamada de localize_games. Comprobado: 1 multijugador, 9 cooperativo y 38 cooperativo en línea.
3. games/together.py: lo que tiene cada uno cruzando LibraryEntry con ImportedTitle, y tres grupos: lo tenéis todos, le falta a alguien (con game.cheapest en sus tiendas) y gratuitos.
4. Página /a-que-jugamos/?con=ana,luis con autocompletar de la gente que sigues. Solo entra quien tiene el perfil visible para ti; un perfil privado tendría que aceptarlo.
5. Ajuste por plataforma: si uno lo tiene en PS5 y otro en PC, decirlo y no darlo por bueno mientras no se sepa si hay juego cruzado.

Para profundizar
- Con que uno lo tenga basta: Remote Play Juntos y Préstamo familiar (la categoría 62 está comprobada; la de Remote Play Juntos hay que comprobarla). Saber qué se puede jugar sin que todos paguen cambia la respuesta.
- Votar: un enlace para el grupo de WhatsApp o Discord, donde cada uno marca qué le apetece y sale un ganador.
- Que quepa en una noche: ordenar por lo que nadie ha terminado y por duración («los tres lo tenéis, 8 h de campaña cooperativa»).
- Rebajas del grupo: avisar cuando baja de precio un juego al que solo le falta a una persona («solo le falta a Luis y hoy está a 4,99 €»).

---

34. Importar Nintendo Switch con capturas de la app Nintendo Store

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es
Subir capturas de la «Actividad de juego» de la app Nintendo Store del móvil y que NextPlay lea los juegos, la primera y la última partida y el tiempo de cada día, para importarlos como los de Steam, PlayStation y Xbox.

Por qué
Switch es la única gran consola que NextPlay no puede importar (Nintendo no tiene API) y en España es de las que más se juega. La app Nintendo Store enseña la actividad por juego y por día, pero no deja copiarla ni exportarla, así que leer las capturas es la única vía (Seth Larson lo hizo con OCR en diciembre de 2025). Backloggd no importa ninguna plataforma.

Pasos previstos

1. Página para subir varias capturas desde el móvil; las imágenes se borran después de leerlas.
2. Leer cada captura con un modelo de visión (o con OCR, como EasyOCR): nombre del juego, primera y última partida y las sesiones de cada día.
3. Emparejar con match_by_name y las plataformas «Nintendo Switch» y «Nintendo Switch 2», y guardarlo como ExternalTitle de una plataforma nueva para que las correcciones sirvan a todos.
4. Pantalla de revisión antes de guardar: lo leído, el juego emparejado y un buscador para corregirlo.
5. Las sesiones por día van directas al «Diario de partidas que se escribe solo»; volver a subir capturas solo añade lo nuevo.

Notas

Con OCR fallan sobre todo los títulos que salen como logo; un modelo de visión (p. ej. Claude) o comparar con los juegos de Switch del catálogo lo resuelve. La app no da tiempos de menos de 15 minutos («unos minutos»). Completa a «Colección física con escáner de código de barras», que añade los juegos pero no las horas.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Requiere una plataforma nueva en ExternalTitle.Platform y en las horas de LibraryEntry, así que conviene hacerla después de «Horas por plataforma sin una columna por tienda». La incógnita es la precisión de la lectura.

Cómo lo haría

1. Prueba antes de construir nada: 10 capturas reales tuyas, un modelo de visión de Claude y comparar lo leído con la realidad. Si no da, no sigo.
2. Página de subida; las imágenes se procesan en memoria y no se guardan nunca: es la promesa de privacidad más sencilla de explicar.
3. Salida estructurada por captura: título, primera y última partida y [{día, minutos}], pasando como pista la lista de juegos de Switch del catálogo.
4. Pantalla de revisión como accounts/imported_titles.html: lo leído, el juego propuesto y un buscador para corregirlo. Nada se guarda sin confirmar.
5. ExternalTitle.Platform.NINTENDO y PlaySession por día exacto, que es mejor dato que el de Steam. Volver a subir capturas solo añade lo nuevo, comparando juego y día.

Para profundizar

- Control parental: la app Nintendo Switch Parental Controls da tiempo por juego y día, y nxapi la lee sin capturas. Es no oficial, con el mismo riesgo que PlayStation; merece una prueba si subir capturas resulta pesado.
- Controlar el gasto: un límite de capturas por usuario y día y la tarea en segundo plano, porque cada lectura cuesta dinero.
- No volver a preguntar: lo corregido alimenta ExternalTitle (nombre leído y juego), y a la siguiente no hace falta adivinarlo.
- Reutilizar el lector: capturas de cualquier plataforma sin API (Epic, Battle.net, móvil) y fotos de la estantería para «Colección física con escáner de código de barras».

---

35. Duración estimada de los juegos

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Mostrar en la ficha cuánto se tarda en terminar un juego (historia, historia + extras, 100 %) y cuánto le falta al usuario según sus horas.

Por qué
Ayuda a decidir qué jugar del pendiente y combina bien con las horas importadas y con las recomendaciones.

Pasos previstos
1. Importar los tiempos de IGDB (endpoint game_time_to_beats), que es oficial y ya tenemos credenciales.
2. Calcular con las horas de la comunidad de NextPlay los tiempos de quienes lo marcaron como Completado.
3. Filtro del catálogo y del pendiente por duración («menos de 10 horas»).

Notas
Ya estaba planeada antes de proponerla.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Sin empezar. Medido en IGDB: 1690 de 2206 juegos tienen fila en game_time_to_beats y 1523 tienen tiempo normal, pero solo 540 con 5 respuestas o más, así que hay que enseñar de cuánta gente sale. Vienen en segundos: hastily, normally y completely. Y hay valores absurdos: Content Warning sale con 21.614 h (3 respuestas), Ark con 1486 h y GTA V Enhanced con 626 h, de modo que sumar sin más el pendiente de tester_steam da 25.278 h. Solo los 66 juegos de 100 h o menos suman 1346 h, con una mediana de 17 h.

Cómo lo haría
1. Campos en Game en minutos (ttb_quick, ttb_normal, ttb_full) y ttb_count, con índice en ttb_normal.
2. Comando fill_time_to_beat por lotes de 500, semanal en nightly, y también al importar juegos nuevos.
3. Ficha: «Unas 12,5 h · según 11 jugadores de IGDB» y, si tienes horas y no lo has terminado, «Te faltan unas 4 h». Con menos de 5 respuestas, «dato orientativo», y fuera de los totales: por eso salen las cifras absurdas.
4. Dato propio: mediana de horas de quienes lo tienen Completado, a partir de 5 personas, calculada de noche y enseñada al lado.
5. Filtro por duración en el catálogo, la biblioteca y la lista de deseos (ver «Catálogo con filtros y orden al nivel de Backloggd (y más)»).

Para profundizar
- Proponer Completado: horas por encima de la duración normal y estado Jugado dan «¿Lo terminaste?» con un clic, sin los juegos de servicio, que siempre la superan. Es la forma más barata de llenar una columna casi vacía (1 de 348).
- Tu fecha de fin: con tus horas por semana (PlaySession), «a este ritmo lo terminas el 12 de diciembre».
- Tu pendiente en horas, sin los valores raros (en tester_steam, 1346 h en 66 juegos): «a tu ritmo, N años». Una cifra que cambia cómo miras el pendiente, en «Retos del año y control del pendiente».
- Precio por hora desde el primer día, sin comunidad: mejor precio entre la duración normal, para 1523 juegos (ver «Cuándo se abandona, si mejora con las horas y cuánto sale la hora»).

---

36. Recomendaciones «Qué jugar después»
PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es
Sugerir juegos del catálogo y de la propia lista de pendientes según lo que el usuario ha valorado alto y más horas ha jugado.

Por qué

Encaja con el nombre de la app (NextPlay) y aprovecha el backlog importado, que en cuentas reales suele tener cientos de juegos sin empezar.

Pasos previstos

1. Primera versión simple: puntuar juegos por géneros y plataformas en común con los mejor valorados del usuario, más nota de la crítica.
2. Bloque «Elige de tu pendiente» que escoge entre los juegos en Pendiente con un botón de «otro».
3. Más adelante, filtrado colaborativo: «a quien le gustó X también le gustó Y» con las valoraciones de la comunidad.
4. Excluir los que ya están completados o abandonados.

Notas

IGDB también devuelve similar_games para cada juego; importarlo daría recomendaciones decentes sin algoritmo propio. Ya estaba planeada antes de proponerla.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. similar_games está en 2196 de 2206 juegos, temas en 2180 y perspectivas en 2178, pero no se guarda ninguno. Con 2 notas en 348 juegos, la señal fuerte tienen que ser las horas, no las estrellas.

Cómo lo haría

1. Guardar similar_games, themes y player_perspectives con fill_game_fields.
2. library/taste.py: un perfil de gustos con peso por juego según horas relativas a su duración (40 h en uno de 10 h dice más que 40 h en uno de 100 h), más la nota si la hay; lo abandonado resta.
3. Candidatos: similar_games de tus más jugados y el resto del catálogo, puntuados por coincidencia, nota y popularidad; fuera lo que ya has jugado.
4. Dos bloques: «Elige de tu pendiente», con un botón «otro» por HTMX, y «Para descubrir». Cada uno dice por qué («porque jugaste 80 h a Hades»).
5. Guardar el resultado un día por usuario y recalcular tras sincronizar. «No me interesa» se guarda y no vuelve a salir.

Para profundizar

- En el momento justo: al pasar un juego a Completado, «¿Y ahora?» con tres de tu pendiente parecidos. Es lo que da nombre a NextPlay, y es cuando de verdad se decide.
- Filtros del momento: que puedas jugar ya y esta noche (menos de 3 h, en castellano, en Deck, en Game Pass o ya en tu biblioteca).
- En rebajas: tu lista de deseos y las recomendaciones ordenadas por afinidad y precio: qué te conviene comprar a ti, no qué está más rebajado.
- Colaborativo cuando haya gente: horas de usuarios parecidos, sin modelos pesados. Antes, «La gente que sigues en la ficha y en los perfiles» da algo muy parecido con nombres.

---

37. Estadísticas del perfil y resumen anual

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Una pestaña de estadísticas en el perfil (horas totales por plataforma, géneros favoritos, juegos completados por año, nota media) y un «Wrapped» anual compartible.

Por qué

Con las horas importadas de Steam, PlayStation y Xbox hay datos de sobra y casi ninguna web los junta todos. Es muy vistoso y anima a vincular cuentas.

Pasos previstos

1. Consultas agregadas sobre LibraryEntry: minutos por plataforma, por género y por estado.
2. Gráficas sencillas en la plantilla (SVG o una librería ligera).
3. Página de resumen del año con los juegos más jugados y una imagen para compartir.
4. Respetar el perfil privado.

Notas

Para «completados por año» haría falta guardar la fecha en que un juego pasa a Completado; ahora solo hay updated_at. Ya estaba planeada antes de proponerla.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Hay cifras en la cabecera (_profile_stats) y los géneros más jugados, sin pestaña ni resumen. Su nota está vieja: completed_at existe desde el 26/10. Los datos dan para mucho: tester_steam lleva 3905 h en Steam y 6975 h en PlayStation, con 37 juegos en las dos.

Cómo lo haría

1. Pestaña ?vista=estadisticas con library/stats.py y agregados SQL: horas por plataforma y por género, estados, juegos por año de lanzamiento y por año jugado.
2. Gráficas SVG desde el servidor, como history.chart, con texto alternativo y sin librerías.
3. Resumen anual en /u/<usuario>/resumen/2026/: más jugados, horas, géneros, logros y reseñas, a partir de PlaySession y, para lo anterior, de los logros y la última partida.
4. Imagen para compartir con la misma pieza de «Vistas previas al compartir y tarjeta para incrustar», en 1080 × 1920 para historias y en apaisado.
5. Respetar el perfil privado y dejar ocultar cifras concretas.

Para profundizar

- Cifras que solo salen de tener precios y horas juntos: cuánto vale tu pendiente, cuánto te sale la hora y cuánto has ahorrado comprando en rebajas si apuntas lo pagado.
- Balance del pendiente: juegos que entran frente a los que terminas cada mes. Un dato incómodo y muy compartible.
- Años pasados: con los logros, el resumen de 2025 ya se puede hacer aunque el diario empezara el 25/10/2026 (hay logros en 36 juegos de ese año en tester_steam).
- Cada cifra enlaza a sus juegos: «Terror: 40 h» lleva a tu biblioteca filtrada. Y compararte con la gente que sigues o con la media de NextPlay.
- Títulos que salen de los datos, para el perfil: «Cazador de platinos», «Terminador de indies»… Se ganan solos y no se pueden falsear, porque salen de horas y logros verificados.

---

38. Recuerdos: «un día como hoy» con tus fechas reales

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 01/11/2026

Qué es

Un bloque en el inicio con lo que pasó tal día como hoy en otros años: el juego que terminaste, un logro que conseguiste o la reseña que escribiste, y los aniversarios de tus juegos favoritos.

Por qué

NextPlay guarda fechas que casi nunca vuelve a enseñar: completed_at, Review.published_at, ImportedTitle.first_played y, con «Logros uno a uno», las de cada logro (en tester_steam, desde 2022). Da un motivo para abrir la web cada día sin pedir nada nuevo al jugador, y first_release_date da aniversarios desde hoy mismo.

Pasos previstos

1. library/memories.py: memories(user, day) junta lo de ese día y ese mes de otros años, sin tablas nuevas.
2. Bloque en la columna derecha de la portada, oculto si ese día no hay nada, y solo para su dueño.
3. «Hace 10 años salió…» de tus favoritos y tu biblioteca, con first_release_date.
4. «Compartir este recuerdo» abre la caja de publicar con el juego ya puesto.

Notas

Sale de la revisión del 01/11/2026. Hoy solo daría aniversarios y pocas fechas; con los logros guardados hay material de varios años.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. No necesita datos nuevos para empezar; gana mucho con «Logros uno a uno».

Para profundizar

- Útil además de nostálgico: «Hace un año que no lo tocas: ¿sigues?», enlazando con el repaso de la biblioteca.
- Fechas redondas: un año de tu primer platino o de tu primera reseña, con un aviso opcional.

---

39. Saber qué juegos están en Game Pass

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Marcar con una insignia los juegos que están en Game Pass (consola y PC), avisar de los que salen pronto y poder filtrar el pendiente por «está en Game Pass».

Por qué

Los IDs de producto de Microsoft ya se guardan (xbox_product_ids) para los precios de Xbox, así que cruzarlos con el catálogo de Game Pass cuesta poco. A quien paga la suscripción le dice qué puede jugar sin comprar nada.

Pasos previstos

1. Leer cada día las colecciones del catálogo público de Game Pass (catalog.gamepass.com), que devuelven IDs de producto de Microsoft.
2. Guardar en cada juego si está en Game Pass de consola, de PC o en EA Play, y si está en «sale pronto».
3. Insignia en las tarjetas y en la ficha, y filtro en el catálogo y en la biblioteca.
4. Aviso en el perfil: «3 juegos de tu pendiente salen de Game Pass este mes».

Notas

Es una API sin documentar, como la de la tienda de Xbox que ya se usa, y podría cambiar. PS Plus no tiene ninguna fuente fiable, así que quedaría fuera.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Comprobado: catalog.gamepass.com/sigls/v2 responde con 619 productos de consola, 513 de PC y 96 de EA Play, más las listas «Agregados recientemente» (31) y «Desaparecerá pronto» (1 hoy), y 261 de los de consola ya están en el catálogo local, por xbox_product_ids. En tester_steam, 26 de los 90 deseados están en Game Pass (696,05 € a precios del 24/10), y 10 de los 120 del pendiente.

Cómo lo haría

1. games/gamepass.py con los identificadores de cada lista, y GamePassEntry (juego, plan, desde y hasta) en vez de un simple booleano, para saber cuándo entró y cuándo salió.
2. Comando update_gamepass en nightly: lo que aparece abre una entrada y lo que desaparece la cierra.
3. Insignia en games/_card.html y en la ficha, y filtro ?gamepass= en el catálogo y en la biblioteca.
4. Preferencia «tengo Game Pass» en Editar perfil, para no molestar a quien no lo tiene. Con ella, aviso de lo de tu pendiente y tu lista de deseos que entra o se va.
5. Como la API no está documentada, un test que falle con claridad si cambia el formato, y que un fallo no borre las entradas abiertas.

Para profundizar

- ¿Te sale a cuenta?: «Este mes, 14 de tu pendiente están en Game Pass: 180 h, y comprarlos costaría 230 €». Se calcula con best_price y lo responde a la medida de cada uno.
- No lo compres: en la lista de deseos y en «Dónde comprarlo», «Está en Game Pass» al lado del precio.
- Historial propio: guardando entradas y salidas, en un año se podrá decir si un juego suele volver. Lo que no se guarda hoy no se recupera.
- PS Plus no tiene fuente pública, pero según psn-api la lista de juegos trae service (por ejemplo, ps_plus) en cada título: al menos se puede marcar lo que tú juegas con la suscripción. Comprobar en la próxima sincronización.

---

40. Vistas previas al compartir y tarjeta para incrustar

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Que al pegar un enlace de NextPlay en WhatsApp, Discord o X salga una tarjeta con la carátula, y una imagen de tu perfil que se actualiza sola para ponerla en foros, Twitch o GitHub.

Por qué

Ahora la cabecera solo tiene título y descripción, así que los enlaces compartidos salen sin imagen. Cada enlace compartido es publicidad gratis.

Pasos previstos

1. Etiquetas Open Graph y de Twitter en base.html, con la carátula en las fichas y el avatar en los perfiles.
2. Vista /u/<usuario>/tarjeta.svg con lo que juegas ahora, las horas del año y el último juego completado.
3. Respetar el perfil privado (tarjeta genérica o 404).
4. Botón «Compartir» en la ficha, las reseñas y el perfil.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar: base.html solo lleva título y descripción. Hay un fallo en su plan: Discord, WhatsApp y X no aceptan SVG en og:image, y muchos foros tampoco incrustan SVG externos. La tarjeta tiene que existir en PNG.

Cómo lo haría

1. {% block og %} en base.html con og:title, og:description, og:url absoluta (SITE_URL) y twitter:card summary_large_image.
2. Imagen de cada página: en la ficha, el arte de games.media a 1280 px; en una reseña, portada, nota y nombre; en un perfil, su tarjeta.
3. accounts/cards.py con Pillow, que ya es dependencia: composición de 1200 × 630 con portadas, nombre y cifras, y una fuente propia en static. Se guarda en media con un nombre según su contenido y se rehace si cambia.
4. /u/<usuario>/tarjeta.png pública (404 si el perfil es privado) con Cache-Control de una hora, y parámetros para elegir qué enseña.
5. Botón «Compartir» con navigator.share y, si no existe, el «Copiar enlace» que ya hay.

Para profundizar

- Una sola función de imagen con varias plantillas: tier list, reto conseguido, resumen anual, mosaico de lista y «¿A qué jugamos?». Es la pieza que más propuestas aprovechan.
- Tarjeta para firmas y README, con ?muestra=jugando o ?tema=oscuro: «Ahora juego a…» se actualiza sola en cualquier foro o perfil de GitHub.
- Que las fichas salgan en Google: JSON-LD de tipo VideoGame y sitemap.xml con django.contrib.sitemaps. Es poco trabajo y las fichas en castellano tienen poca competencia.
- Mosaico propio para las listas: un PNG con sus cuatro portadas, en lugar de una imagen genérica al compartir una lista.
- RSS de las reseñas y de la actividad de cada perfil, con django.contrib.syndication: se conecta a Discord o a un blog sin escribir ninguna API.

---

41. Entrar con Steam o con cuenta Microsoft

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Poder crear la cuenta e iniciar sesión con Steam o con Microsoft, reutilizando los inicios de sesión que ya se usan para vincular esas cuentas.

Por qué

Registrarse con correo y contraseña es la parte que más gente abandona, y los flujos de Steam (OpenID) y Microsoft (OAuth) ya están programados.

Pasos previstos

1. Si la cuenta de Steam o Xbox ya está vinculada a un usuario, iniciar su sesión.
2. Si no, crear el usuario y vincularla en el mismo paso.
3. Steam no da el correo: pedirlo después del primer inicio de sesión.
4. Tests del flujo completo.

Notas

No unir cuentas automáticamente por correo sin verificarlo, porque permitiría entrar en la cuenta de otro.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. steam.verify_login y xbox.xbox_tokens sirven, pero steam_callback y xbox_callback llevan login_required. El obstáculo real: User.email es obligatorio y único, y Steam no da correo.

Cómo lo haría

1. Vistas steam_login y steam_login_callback sin login_required, con su propio state, reutilizando verify_login.
2. Si SteamAccount ya existe, login(request, account.user) y listo.
3. Si no, no crear aún la cuenta: guardar el SteamID ya verificado en la sesión y pedir correo y nombre (propuesto con su persona de Steam). Crear el usuario con set_unusable_password al confirmar, e importar sus juegos.
4. Microsoft igual, añadiendo los permisos openid y email a SCOPE, porque XboxLive.signin no da el correo.
5. No dejar quitar la última forma de entrar: si no tiene contraseña, desvincular pide poner una antes. Tests de todo el flujo, también de un SteamID ya vinculado a otro.

Para profundizar

- Verificar el correo, que hoy no se comprueba en ningún caso: hace falta antes de unir cuentas y para que los correos de avisos no acaben en un buzón ajeno.
- Unir sin riesgos: si alguien con cuenta entra con Steam, proponer vincular tras poner su contraseña. Nunca por coincidir el correo sin verificar, como ya dice su nota.
- Entrar sin contraseña con un enlace por correo, reutilizando los tokens de recuperar la contraseña.
- django-allauth daría Google, Discord y demás, pero cambia el sistema entero: solo merece la pena si se piden; los dos flujos actuales ya están hechos a mano y probados.

---

42. Juegos gratis para quedártelos

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Una sección con los juegos que se pueden reclamar gratis ahora mismo (el semanal de Epic, regalos de GOG o Steam) y cuándo dejan de estarlo.

Por qué

Es contenido que se renueva solo cada semana y hace volver a la web. Epic tiene un endpoint público de promociones y IsThereAnyDeal marca las ofertas al 100 %.

Pasos previstos

1. Leer las promociones gratuitas de Epic para España y las ofertas al 100 % de IsThereAnyDeal.
2. Emparejarlas con el catálogo (Epic por nombre; IsThereAnyDeal ya trae el juego).
3. Bloque en el catálogo con la cuenta atrás de cada una.
4. Si existe la lista de deseos, avisar cuando un juego deseado se regala.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Comprobado: la API de Epic responde para España (hoy 1 gratis y 6 próximas promociones) e IsThereAnyDeal, en /deals/v2 ordenado por rebaja, da lo que está al 100 %, mezclando juegos, DLC y paquetes (el campo type permite separarlos).

Cómo lo haría

1. games/freebies.py: Epic (promotionalOffers con precio 0 y sus fechas) e IsThereAnyDeal (solo type=game), en un modelo Freebie con juego opcional, tienda, enlace, inicio y fin.
2. Emparejar: Epic por nombre con accounts.matching.match_by_name; IsThereAnyDeal da el ID de su juego, que se cruza con Game.itad_ids. Si no está en el catálogo, se importa de IGDB.
3. Comando en nightly y, para Epic, otro pase los jueves por la tarde, cuando cambia su juego de la semana.
4. Bloque en el catálogo y en el inicio con cuenta atrás, filtrado por las tiendas de Profile.shops y por una nota mínima, para no llenarlo de relleno.
5. Aviso si estaba en tu lista de deseos y no lo tienes en la biblioteca, usando las mismas reglas que wishlist.check_alerts.

Para profundizar

- Qué has reclamado: Epic no tiene API de biblioteca, así que «Lo he reclamado» con un clic lo pasa a tu Pendiente con la fecha y la tienda. Con los años nadie recuerda dónde tiene cada juego.
- Próximos gratis: upcomingPromotionalOffers de Epic ya lo dice (comprobado). «La semana que viene es gratis: no lo compres» es un aviso que ahorra dinero.
- Historial: «Epic lo regaló en 2024» junto al precio. Si ya se ha regalado, puede volver a pasar, y eso es parte del veredicto de comprar o esperar.
- Resumen semanal por correo, opcional: una sola cosa bien hecha que hace volver sin ser pesado.

---

43. Steam Deck, parches y «jugando ahora»

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Con el ID de Steam y la clave que ya hay: compatibilidad con Steam Deck y Linux en la ficha, las últimas noticias y parches de los juegos que estás jugando y un indicador «jugando ahora» en el perfil.

Por qué

Son tres datos que Steam da casi gratis y que ahora no se aprovechan. La importación ya llama a GetPlayerSummaries, que también dice a qué está jugando el usuario en ese momento.

Pasos previstos

1. Steam Deck: leer el estado de verificación de Valve (verificado, jugable, no compatible) y la nota de ProtonDB para Linux; filtro «jugable en Deck» en el pendiente.
2. Parches: bloque «Novedades de tus juegos» con las últimas noticias de los juegos en Jugando (GetNewsForApp, sin clave).
3. Jugando ahora: al abrir un perfil con Steam vinculado, mostrar «Jugando a X ahora» si Steam lo indica, con caché de un par de minutos.
4. Guardar en caché las tres cosas para no saturar la API.

Notas

Los endpoints de Steam Deck y ProtonDB no están documentados. «Jugando ahora» solo funciona si el perfil de Steam es público.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Comprobado: el informe de compatibilidad de Valve responde (Hades, categoría 3, verificado), ProtonDB también (platino, 757 informes) y GetNewsForApp filtra los anuncios del estudio. Las dos primeras no están documentadas. La Deck ya se usa para la plataforma de las reseñas (deck_minutes).

Cómo lo haría

1. Game.deck_status (0 a 3) y Game.proton_tier con su fecha; un comando en nightly por tandas, como localize_games, solo para los que tienen ID de Steam.
2. Insignia en la ficha y filtro ?deck= en el catálogo y en la biblioteca.
3. Novedades: GetNewsForApp con feeds=steam_community_announcements para los juegos en Jugando, en un bloque cargado con HTMX y una caché de 6 horas, sin guardar nada.
4. Jugando ahora: steam.player_summary ya llama a GetPlayerSummaries, solo falta leer gameid y gameextrainfo. Con caché de 2 minutos y cargado aparte, para que no frene el perfil.

Para profundizar

- Tu pendiente para la Deck: si tienes deck_minutes en algún juego ya se sabe que tienes una, sin preguntar. Filtro «verificado», ordenado por duración.
- Cuándo volver: detectar una actualización grande o la salida de acceso anticipado en un juego que abandonaste o tienes pendiente, y proponer volver. Las noticias sirven para algo más que leerlas.
- Si sigue vivo: GetNumberOfCurrentPlayers no necesita clave (comprobado: 3107 en Hades). En un multijugador, antes de comprarlo, importa más que la nota.
- Tu gente ahora: con el sondeo de 100 SteamID por llamada de «Sincronización automática de la biblioteca», un bloque en la portada con quién está jugando a qué.

---

44. Encontrar a tus amigos de Steam y Xbox en NextPlay

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Al vincular Steam o Xbox, sugerir seguir a tus amigos de esas plataformas que ya están en NextPlay, y un enlace para invitar a los que no.

Por qué

Una red social nueva está vacía al principio. Las cuentas vinculadas permiten saber quién es amigo de quién sin que el usuario busque a nadie.

Pasos previstos

1. Steam: leer la lista de amigos de cada cuenta vinculada (GetFriendList, si es pública) y cruzarla con las cuentas de Steam de NextPlay.
2. Xbox: leer los amigos en el momento de vincular, mientras se tiene el token.
3. Pantalla «Tus amigos ya están aquí» con botón de seguir a todos.
4. Enlace de invitación personal.

Notas

Depende de la propuesta de seguir a otros jugadores.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Seguir ya está hecho, con su botón y sus avisos. Falta PlayStation en la propuesta: según psn-api, el mismo token da los amigos.

Cómo lo haría

1. steam.friend_ids(steam_id) con ISteamUser/GetFriendList y un cruce con SteamAccount; Xbox con peoplehub y PlayStation con su lista de amigos, en el momento de sincronizar y con su token de ese momento (comprobar las dos).
2. PlatformFriend (usuario, plataforma e ID externo), para detectar a quien llega después sin volver a pedirlo todo. Se borra al desvincular.
3. Pantalla tras vincular: «Tus amigos ya están aquí», con «Seguir a todos» en un solo POST.
4. Aviso a los dos cuando llega alguien nuevo: «Tu amigo de Steam X se ha unido».
5. Privacidad: una opción «No dejar que me encuentren por mis cuentas vinculadas» y solo perfiles públicos.

Para profundizar

- Enlace de invitación con ?de=usuario: al registrarse se siguen los dos. Un solo enlace que se manda por el grupo, sin correos masivos.
- Sugerir con datos: «Tu amigo de Steam, coincidís en 40 juegos», con la compatibilidad de «La gente que sigues en la ficha y en los perfiles».
- El camino de vuelta: desde el perfil de quien sigues, añadirlo en su plataforma (steam://friends/add/<id> y las insignias que ya existen).
- Una vez por semana en nightly para los de Steam, una sola llamada por cuenta: tu lista de amigos cambia y no hay por qué volver a vincular.

---

45. Precios de Nintendo eShop y PlayStation Store

PRIORIDAD MEDIA · EN CURSO · Funcionalidad · propuesta el 17/10/2026

Qué es

Añadir los precios de la eShop de Nintendo y de la tienda de PlayStation en la ficha, igual que ya se hace con PC y Xbox.

Por qué

El README dice que para PlayStation y Nintendo no hay fuente de precios y que la ficha solo enlaza a una búsqueda en su tienda. Nintendo sí tiene una API de precios pública, aunque no esté documentada.

Pasos previstos

1. Nintendo: buscar el NSUID de cada juego con la API de búsqueda de Nintendo Europa y emparejarlo por nombre.
2. Pedir el precio y el descuento con la API de precios de la eShop (api.ec.nintendo.com) para ES.
3. PlayStation (experimental): leer el precio de la API que usa la web de PlayStation Store.
4. Mostrarlos como dos tiendas más en la ficha y meterlos en update_prices.

Notas

Las dos son sin documentar, como la de Xbox. La de PlayStation es la más frágil; la de Nintendo la usan desde hace años muchas webs de ofertas. Nintendo hecha el 26/10/2026 (games/nintendo.py): update_prices busca el NSUID por el nombre exacto del juego y por su nombre en España (sin contar mayúsculas, tildes ni signos) en el buscador de Nintendo Europa, que tiene el título original y sus traducciones; lo guarda en el juego, con un NSUID manual en el admin para corregirlo, y pide el precio a api.ec.nintendo.com de 50 en 50. Si el mismo título está en Switch y en Switch 2 elige el de Switch. La ficha enseña la tienda «Nintendo eShop» en lugar del enlace de búsqueda. Probada con los 400 juegos más populares de IGDB: 66 de los 101 de Switch emparejados y 65 con precio; los demás se

venden con otro nombre (Complete Edition, remasters, colecciones), ya no se venden o tienen un homónimo en el catálogo. Falta PlayStation: los términos de playstation.com prohíben el acceso automatizado a sus datos; la alternativa es la API de PlatPrices (clave a petición, gratis sin fines comerciales).

REVISIÓN DEL 01/11/2026

Cómo está hoy

Nintendo está hecho y fusionado, aunque en la base de datos local hay 0 NSUID porque update_prices no corre desde el 24/10. PlayStation está parado por una razón de peso: los términos de playstation.com prohíben el acceso automatizado. Tal como está, la propuesta no puede cerrarse: la dividiría.

Cómo lo haría

1. Lanzar update_prices (con nightly se arregla solo) para ver la eShop en local.
2. Dividirla: Nintendo a «hecha» y «Precios de PlayStation» aparte, para no tener una «en curso» que no puede avanzar.
3. Pedir la clave gratuita de PlatPrices (sin fines comerciales) y probar el precio, el mínimo y las rebajas para España. Si sirve, games/playstation.py con la misma forma que games/nintendo.py y su shop_id propio.
4. Si no hay clave, dejar el enlace de búsqueda que ya existe. Sin scraping de la tienda.

Para profundizar

- Historial propio de consola: Game.price_history solo tiene PC, porque sale de IsThereAnyDeal, y los precios de Xbox y Nintendo se sobrescriben cada noche. Guardar cada cambio en una tabla empieza un dato que nadie da en España y que no se puede recuperar después. Sin él, «mínimo histórico» y «¿compro o espero?» se quedan solo en PC.
- Mejorar el emparejado de Nintendo (66 de 101): los que se venden como Complete Edition o colección se pueden buscar con store_alternatives y los nombres alternativos, y el «no es este» corregido por un usuario sirve a todos, como en ExternalTitle.
- Cruzar con las importaciones: el titleId de PlayStation (CUSA o PPSA) es un ID real por el que buscar en PlatPrices, en vez del nombre.
- Comparar entre plataformas: «en PC cuesta 10 € menos que en tu consola», para quien tiene varias.

---

46. ¿Cuándo bajará? Próxima rebaja prevista en el veredicto

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Cuando el veredicto dice «suele bajar más: espera», decir hasta cuándo y a cuánto: «en las rebajas de invierno de Steam del año pasado bajó a 9,99 €; empiezan dentro de 12 días».

Por qué

El veredicto de ahora dice que esperes pero no cuánto, y sin una fecha la gente acaba comprando. NextPlay ya guarda dos años de historial por tienda (Game.price_history); cruzarlo con el calendario de rebajas de las tiendas no lo hace Backloggd, que no tiene precios, ni casi ningún comparador.

Pasos previstos

1. Tabla de rebajas (nombre, tienda y fechas) que rellena un administrador con el calendario que publica Valve y los anuncios de Epic, GOG y Xbox.
2. En history.py, buscar a cuánto estuvo el juego en la misma rebaja del año anterior y cada cuánto se rebaja fuerte (días entre rebajas).
3. Veredicto con fecha: «Espera a las rebajas de invierno: el año pasado bajó a 9,99 €» o «se rebaja más o menos cada 3 meses; la última fue hace 80 días».

4. En la lista de deseos, aviso unos días antes: «4 juegos de tu lista bajaron en las rebajas que empiezan el jueves».
5. Avisar también cuando un juego de la lista entra en un bundle (IsThereAnyDeal también los da).

Notas
Es una estimación con lo que pasó otros años, no una promesa: el texto tiene que decirlo así.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Sin empezar. games/history.py ya calcula el veredicto, last_big_sale y series por tienda, pero solo 21 juegos tienen historial guardado, porque se pide al abrir la ficha. Para la lista de deseos (90 juegos en tester_steam) habría que pedirlo de noche.

Cómo lo haría
1. SaleEvent (nombre, tienda, inicio, fin y si está confirmada), que se rellena en el admin con el calendario que publica Valve y los anuncios de las otras tiendas.
2. history.next_sale(game, now): para el próximo evento, a cuánto estuvo en esa rebaja el año anterior, con lowest_series y las fechas del evento.
3. Frecuencia: días entre rebajas fuertes, generalizando last_big_sale.
4. verdict añade el texto con fecha, y games/_price_history.html lo enseña. Siempre como «el año pasado bajó a…»: es una estimación, no una promesa.
5. Pedir de noche el historial de lo que está en listas de deseos, en una tarea de nightly con su límite. Bloque «Se espera rebaja» en la lista de deseos y aviso unos días antes.

Para profundizar
- Medir el acierto: guardar cada predicción y compararla al acabar la rebaja, para decir «acertamos en 8 de 10». Da credibilidad y deja afinar el texto.
- Plan de compra: con un presupuesto («40 €») y tu lista de deseos, qué comprar en la próxima rebaja, combinando bajada prevista y lo que te interesa.
- Bundles: avisar cuando un juego de tu lista entra en uno. IsThereAnyDeal tiene un endpoint para ello (comprobar /games/bundles/v2).
- Cuándo no esperar: «lleva 2 años a este precio y nunca baja de 20 €» también es una respuesta. Decirlo evita esperas inútiles.
- Totales en la lista de deseos, como ya tienen las listas: en tester_steam, 86 deseados con precio suman 2893 € a precios del 24/10 y 1497 € en sus mínimos históricos. Esa diferencia es el mejor argumento para esperar.

---

47. Tareas en segundo plano para lo que tarda

PRIORIDAD MEDIA · PENDIENTE · Infraestructura · propuesta el 01/11/2026

Qué es
Sacar de la petición las importaciones largas y otros trabajos lentos, con un aviso de progreso, usando el sistema de tareas de Django 6.

Por qué
La importación de Steam se hace dentro de la petición (accounts.views._import_steam: Steam, IGDB y los logros de 40 juegos), y en producción puede agotar el tiempo de espera. Con «Logros uno a uno», Playnite o las capturas de Switch tardaría más. Django 6.0.9, el que usa NextPlay, ya trae django.tasks para definir y encolar tareas (comprobado), aunque no incluye quien las ejecute.

Pasos previstos

1. Backend de base de datos (el paquete django-tasks, del autor de esa parte de Django; comprobar que sigue al día) y su proceso worker. En local y en los tests, el ImmediateBackend de Django: todo sigue igual con runserver.
2. ImportRun (usuario, plataforma, estado, paso, hecho, total y resultado), que la tarea va actualizando.
3. Steam primero: «Sincronizar» encola y lleva a una página que pregunta con HTMX (hx-trigger=every 2s) hasta acabar. Los mensajes de _report_import pasan a un aviso en la campana.
4. PlayStation y Xbox se quedan en la petición, con «Formularios que avisan mientras trabajan»: el NPSSO y el código de Microsoft no pueden guardarse en una cola, porque la web promete que no se guardan.
5. nightly sigue siendo un comando: para lo que se programa a una hora, cron basta.

Notas

Sale de la revisión del 01/11/2026. No corre prisa mientras NextPlay esté en local: es más necesaria al publicar o si la importación crece mucho.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. Hay ejecución en paralelo dentro de la petición (ThreadPoolExecutor en los logros de Steam y los trofeos), pero nada fuera de ella.

Para profundizar

- Una tarea por usuario también de noche: un fallo o un bloqueo de una cuenta deja de frenar a los demás, y se puede reintentar solo lo que falló.
- Importar en dos tiempos: juegos y horas al momento, para que se vea el resultado, y logros y emparejados difíciles después, con un aviso al acabar.
- Si no compensa un worker aparte, un hilo en el mismo proceso para los trabajos cortos: sin cola ni persistencia, y por tanto compatible con no guardar el NPSSO.

---

48. Cuánto has gastado y ahorrado: compras detectadas al sincronizar

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 01/11/2026

Qué es

Cuando un juego de tu lista de deseos de Steam desaparece de ella y aparece en tu biblioteca, es que lo has comprado: apuntarlo con el precio de ese día para saber cuánto has gastado, cuánto te has ahorrado y cuánto te sale la hora, sin teclear nada.

Por qué

El precio pagado está previsto a mano en «Colección física con escáner de código de barras», y a mano casi nadie lo rellena. steam.import_wishlist ya detecta lo que deja de estar en la lista y update_library, lo que entra nuevo; falta cruzarlos. En tester_steam, 86 de los 90 deseados ya tienen precio guardado.

Pasos previstos

1. Modelo Purchase (usuario, juego, tienda, precio, precio normal, fecha y si está confirmado), creado en la misma sincronización cuando coinciden las dos cosas.
2. Precio estimado: el de Steam guardado en GamePrice, pidiéndolo en ese momento si es viejo.
3. Aviso para confirmarlo o corregirlo («¿Lo compraste a 14,99 €?»), o decir que fue un regalo.
4. Página privada en el perfil: gastado por año, ahorrado frente al precio normal y coste por hora con las horas importadas.

Notas

Sale de la revisión del 01/11/2026. Solo funciona con Steam y el precio es una estimación, porque Steam no dice lo que se pagó: el texto tiene que decirlo así. Si se compró en otra tienda, lo corrige el jugador.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. Las dos detecciones existen por separado.

Para profundizar

- «Esto te ha ahorrado NextPlay»: si compras tras un aviso de precio, sumar la diferencia. Es la cifra que mejor explica para qué sirve la web.
- Presupuesto de rebajas: un tope por mes o por rebajas, y cuánto llevas.
- Cruzarlo con el pendiente: «Has comprado 12 juegos este año y has empezado 3».

---

49. Importar Epic, GOG, EA y Battle.net con Playnite, y CSV de otros trackers

PRIORIDAD MEDIA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Importar las horas de las tiendas sin API (Epic, GOG, EA, Ubisoft, Battle.net, Amazon…) desde la exportación de Playnite, el lanzador que las junta todas, y aceptar el CSV de otros trackers para que quien llega se traiga su historial.

Por qué

De Epic, GOG y Battle.net NextPlay solo guarda el nombre de usuario (en «Otras plataformas»), porque no dan las horas. Playnite ya las lee en el PC del usuario y se pueden exportar. Y quien viene de otro tracker no quiere empezar de cero: Backloggd no tiene importación ni exportación (las dos siguen en su hoja de ruta), así que NextPlay llegaría antes.

Pasos previstos

1. Aceptar el CSV de la extensión de exportación de Playnite (nombre, tienda de origen, plataforma, tiempo jugado, última actividad y estado).
2. Emparejar con match_by_name y guardar cada tienda como plataforma nueva de ExternalTitle («epic», «gog»…), aprovechando todo el sistema de correcciones.
3. Pasar el estado de Playnite (Completado, Abandonado, Jugando…) solo a los juegos nuevos, sin pisar lo que haya puesto el usuario.
4. Importador de CSV genérico con un paso para decir qué columna es cada cosa, para los trackers que exportan (Grouvee y otros).
5. Más adelante, una extensión de Playnite propia que sincronice sola con un token personal del usuario.

Notas

Comprobar las columnas exactas antes de empezar: en el foro de Playnite recomiendan la extensión Library Exporter Advanced. No leer la web de Backloggd para importar (es frágil y seguramente va contra sus condiciones); si saca su exportación a CSV, aceptarla el primer día.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Antes hay que resolver que LibraryEntry tiene dos columnas por plataforma: ver «Horas por plataforma sin una columna por tienda». Tampoco se ha visto aún un CSV real de Library Exporter Advanced.

Cómo lo haría

1. Exportar un CSV real desde Playnite y fijar las columnas antes de escribir nada.
2. accounts/csv_import.py y una página de subida: csv.DictReader detectando separador y BOM, un paso para decir qué columna es cada cosa si no es de Playnite, y una vista previa de 20 filas.
3. Una plataforma por tienda en ExternalTitle.Platform (epic, gog, ea, ubisoft, battlenet, amazon); como ID, el de la tienda si lo trae o el nombre normalizado, como ya hacen PlayStation y Xbox («name:…»).
4. match_by_name por lotes, con MAX_SEARCHES por importación: los que no salgan quedan en «Sin emparejar», y se reintentan al volver a importar. Con muchos juegos, como tarea en segundo plano.
5. Estado solo en los juegos nuevos, sin pisar lo que haya puesto el usuario, y la última actividad a last_played.

Para profundizar

- Extensión propia de Playnite: un token personal en Editar perfil y un POST a /api/sync. Sincroniza solo las tiendas sin API y, como Playnite cuenta el tiempo de cada sesión, da mejores datos que Steam.
- GOG Galaxy: su base de datos local también junta varias tiendas; un exportador daría una segunda vía para quien no use Playnite.
- Exportar en el mismo CSV: entrada y salida libres generan confianza, y Backloggd aún no las tiene.
- Plantillas por tracker: Grouvee y otros, con las columnas ya mapeadas, para que importar sea un solo paso.

---

50. Preguntas sobre un juego que no destripan nada

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 01/11/2026

Qué es

Preguntar algo de un juego («¿merece la pena seguir tras el capítulo 3?») y que las respuestas se plieguen según lo que lleve jugado quien las lee.

Por qué

Las reseñas ya se pliegan cuando van por delante de quien las lee (AHEAD_HOURS), con horas verificadas que otros trackers no tienen. Llevarlo a preguntas da algo que ni foros ni wikis pueden dar: saber cuánto había jugado quien contesta y no destripar lo que te queda.

Pasos previstos

1. Modelos Question y Answer, que guardan las horas y el estado de quien escribe, como Review.snapshot.
2. Cada respuesta se pliega si va por delante de quien lee, con la misma regla que las reseñas; con los logros se afina por avance real.
3. Sección en la ficha cargada con HTMX; las respuestas se votan y se puede marcar la mejor.
4. Denuncias con el Report que ya existe.

Notas

Sale de la revisión del 01/11/2026. Necesita gente para tener sentido: no antes de abrir NextPlay.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Propuesta nueva. La regla de plegado y las denuncias ya están hechas.

Para profundizar

- Que llegue a quien lo sabe: «Alguien pregunta por un juego que terminaste», con un límite para no molestar.

- Consejos por tramos: lo que conviene saber a las 5 h, a las 20 h y antes del final, sin destripar lo siguiente.

51. Detalles sueltos: recuadros de los formularios, el corazón de las listas y pestañas cortadas

PRIORIDAD BAJA · EN CURSO · Diseño · propuesta el 28/10/2026

Qué es

Tres detalles de las capturas a 390 px: «Tipo de lista» y «Niveles» del editor de listas llevan el recuadro blanco que el navegador pone a los fieldset; el botón de las listas que añade a «Lo quiero» los juegos que te faltan sigue con el □ de texto en vez del icono del corazón; y en el móvil las pestañas de Editar perfil se cortan («Redes socia…») sin nada que diga que se pueden deslizar.

Por qué

Los vio el repaso con anti-ui-slop del 28/10/2026. Son rápidos y desentonan: el resto de formularios ya quitan ese recuadro y los demás botones de «Lo quiero» usan el icono.

Pasos previstos

1. Un estilo común para fieldset y legend en app.css (sin borde ni relleno y la leyenda como las etiquetas), en vez de repetirlo en .settings-card, .challenge-form y .report-form.
2. lists/detail.html: cambiar el □ por el icono heart de _icon.html.
3. Difuminar el borde de .tabs por el lado en el que quedan pestañas cuando se pueden deslizar.

Notas

Se pueden hacer de paso con cualquiera de las otras mejoras de diseño. 28/10/2026: en la rama listas-arrastrar, de paso con «Listas: colocar las carátulas arrastrándolas», ya están los dos primeros: un estilo común quita el recuadro a todos los fieldset (y de paso alinea «Tipo de reto» con los demás campos del formulario de retos) y el botón de las listas usa el icono del corazón. Falta difuminar las pestañas. 02/11/2026: en la rama tierlists-biblioteca, como el perfil tiene una pestaña más («Tier lists»), la pestaña activa ya se centra en su barra al entrar (un script en base.html para todas las .tabs, que solo mueve la barra).

REVISIÓN DEL 02/11/2026

Cómo está hoy

Dos de tres hechos, y la pestaña activa ya se centra al entrar. Falta el difuminado de las pestañas: .tabs no lleva ninguna máscara.

Cómo lo haría

1. En app.css, una máscara con degradado en .tabs dentro de @supports, usando animation-timeline: scroll(self inline) para que solo aparezca en el lado en el que queda algo por ver. Sin JavaScript.
2. En los navegadores sin eso, una máscara fija a la derecha, que es mejor que nada.
3. Probar a 390 px en Editar perfil y en el perfil, y pasar a «hecha».

Para profundizar

- Un pase de detalles cada dos semanas con capturas a 390 px, vaciando esta propuesta de una vez en lugar de tener muchas pequeñas.
- Capturas automáticas con Playwright en las páginas principales, para ver qué ha cambiado sin tener que mirarlo a mano.

52. Publicar NextPlay y abrir una beta antes de las rebajas de invierno

PRIORIDAD BAJA · PENDIENTE · Lanzamiento · propuesta el 21/10/2026

Qué es

Poner NextPlay en internet con su dominio, abrir una beta cerrada con 10 o 20 personas y llegar a las rebajas de invierno de Steam (del 17 de diciembre al 4 de enero) con gente usándola.

Por qué

El objetivo es que la use gente y crezca, y ahora solo funciona en local: no hay servidor de aplicaciones (gunicorn), con DEBUG apagado Django no sirve los estáticos, los correos (recuperar la contraseña y avisos de precio) salen por la consola y la tarea nocturna no está programada. Las rebajas de invierno son el mejor momento para enseñar lo que Backloggd no tiene: precios, mínimos históricos y avisos de la lista de deseos.

Pasos previstos

1. Antes de dejar entrar a nadie: sacar el .env del repositorio y cambiar las claves, la parte del RGPD y un botón para denunciar reseñas y bloquear usuarios (estaba en el corte 4 del README).
2. Producción: gunicorn y whitenoise, PostgreSQL, un servidor en la UE, dominio con HTTPS, un proveedor de correo de verdad, copias de seguridad de la base de datos y aviso por correo de los errores (ADMINS).
3. Poner la dirección pública en la URI de vuelta de la aplicación de Azure, en el realm de Steam, en SITE_URL y en DJANGO_ALLOWED_HOSTS, y programar nightly con cron.
4. Beta cerrada con 10 o 20 conocidos y un canal de Discord para que cuenten fallos y qué echan en falta.
5. Medir lo justo: cuántos vinculan alguna plataforma y cuántos vuelven la semana siguiente.
6. Con la beta estable, enseñarla en comunidades de videojuegos en español (subreddits, servidores de Discord, Mediavida) con algo que se pueda compartir: la tarjeta del perfil o «¿A qué jugamos?».

Notas

Primera meta realista: 50 personas que vuelvan cada semana. La importación de PlayStation usa una API no oficial: dejarla marcada como experimental. Si algún día se monetiza (anuncios o enlaces de afiliado), revisar antes las condiciones de uso comercial de IGDB e IsThereAnyDeal. 21/10/2026: aparcada; de momento el usuario prefiere centrarse en el perfil, las listas, los retos y las reseñas. 24/10/2026: las reseñas y los comentarios ya tienen botón de denunciar; antes de publicar faltan bloquear usuarios y llevar las imágenes que se suben (perfiles y reseñas) a un almacenamiento aparte, porque ahora se guardan en MEDIA_ROOT del propio servidor.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Aparcada por decisión tuya (21/10), y sigo sin empujarla. Comprobado hoy, buenas noticias: check --deploy solo da 2 avisos menores (HSTS), los 422 tests pasan en 33 s y las migraciones están al día. Faltan gunicorn y whitenoise (no están en requirements.txt), correo de verdad, CACHES, un almacén para las imágenes, probar con PostgreSQL y CI (no hay carpeta .github), y he encontrado tres problemas que solo saldrían en producción (pasos 3 a 5). La fecha de su título, 17/12, solo se sostiene si se decide pronto.

Cómo lo haría

1. Lo único que haría ya, aunque nunca se publique: GitHub Actions con los tests en cada PR, en SQLite y en PostgreSQL. En local se prueba con SQLite y en producción iría PostgreSQL, y JSONField, update_conflicts o los órdenes pueden comportarse distinto.
2. Cuando toque: gunicorn, whitenoise, una caché compartida y las imágenes en un almacén compatible con S3 en la UE (django-storages). La compresión importa: app.css pasa de 136 a 28 KB, el perfil de 207 a 23 KB y la lista de deseos de 248 a 15 KB, y hoy no hay nada que comprima.

3. Que los errores se vean: settings no define LOGGING ni ADMINS, y con DEBUG apagado un error 500 no sale en ningún sitio (comprobado: django.request no escribe nada). Añadir LOGGING a la consola y un servicio de errores.
4. Que una visita sin sesión no gaste cuota: games.views.game_media y price_history llaman a IGDB y a IsThereAnyDeal y escriben en la base de datos en un GET, sin pedir sesión. Hay 2188 juegos sin imágenes pedidas y 1182 sin historial: un buscador recorriendo las fichas son más de 3000 llamadas. Sin sesión, enseñar lo que haya; pedirlo solo con sesión o de noche para los más vistos.
5. Sin esperas dentro de una petición: ITADClient._send duerme hasta 5 minutos tras un 429 (time.sleep), y desde price_history eso bloquea un proceso web, que gunicorn mataría a los 30 s. En la web, sin reintentos; los reintentos, solo en los comandos.
6. Servidor pequeño en la UE con PostgreSQL, copias diarias con pg_dump a otro sitio, correo transaccional y nightly por cron (ver «Sincronización automática de la biblioteca»).
7. Ensayar el paso de SQLite a PostgreSQL con dumpdata y loaddata antes del día D.
8. Direcciones públicas: URI de vuelta de Azure, realm de Steam, SITE_URL y DJANGO_ALLOWED_HOSTS.
9. Requisitos para dejar entrar a nadie: claves cambiadas, «Borrar la cuenta, descargar mis datos y privacidad (RGPD)», «Bloquear, silenciar y quitar seguidores» y «Páginas de error propias: no encontrada, fallo del servidor y formulario caducado». Después, beta por invitación.

Para profundizar

- Condiciones de uso: IGDB es gratis para uso no comercial; si algún día hay afiliados o anuncios, hablar antes con Twitch. Mirar también las de IsThereAnyDeal, y el aviso que piden para los enlaces y los datos.
- Seguridad de las cuentas: un límite de intentos de inicio de sesión y la política de contenido (CSP) que trae Django 6. Comprobar que el NPSSO no llega nunca a los logs, ni siquiera en un error.
- Un entorno de pruebas público con datos de seed_test_profile, para enseñar NextPlay sin arriesgar cuentas de nadie.
- Vigilancia barata: un servicio de errores con plan gratuito y una dirección /salud/ para un monitor externo, que avise de que algo se ha caído antes que los usuarios.

---

53. Sincronización automática de la biblioteca

PRIORIDAD BAJA · EN CURSO · Infraestructura · propuesta el 17/10/2026

Qué es

Que la biblioteca de Steam se actualice sola cada noche (horas y juegos nuevos) y que update_prices se programe de verdad, sin tener que pulsar nada.

Por qué

Ahora todo es manual: el botón de sincronizar y el comando de precios. Las alertas de precio, Game Pass y los logros necesitan tareas que se ejecuten solas.

Pasos previstos

1. Comando sync_libraries que recorra las cuentas de Steam vinculadas.
2. Programar las tareas: cron del servidor o un planificador de Django (django-q2, Celery beat).
3. Repartir las llamadas a lo largo de la noche para no pasar los límites de Steam e IGDB.
4. Mostrar en el perfil la fecha de la última sincronización automática y un interruptor para desactivarla.

Notas

Xbox solo se podría si se guarda cifrado el token de refresco, que ahora no se guarda a propósito. PlayStation no se puede sin guardar el NPSSO, y se decidió no guardarlo. Hecha el 18/10/2026 con cron (sin Celery ni django-q): comandos sync_libraries (pausa entre usuarios, salta a los sincronizados en las últimas 12 h, un fallo no para a los demás) y nightly (Steam y después precios), interruptor y fecha de la última sincronización en el panel de Steam. Probada con una cuenta real de Steam. Falta programarla de verdad: cron en el servidor o Programador de tareas de Windows (instrucciones en el README). 01/11/2026: el usuario la programará cuando despliegue NextPlay; en local se queda sin programar y se sincroniza a mano.

REVISIÓN DEL 01/11/2026

Cómo está hoy

El comando existe y funciona; lo único que falta es programarlo, y lo harás al desplegar (decisión tuya del 01/11/2026), así que pasa a prioridad baja. Mientras tanto, sincronizar a mano sigue apuntando sesiones: solo tu propio historial queda menos detallado. Por eso hay 8 sesiones y los precios son del 24/10, y mientras no haya más usuarios no afecta a nadie más.

Cómo lo haría

1. Al desplegar: cron en el servidor, con la línea que ya está en el README, como parte de «Publicar NextPlay y abrir una beta antes de las rebajas de invierno». Conviene que sea de lo primero: desde ese día, lo que no se apunta cada noche no se recupera.
2. Que nightly deje rastro: un registro con rotación y un modelo pequeño TaskRun (tarea, inicio, fin, resultado y cifras) visible en el admin. Si en 36 h no hay ninguno, un aviso al administrador en la campana.
3. Añadir a Command.TASKS lo que vaya apareciendo (close_challenges, update_releases…): cada una sigue aunque falle la anterior.
4. Si alguna vez se usa en Windows: el schtasks del README no hace nada si el ordenador está apagado a las 4:00; Register-ScheduledTask con -StartWhenAvailable se lanza en cuanto se enciende. Corregirlo en el README cuando se toque.

Para profundizar

- Sesiones mejores con playtime_2weeks: GetOwnedGames trae lo jugado en las dos últimas semanas (comprobado: 7 juegos en tester_steam). En la primera importación se pueden crear sesiones en vez de no apuntar nada, y una noche perdida se recupera.
- «Jugando ahora» y sesiones con hora: GetPlayerSummaries acepta 100 SteamID por llamada y dice a qué juega cada uno. Un sondeo cada 10 minutos da el principio y el fin de cada sesión gastando muy poco de las 100.000 llamadas diarias.
- Recordatorio para lo que no puede ser automático: «Hace 30 días que no sincronizas PlayStation», con el enlace directo. No guarda nada de Sony, que fue lo que se decidió.
- Ahorrar de noche: saltarse a quien lleva 60 días sin entrar; sync_libraries ya va del más antiguo al más nuevo.

---

54. Páginas de error propias: no encontrada, fallo del servidor y formulario caducado

PRIORIDAD BAJA · PENDIENTE · Diseño · propuesta el 28/10/2026

Qué es

No hay plantillas 404.html, 500.html ni 403_csrf.html, así que con DEBUG apagado Django enseña sus páginas por defecto, en inglés y sin estilo. Hacerlas con el aspecto de NextPlay: «No encontramos esta página» con el buscador y enlaces al inicio y al catálogo, un 500 que no dependa de la base de datos y la del formulario caducado con «Vuelve atrás, recarga la página y envíalo otra vez».

Por qué

Es el estado de error de las páginas enteras, que el repaso con anti-ui-slop del 28/10/2026 dejó sin cubrir: los avisos de fallo nuevos solo sirven para lo que va por HTMX. En local no se nota porque DEBUG enseña la página técnica, pero hacen falta antes de publicar.

Pasos previstos

1. templates/404.html sobre base.html, con el buscador y enlaces al inicio y al catálogo.
2. templates/500.html que se valga sola (sin consultas ni nada que pueda volver a fallar), con el CSS de static.
3. templates/403_csrf.html para el formulario caducado y templates/403.html.
4. Un test con DEBUG apagado que pida una dirección que no existe y compruebe que sale la página propia.

Notas

Va con «Publicar NextPlay y abrir una beta antes de las rebajas de invierno».

REVISIÓN DEL 01/11/2026

Cómo está hoy

Igual: no hay ninguna de estas plantillas en templates/. Con DEBUG apagado sale la página de Django, en inglés.

Cómo lo haría

1. templates/404.html y 403.html sobre base.html, con el buscador y enlaces al inicio y al catálogo.
2. templates/500.html sin extends ni etiquetas que puedan fallar: HTML con un poco de CSS en la propia página, porque si ha fallado la base de datos, base.html y sus avisos también fallarían.
3. templates/403_csrf.html para el formulario caducado, explicando qué hacer. HTMX ya lo avisa por su lado en base.html.
4. Tests con override_settings(DEBUG=False) y el cliente con raise_request_exception=False: una dirección que no existe, un fallo provocado y un POST sin CSRF.

Para profundizar

- Un 404 útil en juegos: si /juegos/<slug>/ no existe, buscar ese nombre en IGDB y ofrecer traerlo. Cada 404 hace crecer el catálogo en vez de ser un callejón sin salida.
- Registrar los 404 con su procedencia, para encontrar enlaces rotos en vez de esperar a que alguien avise.
- 429 y mantenimiento: páginas propias para cuando haya límites de peticiones o una actualización en curso.
- Con personalidad propia: la 404 es una de las pocas páginas que se recuerdan; textos con guiños a videojuegos escritos para NextPlay.

---

55. Retos del año y control del pendiente

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Ponerte un reto («terminar 20 juegos en 2027») con su barra de progreso, ver cuánto vale tu pendiente al precio de hoy y repasar los juegos que llevan meses en «Jugando» sin tocarse.

Por qué

Las importaciones llenan el pendiente con cientos de juegos y dejan en «Jugando» juegos que ya no juegas. Un reto y un poco de vergüenza («tu pendiente vale 1.240 €») animan a terminarlos.

Pasos previstos

1. Modelo de reto anual por usuario y barra de progreso en el perfil.
2. Guardar la fecha en que un juego pasa a Completado (ahora solo hay updated_at).
3. Valor del pendiente sumando el mejor precio actual de cada juego.
4. Asistente «Revisa tu biblioteca» con los juegos en Jugando sin partidas en 6 meses (las fechas de la última partida ya se importan) para pasarlos a Abandonado o Pendiente.

Notas

Con la duración estimada de los juegos se podría añadir «a tu ritmo terminarías el pendiente en N años». 04/11/2026: de la fase 2 de los retos llega una pieza más para el repaso: proponer «¿Lo has terminado?» al llegar a las horas o al logro del final, para que los retos de terminar juegos no se queden a cero con bibliotecas importadas.

REVISIÓN DEL 01/11/2026

Cómo está hoy

La mitad ya no hace falta: los retos cubren el anual (un reto privado «20 juegos en 2027») y completed_at existe. Queda lo más interesante. En tester_steam, el pendiente son 120 juegos, 447,91 € contando solo los 34 con precio, porque a 85 nunca se les han pedido (update_prices no corre desde el 24/10); 2 de los 7 en Jugando llevan más de 6 meses sin tocarse, y 220 están en Jugado sin saber si se terminaron.

Cómo lo haría

1. Renombrarla «Valor del pendiente y repaso de la biblioteca» y quitar lo del reto anual, con un botón «Reto del año» que abra el formulario de retos con las fechas ya puestas.
2. library/backlog.py: value(user) suma game.cheapest en sus tiendas y dice sin disimular lo que falta: «447,91 € en 34 de tus 120 juegos». Sin inventarse el resto.
3. Asistente /biblioteca/repasar/: una tarjeta por juego con botones grandes («Lo terminé», «Lo dejo», «Sigo», «Al pendiente»); con HTMX, responder trae la siguiente. Primero los de Jugando sin tocar y luego los de Jugado con más horas respecto a su duración.
4. LibraryEntry.reviewed_at, para no volver a preguntar. Lo marcado aquí se agrupa en la actividad («Ha ordenado 40 juegos»), en lugar de llenar el feed con 40 tarjetas.
5. Al terminar, un resumen: «Has ordenado 40 juegos: 12 terminados y 9 abandonados».

Para profundizar

- Pistas para contestar en un clic: «Llevas 40 h y suele durar 30: ¿lo terminaste?», o el logro del final si existe. Medido en tester_steam: de sus 220 en Jugado, 40 tienen entre 1 y 4 veces la duración normal (Elden Ring, Hollow Knight, Red Dead Redemption 2, God of War…), 20 están entre el 70 y el 100 % y 61 no llegan al 15 %, así que 121 tendrían ya una respuesta propuesta. Hay que excluir los juegos de servicio: Fortnite, Dead by Daylight o Rocket League también salen, porque IGDB les da modo de un jugador. Es lo que llena Completado, y de eso dependen los retos, las estadísticas y la duración de la comunidad. La revisión lo convierte en una de las piezas más útiles de la lista.
- Fecha aproximada: al marcarlo terminado aquí, usar la última partida o el logro final como completed_at, y no la fecha de hoy, para que no cuente en un reto que no le toca.
- Proyección: «A tu ritmo terminarías tu pendiente en N años», con la duración de los juegos y PlaySession, y juegos añadidos frente a terminados este año.
- Lo que ya podrías jugar gratis: cuánto de tu pendiente está en Game Pass, o cuánto te ahorrarías cancelando la suscripción.

56. Etiquetas de la comunidad y «¿Qué juego esta noche?»

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Que quien ha jugado un juego le ponga etiquetas como «relajante», «difícil», «para ratos cortos» o «buena historia», y un buscador rápido por tiempo, compañía y ánimo.

Por qué

Los géneros de IGDB («Aventura», «RPG») no dicen cómo se siente un juego. Las etiquetas de la comunidad sí, y permiten buscar por situación y no por categoría.

Pasos previstos

1. Lista cerrada de etiquetas (ánimo, dificultad, duración de sesión, accesibilidad, cooperativo local...).
2. Votarlas solo si el juego está en tu biblioteca y no está en Pendiente.
3. Mostrar las más votadas en la ficha y filtrar por ellas en el catálogo.
4. Formulario «¿Qué juego esta noche?»: cuánto tiempo tienes, solo o acompañado y qué te apetece; busca primero en tu propia biblioteca.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar y, sin usuarios, sin datos. Pero hay de dónde arrancar: las categorías de Steam incluyen cooperativo y accesibilidad en español (comprobado) e IGDB tiene temas en 2180 juegos y perspectivas en 2178.

Cómo lo haría

1. Modelos Tag (nombre y familia: ánimo, dificultad, sesión, accesibilidad, compañía) y GameTagVote, solo para quien tiene el juego y no lo tiene en Pendiente.
2. Sembrar con los temas de IGDB y las categorías de Steam como votos «del sistema», marcados como tales, para que no empiece vacío.
3. Ficha: las más votadas como chips, votables con HTMX. Pedirlas en el mejor momento: al pasar a Completado, «¿Cómo es?» con cinco chips.
4. Filtro en el catálogo con Exists, en el mismo sitio que los demás filtros.
5. «¿Qué juego esta noche?»: tres preguntas (tiempo, solo o acompañado y qué te apetece) que buscan primero en tu biblioteca y tu pendiente.

Para profundizar

- Etiquetas que salen de los datos: «para ratos cortos» (mediana de sesión baja, con PlaySession), «largo de verdad» (duración) o «difícil» (porcentajes bajos de logros). No piden nada a nadie y funcionan desde el primer día.
- Accesibilidad como filtro propio: opciones de subtítulos, alternativas de color, dificultad ajustable o sin restricciones de tiempo. Steam ya lo da, con esos nombres en español.
- Palabras nuestras: «de sofá», «para jugar con mi hermano», «para desconectar». La lista cerrada es parte del carácter de NextPlay; cuidarla en castellano la distingue de las etiquetas traducidas de Steam.
- Un botón «¿Qué juego?» siempre a mano en la barra: es lo que promete el nombre de la web.

57. Ranking personal por duelos

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Un juego rápido de «¿cuál te gustó más?» entre dos juegos que has jugado, que va construyendo tu top personal de verdad.

Por qué

Con estrellas de 1 a 5 casi todo acaba en 4 o 5 y no se puede ordenar. Comparar de dos en dos es más fácil y engancha.

Pasos previstos

1. Página de duelos con dos juegos de tu biblioteca (completados o jugados) y un botón «saltar».
2. Puntuación tipo Elo por usuario y juego, que elige los duelos más útiles (juegos con puntuación parecida).
3. Tu top 10 en el perfil, ordenado por esa puntuación.
4. Más adelante, usar estas preferencias en las recomendaciones.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. Con 2 de 348 juegos valorados, tiene más sentido del que parecía: comparar dos juegos cuesta menos que poner nota.

Cómo lo haría

1. Inserción binaria en vez de Elo: para colocar un juego en tu orden bastan unas 8 comparaciones con 256 juegos, y no hay empates. Elo pide muchos duelos para estabilizarse y necesita elegirlos bien.
2. Guardar el resultado como una GameList numerada normal, usando GameList.move: se ve, se comparte y se comenta como cualquier lista, sin modelos nuevos salvo el de la sesión de duelos.
3. Página /duelos/: dos portadas grandes, flechas del teclado, «No lo he jugado» y «Saltar»; cada respuesta por HTMX.
4. Orden de entrada: primero los más jugados por horas, que son los que uno recuerda.
5. Tu top 10 en una sección nueva del escaparate (SECTIONS), y que pueda rellenar los favoritos.

Para profundizar

- Clasificar la fila «Sin clasificar» de una tier list por duelos: en el móvil es mucho más cómodo que arrastrar, y coloca en niveles según el orden.
- Notas sugeridas: del orden se puede deducir una nota aproximada y proponerla en un paso. Otra vía de llenar una columna casi vacía.
- Duelo público de la semana: dos juegos, votan todos y se acumula un ranking de la comunidad; fácil de contestar y de compartir.
- Como señal para «Recomendaciones «Qué jugar después»»: preferencias claras con muy poco esfuerzo.

58. Directos de Twitch en la ficha del juego

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Mostrar en la ficha los directos en español que hay ahora mismo de ese juego, con el número de espectadores y un enlace a Twitch.

Por qué

Las credenciales de IGDB son las de una aplicación de Twitch, así que sirven también para su API sin pedir ninguna clave nueva. Da vida a la ficha y ayuda a decidir si un juego te va a gustar.

Pasos previstos

1. Pedir a la API de Twitch (Helix) el ID de categoría del juego a partir de su ID de IGDB y guardarlo.
2. Pedir los directos de esa categoría filtrando por idioma (español primero) y quedarse con los 3 o 4 con más espectadores.
3. Guardar la respuesta en caché unos minutos para no llamar a Twitch en cada visita.
4. Bloque «En directo ahora» en la ficha, oculto si no hay directos.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Comprobado: las credenciales de IGDB sirven para Helix. games?igdb_id=1942 da The Witcher 3 con su ID de Twitch, y streams con language=es y clips responden.

Cómo lo haría

1. games/twitch.py reutilizando IGDBClient._token y el mismo client_id. Game.twitch_id se guarda la primera vez que se pide.
2. streams(game, language=«es», first=4), con 3 minutos de caché.
3. Fragmento cargado con HTMX, como games:media, para no frenar la ficha; oculto si no hay directos.
4. Tarjeta con título, espectadores y miniatura, con enlace a Twitch. El reproductor, solo al pulsar, como el de YouTube, para no meter a Twitch en cada visita.
5. Si no hay ninguno en español, «N directos en otros idiomas».

Para profundizar

- Gente de NextPlay en directo: SOCIAL_LINKS ya guarda el usuario de Twitch y streams?user_login= admite 100 a la vez. Insignia «En directo» en el perfil y un bloque con la gente que sigues. Cuesta muy poco y es algo propio.
- Clips en el carrusel de la ficha, como un filtro más (helix/clips responde, comprobado).
- Lo que se ve hoy: popularity_primitives de tipo 34 da las horas vistas en 24 h, sin llamar a Twitch por cada juego, para un bloque «Lo más visto».
- Aviso opcional si alguien que sigues empieza un directo con un juego de tu lista de deseos: verlo jugado ayuda a decidir la compra.

---

59. Buscar grupo para jugar

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Publicar «busco gente para X en PS5 esta noche» en los juegos multijugador, con tu PSN ID, gamertag o código de amigo ya puesto.

Por qué

Los usuarios ya guardan sus IDs de PlayStation, Xbox, Switch, Epic y Battle.net, pero ahora solo se ven como insignias. Esto les da un uso real.

Pasos previstos

1. Modelo LFGPost (usuario, juego, plataforma, cuándo, idioma, micrófono, texto) que caduca a las pocas horas.
2. Solo en juegos con modos multijugador (necesita los datos de la ficha más completa).

3. Página de búsquedas activas filtrable por juego y plataforma, y aviso a quien tiene el juego en su biblioteca.
4. Botón de denunciar y límite de publicaciones por usuario.

REVISIÓN DEL 01/11/2026

Cómo está hoy
Sin empezar. Depende de los modos multijugador (838 juegos en IGDB) y de «Bloquear, silenciar y quitar seguidores»: sin bloqueo no lo abriría a desconocidos.

Cómo lo haría
1. LFGPost (usuario, juego, plataforma, cuándo, idioma, micrófono, plazas, texto y caducidad, 24 h como mucho). Nightly borra los caducados.
2. Solo con juegos que tienes y que tienen multijugador.
3. Página /grupos/ filtrable por juego y plataforma, y un bloque en la ficha de esos juegos.
4. Contacto: «Me apunto» enseña sus ID de plataforma y Discord, y el autor ve quién se ha apuntado. No se expone nada a quien solo mira.
5. Límite de publicaciones por usuario, denuncias con Report y avisar solo a quien lo pida para ese juego.

Para profundizar
- Sin chat propio, a propósito: moderarlo es caro y arriesgado. Llevar a la gente a Discord o a su plataforma.
- Emparejar por nivel: con las horas verificadas, «busca gente con menos de 20 h» o «ya te lo has pasado»; es algo que los foros no pueden hacer.
- Partidas con fecha: «sábado a las 22:00», añadible a tu calendario, con un aviso una hora antes.
- Empezar solo entre gente que se sigue: mucho menos arriesgado que abrirlo a todo el mundo, y da una versión pequeña que ya sirve.

---

60. Joyas ocultas y juegos en los que la comunidad discrepa

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es
Una página de descubrimiento con los juegos poco populares que la comunidad valora muy alto, y otra con los que más difieren entre la nota de la crítica y la de los jugadores.

Por qué
Usa solo datos que ya existen (nota de la crítica, valoración de la comunidad y popularidad), así que es barato y da contenido que no ofrece IGDB.

Pasos previstos
1. Consulta con un mínimo de valoraciones para que no salgan juegos con un solo voto.
2. Comparar la nota de la crítica (0-100) con la de la comunidad pasada a la misma escala.
3. Dos listados en el catálogo: «Joyas ocultas» y «La comunidad no está de acuerdo».

Notas
Tiene más sentido cuando haya bastantes usuarios valorando.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Su nota dice que hacen falta muchos usuarios, y no es verdad: IGDB tiene nota de usuarios (rating) en 2148 de 2206 juegos, además de la de la crítica. Se puede hacer hoy, y no se guarda porque no está en GAME_FIELDS.

Cómo lo haría

1. Game.user_rating y Game.user_rating_count con fill_game_fields, y añadir los dos a GAME_FIELDS.
2. Joyas: popularity en la parte baja del catálogo y nota de 80 o más, con un mínimo de votos.
3. Discrepancias: diferencia de 15 puntos o más entre crítica y usuarios, en dos listados, uno hacia cada lado.
4. Páginas /descubrir/joyas/ y /descubrir/discrepancias/, paginadas y enlazadas desde el catálogo. Cuando NextPlay tenga bastantes notas, se suma su media a las de IGDB.
5. Ojo con el catálogo: hoy se llena por popularidad, así que al ampliarlo saldrán más joyas de verdad.

Para profundizar

- A tu medida: joyas de tus géneros que no tienes, en un bloque del inicio que cambie cada semana.
- Joyas baratas: las que además están rebajadas o por debajo de cierto precio. Es lo que hace pasar de mirar a comprar.
- Joyas en castellano: filtrar por spanish_audio o spanish_text, algo que no se puede buscar en casi ningún otro sitio.
- Como lista mensual: generarla sola como GameList pública de una cuenta de NextPlay, para que salga en el feed y se pueda comentar.

---

61. Cuándo se abandona, si mejora con las horas y cuánto sale la hora

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Con las horas importadas de los usuarios, enseñar en la ficha a cuántas horas se suele abandonar un juego, si la nota sube con las horas («a quien pasa de las 10 h le gusta mucho más») y cuánto sale cada hora al precio de hoy.

Por qué

Son datos que solo tiene quien conoce las horas reales de su comunidad; Backloggd solo hace medias de lo que la gente apunta a mano (desde octubre de 2025). Responden a dudas típicas antes de comprar o de dejar un juego: «¿mejora luego?» y «¿vale lo que cuesta?».

Pasos previstos

1. Por juego, mediana de horas de quienes lo tienen en «Abandonado» (y, con el diario, de quienes llevan meses sin tocarlo).
2. Nota media por tramos de horas (menos de 2 h, de 2 a 10 h y más de 10 h) para el «¿mejora con las horas?».
3. Precio por hora: mejor precio de hoy entre la mediana de horas de quienes lo han jugado («sale a 0,40 € la hora»).
4. En la biblioteca, cuánto te ha salido a ti la hora si apuntas lo que pagaste (el campo de precio pagado de «Colección física con escáner de código de barras»).

Notas

Necesita bastantes usuarios por juego: enseñarlo solo a partir de unos 20. La mediana de horas de quienes lo completan ya está en «Duración estimada de los juegos».

REVISIÓN DEL 01/11/2026

Cómo está hoy

Casi todo necesita usuarios (20 por juego), con una excepción: el precio por hora se puede dar ya con la duración de IGDB (1523 juegos) y el mejor precio.

Cómo lo haría

1. Empezar por el precio por hora: mejor precio entre la duración normal, «sale a 0,40 € la hora de historia», en la ficha y en la lista de deseos, diciendo de dónde sale.
2. library/insights.py: cálculo nocturno por juego guardado en un JSON en Game, a partir de 20 personas. Nunca al abrir la ficha.
3. Mediana de horas de quienes lo tienen en Abandonado y nota por tramos de horas (menos de 2, de 2 a 10 y más de 10).
4. En la ficha solo cuando haya datos; sin datos no se enseña nada.
5. Tu coste real por hora, cuando exista el precio pagado de «Colección física con escáner de código de barras».

Para profundizar

- Abandono sin depender de que lo marquen: sin sesiones en 6 meses y con menos de la mitad de la duración. La gente rara vez marca Abandonado.
- Abandono temprano con logros: en Steam, la caída entre los primeros logros da una idea desde hoy, sin usuarios propios.
- «¿Mejora con las horas?» con las reseñas: cada una guarda sus horas y su nota (Review.hours y rating). Una curva de nota según horas es un dato que solo se puede sacar con horas verificadas.
- Ordenar la lista de deseos por precio por hora: una forma distinta de decidir en rebajas.
- Historial de tu nota: hoy LibraryEntry.rating se sobrescribe. Guardar cada cambio con sus horas enseña «le diste 4,5 a las 5 h y 3 a las 40 h», a ti y, con bastante gente, como curva del juego.

---

62. Colección física con escáner de código de barras

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 17/10/2026

Qué es

Apuntar en qué plataforma y formato tienes cada juego (físico, digital, suscripción) y añadir juegos físicos escaneando el código de barras de la caja con el móvil.

Por qué

Nintendo Switch no tiene ninguna forma de importar la biblioteca y mucha gente tiene colección física. Además, la biblioteca no guarda en qué plataforma tienes cada juego.

Pasos previstos

1. Campos nuevos en la biblioteca: plataforma, formato, edición y, opcional, precio pagado.
2. Página de escaneo para móvil con la API BarcodeDetector del navegador (y una librería de respaldo para iPhone, que no la tiene).
3. Tabla compartida de códigos de barras: la primera persona que escanea un juego elige cuál es y a partir de ahí sale solo para todos.
4. Contador de la colección por plataforma en el perfil.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. LibraryEntry no guarda plataforma ni formato, aunque ImportedTitle.platforms ya sabe dónde se jugó lo importado.

Cómo lo haría

1. OwnedCopy (usuario, juego, plataforma, formato, edición, precio pagado y fecha), aparte de LibraryEntry: se pueden tener varias copias del mismo juego.
2. Lo digital se deduce de las importaciones: no pedirlo a mano.
3. Página /escanear/ con BarcodeDetector y una biblioteca de respaldo (ZXing) para iPhone. Requiere HTTPS para la cámara, así que en local solo se prueba desde localhost o con un túnel.
4. BarcodeMapping compartido (código y juego), con origen y bloqueo como ExternalTitle: el primero que lo escanea elige y sirve para todos.
5. Contador de la colección por plataforma en una sección nueva del escaparate.

Para profundizar

- Foto de la estantería: un modelo de visión lee los lomos y propone los juegos, con pantalla de revisión. Cincuenta juegos en una foto, y es el mismo lector que el de «Importar Nintendo Switch con capturas de la app Nintendo Store».
- Préstamos: «se lo dejé a Luis», con recordatorio. Pequeño y muy de jugadores.
- Formato automático: según psn-api, PlayStation dice si un juego viene de PS Plus (service). Comprobarlo; evitaría preguntarlo.
- Coste por hora real al cruzar el precio pagado con las horas importadas, y el valor total de tu colección a precio de hoy.

---

63. ¿Qué edición compro? Precios de cada edición y remaster

PRIORIDAD BAJA · PENDIENTE · Funcionalidad · propuesta el 21/10/2026

Qué es

Comparar en la ficha el precio de cada edición y remaster que se vende («la Edición completa con los dos DLC cuesta 3 € más») y avisar si ya tienes otra versión del juego.

Por qué

NextPlay ya guarda las ediciones y remasters de IGDB con sus IDs de tienda (store_alternatives), pero solo los usa cuando el original no se vende. Backloggd junta las ediciones en una ficha sin precios, y qué versión comprar es una de las dudas más típicas en rebajas.

Pasos previstos

1. En update_prices, pedir los precios de todas las ediciones y no solo de la primera que tenga ofertas (al menos en los juegos más populares).
2. Bloque «Ediciones» en la ficha con el mejor precio de cada una y la diferencia con la estándar.
3. Si el usuario ya tiene el juego y existe un remaster, decirlo: «tienes el original; el remaster está a 19,99 €».

Notas

Multiplica las llamadas a IsThereAnyDeal de los juegos con ediciones: limitarlo o pedirlo al abrir la ficha, como el historial.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Sin empezar. prices.update_prices para en la primera edición con ofertas, y store_alternatives ya está guardado.

Cómo lo haría

1. No en nightly, porque multiplicaría las llamadas: pedirlo al abrir la ficha, con HTMX y un día de caché, como games.views.price_history.
2. games/editions.py: [{nombre, mejor precio, diferencia}] con _lookup y _fetch_prices de games/prices.py, sin guardar GamePrice.
3. Bloque «Ediciones» en «Dónde comprarlo», solo si hay más de una a la venta, con la diferencia sobre la estándar.
4. Si tienes el original y hay remaster (remasters está en 278 juegos): «Tienes el original; el remaster está a 19,99 €».

Para profundizar

- Qué incluye: con los dlcs de IGDB, «incluye 2 DLC». Y comparar la completa con la base más sus DLC comprados por separado en su mínimo.
- ¿Merece el remaster?: nota del original frente a la del remaster, y el precio de pasar de uno a otro.
- Cuál viene en castellano: a veces solo una versión lo trae, y Steam lo dice por app.
- Aviso en la lista de deseos cuando la edición completa baja más que la base.

---

64. Miniatura ya encuadrada de la foto de perfil para las listas

PRIORIDAD BAJA · PENDIENTE · Rendimiento · propuesta el 22/10/2026

Qué es

Al guardar el perfil, generar una versión pequeña (por ejemplo, 96 × 96) de la foto subida con su encuadre ya aplicado, para la barra superior, las reseñas y el feed; la foto entera queda para el perfil y el editor.

Por qué

Desde que las fotos se guardan enteras (hasta 800 px) para poder encuadrarlas, en las listas se descarga la foto grande para enseñarla a 40 px. Con pocos usuarios no se nota, pero un feed largo con muchas fotos distintas pesaría bastante más.

Pasos previstos

1. Al guardar, recortar con Pillow la parte que deja ver el encuadre (o la imagen entera con su fondo desenfocado) y guardarla aparte en WebP pequeño.
2. Usarla en accounts/_avatar.html con los tamaños sm y md.
3. Rehacerla cuando cambien la foto o su encuadre, y borrar del disco la anterior.

Notas

Sale del encuadre de imágenes del 22/10/2026.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Igual: accounts/_avatar.html carga la foto entera (hasta 800 px) en cualquier tamaño. Cambiaría su plan: recortar en el servidor obliga a repetir en Python los cálculos de zoom y posición que ya hace el CSS.

Cómo lo haría

1. Una copia entera pequeña (240 px) con el mismo encuadre por CSS: cambiar el encuadre no obliga a regenerarla y no hay matemáticas duplicadas. Con el zoom máximo se verá algo más blanda, y a 40 px apenas se nota.
2. Profile.avatar_thumb, y «avatar_thumb» en accounts.images.KINDS; se genera en ProfileForm.save y se borra la anterior.
3. avatar_image devuelve la pequeña para los tamaños sm y md; lg sigue con la grande. accounts.signals.delete_profile_images también la borra.
4. Un comando para generarla a quienes ya tienen foto.

Para profundizar

- Medir antes con Lighthouse en la portada: las portadas de juegos (t_cover_big para todo) pesan bastante más que los avatares, y ahí se ahorra de verdad.
- Cabecera y fondo en dos tamaños, con srcset: en el móvil se descarga una imagen de hasta 2000 px para una banda de 340.
- AVIF además de WebP y Cache-Control largo: los nombres aleatorios hacen que estas imágenes nunca cambien.

---

65. Traducir al español los comentarios antiguos en inglés

PRIORIDAD BAJA · PENDIENTE · Documentación · propuesta el 17/10/2026

Qué es

Pasar al español los comentarios sueltos (#) del código que ya existía antes de esta sesión.

Por qué

Todo el código nuevo se comenta en español, pero los comentarios del código anterior (modelos, IGDB, precios...) siguen en inglés.

Pasos previstos

1. Recorrer accounts, games, library y config y traducir los comentarios # y de plantillas.
2. No tocar las docstrings, que ya están en español.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Es mucho menos de lo que parecía: unos 38 comentarios en 10 archivos (15 en games/models.py y 6 en games/igdb.py; recuento con un script, puede faltar alguno) y ninguno en plantillas.

Cómo lo haría

1. Traducirlos de una sentada, sin tocar nada de código.
2. Comprobar en el diff que solo cambian líneas de comentario.
3. Lanzar los tests, regenerar el PDF de funciones y pasar a «hecha» en un commit pequeño. Se hace en un rato.

Para profundizar

- Que no vuelva: el mismo script como test del proyecto, avisando de comentarios que parecen en inglés.
- Mensajes internos: algunos errores siguen en inglés, como el ImproperlyConfigured de config/settings.py.
- Un test que compruebe que toda función tiene docstring: la regla ya existe y generar_pdf.py depende de ella.

66. Perfil muy personalizable: foto, cabecera, fondo, colores y escaparate

HECHA · Funcionalidad · propuesta el 21/10/2026

Qué es

Que cada usuario deje su perfil a su gusto: foto de perfil propia, cabecera con una imagen o con el arte de un juego, fondo de página, color del perfil, diseño de la cabecera, forma de la foto, estado con emoji, pronombres, ubicación, redes sociales, juegos favoritos, reseña destacada y qué secciones se ven y en qué orden.

Por qué

El usuario quiere un perfil muchísimo más personalizable que el de Backloggd. Ahora en NextPlay solo se cambian el nombre visible y la biografía, y la foto sale de Steam, PlayStation o Xbox.

Pasos previstos

1. Subida de imágenes con Pillow: se comprueban, se recortan, se pasan a WebP y se les quitan los datos EXIF (por ejemplo, la ubicación de las fotos del móvil).
2. Foto de perfil propia o elegir cuál se enseña (la subida, Steam, PlayStation, Xbox o solo la inicial) y su forma (redonda, redondeada o hexágono).
3. Cabecera con una imagen subida o con el arte de un juego de la biblioteca (artworks y capturas de IGDB).
4. Fondo de la página del perfil (ninguno, el de la cabecera o una imagen propia) con un control para oscurecerlo.
5. Color del perfil (paleta o el que quieras) y diseño de la cabecera (clásico o centrado).
6. Estado con emoji, pronombres, ubicación y enlaces a Twitch, YouTube, X, Bluesky, Instagram, TikTok, Discord y una web.
7. Escaparate: hasta 5 juegos favoritos (se marcan desde la ficha del juego), reseña destacada y orden y visibilidad de las secciones.
8. Vista previa en directo en Editar perfil.

Notas

Las imágenes se guardan en MEDIA_ROOT; en producción habrá que servirlas aparte o con un almacenamiento externo (por ejemplo, S3). Hecha el 21/10/2026 (aún sin commit): modelo FavoriteGame y campos nuevos en Profile, accounts/images.py, Editar perfil con vista previa en directo, selector de imágenes de IGDB con HTMX, botón de favoritos en la ficha, escaparate en el perfil (solo en su vista principal) y 27 tests nuevos. Probada en el navegador con la cuenta tester, también en móvil. El 22/10/2026, a petición del usuario, la vista previa pasó de enseñar solo la cabecera a enseñar el perfil entero a la vez en escritorio (1280 px) y en móvil (390 px): el servidor pinta la página de verdad con lo que hay en el formulario, sin guardar nada (10 tests nuevos). Ese mismo día, también a petición suya, las imágenes dejaron de recortarse solas: se guardan enteras y cada una tiene su encuadre (arrastrarla en su marco, zoom o «entera» con una copia desenfocada en el hueco), con marcos de escritorio y de móvil para la cabecera y el fondo; el navegador reduce antes de subirlas las que no caben.

67. Reseñas completas: medias estrellas, una por partida y página propia

HECHA · Funcionalidad · propuesta el 21/10/2026

Qué es

Llevar las reseñas al nivel de las de Backloggd sin copiar su diseño: valoraciones con medias estrellas, varias reseñas del mismo juego (una por partida o rejugada) con plataforma y fechas, una página propia para enlazar cada reseña, texto con formato (negrita, cursiva, enlaces y spoilers dentro del texto) y borradores.

Por qué

Al usuario le parece que la parte de reseñas de Backloggd es la mejor. Ahora en NextPlay hay una sola reseña por juego y usuario dentro de la biblioteca, con estrellas enteras del 1 al 5, y solo se lee en la ficha y en el perfil.

Pasos previstos

1. Valoración en medias estrellas (de 0,5 a 5), guardada como un número del 1 al 10 para no perder precisión; las notas que ya hay se multiplican por 2 en la migración.
2. Modelo de reseña aparte de LibraryEntry (usuario, juego, partida, plataforma, fechas de inicio y fin, horas, texto y spoilers) para poder reseñar un mismo juego varias veces.
3. Página propia de cada reseña (/u/<usuario>/resena/<id>/) con su juego, la valoración y, más adelante, sus me gusta y comentarios.
4. Formato sencillo y seguro: negrita, cursiva, enlaces y ||spoilers|| dentro del texto, sin permitir HTML.
5. Guardar como borrador antes de publicarla.
6. Mejoras propias: las horas con las que se escribió y los spoilers según tu progreso (ver «Reseñas con horas verificadas y spoilers según tu progreso»).

Notas

Las reseñas son un patrón común (Letterboxd, Goodreads, Steam): se pueden hacer igual de completas con un diseño y unos textos propios. Va junto con «Me gusta y comentarios en las reseñas». Hecha el 23/10/2026 en la rama resenas-completas y fusionada con main, a petición del usuario («son super simples»): modelo Review aparte de LibraryEntry con varias reseñas por juego (plataforma, fechas, rejugada), notas de 1 a 10 medias estrellas también en la biblioteca y en la media de la comunidad, editor propio con barra de formato, vista previa en directo y borradores, formato seguro (library/markup.py: negrita, cursiva, enlaces y ||spoilers|| en línea), página propia /u/<usuario>/resenas/<id>/ con «Copiar enlace», y al publicar se guardan las horas y el estado de la biblioteca. Las migraciones pasan solas las reseñas y notas que había (las horas de las reseñas viejas son las de ese momento). 13 tests nuevos.

---

68. Reseñas: lo mejor y lo peor, datos de la partida rellenos solos e imágenes

HECHA · Funcionalidad · propuesta el 23/10/2026

Qué es

Tres mejoras del editor de reseñas que propuso el usuario: una lista de «Lo mejor» y otra de «Lo peor», que la plataforma y las fechas de la partida se rellenen solas con lo que dejan las importaciones de Steam, PlayStation y Xbox, y poder añadir imágenes.

Por qué

«Lo mejor / Lo peor» es el formato de las revistas de videojuegos de aquí y deja leer una reseña de un vistazo. Las importaciones ya reciben la plataforma (PS4 o PS5 en PlayStation, la consola o el PC en Xbox) y las fechas de partida, pero ahora solo se usan para emparejar y se tiran. Las imágenes hacen las reseñas más vivas y Backloggd no las tiene.

Pasos previstos

1. Lo mejor y lo peor: hasta 5 puntos cortos en cada lista (opcionales), con iconos en la tarjeta y en la página de la reseña; en la portada y en el perfil sirven de resumen cuando el texto es largo.
2. Mejora propia: al escribir un punto, sugerir los que ya han puesto otros en el mismo juego, para que se repitan las mismas frases y la ficha pueda enseñar «Lo que más destacan» con cuánta gente lo dice.
3. Autorrelleno de la plataforma: guardar en cada título importado dónde se jugó (PC en Steam, PS4 o PS5 por la categoría de PlayStation, los dispositivos de Xbox; y Steam Deck si GetOwnedGames trae playtime_deck_forever) y elegirla sola en el editor; si se jugó en varias, la de más horas, diciéndolo.
4. Autorrelleno de fechas: la primera partida de PlayStation (firstPlayedDateTime, que la importación recibe y ahora no lee) para «Lo empecé el» y, si el juego está como Completado, la última partida de

la plataforma para «Lo terminé el». Con el «Diario de partidas que se escribe solo» serían exactas.

5. Todo lo rellenado lleva al lado de dónde sale («De tu cuenta de PlayStation») y se puede cambiar; las horas y el estado ya se guardan solos al publicar.
6. Imágenes: hasta 4 por reseña, pasadas a WebP y sin EXIF con lo que ya usa el perfil (accounts/images.py), cada una marcable como spoiler (borrosa hasta pulsarla) y vistas en el visor a pantalla completa del carrusel. Se puede elegir también una captura oficial del juego (las del carrusel) sin subir nada.
7. Antes de abrir las imágenes al público: el botón de denunciar de «Me gusta y comentarios en las reseñas» y guardar los archivos fuera del servidor en producción.

Notas

Propuesta por el usuario el 23/10/2026. Orden recomendado: lo mejor y lo peor y el autorrelleno primero (poco trabajo, salen de datos que ya existen) y las imágenes después, compartiendo el modelo de imagen con «Publicaciones con imágenes asociadas a un juego» para no hacerlo dos veces. 23/10/2026: hechos en la rama resenas-completas, fusionada con main, lo mejor y lo peor (Review.pros y Review.cons, cinco casillas por lista, con formato), las sugerencias de lo que han escrito otros (library/points.py, con <datalist>), «Lo que más destacan» y «Lo que menos gusta» en la ficha (a partir de dos reseñas que coinciden) y el autorrelleno (library/autofill.py): ImportedTitle guarda ahora las plataformas y la primera partida; en lo importado antes, la consola de PlayStation sale del ID del juego (CUSA es PS4 y PPSA, PS5) y lo de Steam es PC. Probado con las importaciones reales de tester_steam (Apex Legends: elige PS4 con 134 h frente a 25 h en PC). La primera partida de PlayStation llegará con la próxima sincronización de PlayStation. Faltan las imágenes y Steam Deck. 24/10/2026: hechas en la rama resenas-imagenes-y-denuncias (aún sin commit) las imágenes y la Steam Deck. Imágenes: hasta 4 por reseña, con un modelo abstracto (AttachedImage) que podrán usar también las publicaciones; las subidas se pasan a WebP sin EXIF en dos tamaños (1920 px para el visor y una miniatura de 720 px) y el navegador reduce antes de subirlas las que no caben; cada una se puede marcar como spoiler (borrosa hasta pulsarla) y se puede elegir cualquier captura o arte oficial de la ficha sin subir nada. Se ven en la ficha, el perfil, la portada y la página de la reseña, con un visor a pantalla completa como el del carrusel, y sus archivos se borran al quitarlas o al borrar la reseña. Steam Deck: la importación de Steam guarda los minutos jugados en ella (playtime_deck_forever) y, en los juegos de PC, el editor la ofrece como plataforma (una propia, porque IGDB no la tiene) y la elige si es donde más se ha jugado. El botón de denunciar ya está (ver «Me gusta y comentarios en las reseñas»); guardar las imágenes fuera del servidor queda para cuando se publique NextPlay. 16 tests nuevos entre las tres propuestas de reseñas terminadas ese día.

---

69. Me gusta y comentarios en las reseñas

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Reaccionar a las reseñas con «me gusta», comentarlas y ordenar las de la ficha por las más útiles.

Por qué

Las reseñas ya existen, pero no hay forma de interactuar con ellas; es la interacción social más barata de añadir.

Pasos previstos

1. Modelo ReviewLike (usuario, entrada de biblioteca) y contador en la reseña.
2. Comentarios cortos con moderación básica (borrar los propios y que el autor pueda borrar los de su reseña).
3. Orden de reseñas en la ficha: más útiles / más recientes.
4. Botón de denunciar reseña para el admin.

Notas

Ya estaba planeada antes de proponerla. 21/10/2026: sube a alta porque el usuario quiere unas reseñas al nivel de las de Backloggd. 23/10/2026: hechos en la rama resenas-completas los «me gusta» (no en las tuyas, con HTMX), los comentarios con el mismo formato (los borran quien los escribe y el autor de la reseña), los avisos en la campana (un «me gusta» de la misma persona avisa una sola vez) y el orden «Más gustadas». Falta el botón de denunciar. 24/10/2026: hecho el botón de denunciar en la rama resenas-imagenes-y-denuncias (aún sin commit), para las reseñas y también para los comentarios: modelo Report genérico (con contenttypes, para que sirva luego para

publicaciones o grupos) con motivo y detalles, una denuncia por persona y cosa denunciada, aviso en la campana a los administradores y página de denuncias en el admin con filtro de pendientes, enlaces a lo denunciado y la acción «Marcar como revisadas». En las listas, el botón lleva a la página de la reseña con el formulario ya abierto.

---

70. Listas personalizadas de juegos

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Que cada usuario cree listas con nombre y descripción («Mejores metroidvanias», «Para jugar en pareja»), ordenadas y públicas o privadas.

Por qué

El README ya promete listas y aún no existen. Son el contenido que más se comparte en webs como Letterboxd o Backloggd.

Pasos previstos

1. Modelos GameList (usuario, título, descripción, pública) y GameListItem (lista, juego, posición, nota).
2. Crear, editar y reordenar desde el perfil (arrastrar con HTMX o botones subir/bajar).
3. Botón «Añadir a una lista» en la ficha del juego.
4. Mostrar en la ficha en qué listas públicas aparece el juego.
5. Mejoras sobre Backloggd: el progreso de quien la mira («has jugado 7 de 20»), modo tier list (S, A, B, C…), listas colaborativas con otros usuarios y portada en mosaico con las carátulas.

Notas

Ya estaba planeada antes de proponerla. 21/10/2026: sube a alta porque el usuario las quiere ya. 25/10/2026: con lo social ya hecho, crear una lista sale en el feed y en la actividad («Ana creó la lista…»), y los «me gusta» y los comentarios reutilizan los de las reseñas y las publicaciones (library/_comments.html). Mejora que solo puede tener NextPlay: «Lo que te falta», los juegos de la lista que no tienes con lo que cuestan hoy en tus tiendas y en su mínimo histórico (solo los que tienen precio: PC y Xbox) y un botón para pasarlos todos a «Lo quiero», con sus avisos de precio. Y «Convertir en reto», que crea un reto de tipo lista concreta (ver «Retos: crea los tuyos y únete a los de otros»). 25/10/2026: fase 1 hecha en la rama listas-personalizadas (aún sin commit), en la app nueva lists: modelos GameList (título, descripción con el formato de las reseñas, pública o privada y numerada) y ListItem (puesto y comentario); página de cada lista en /u/<usuario>/listas/<id>/ con el mosaico de las primeras portadas y el color del perfil de su autor, en portadas o con los comentarios; progreso de quien mira (jugados, terminados, horas y el estado de cada juego) y «Lo que te falta» con lo que cuesta hoy en sus tiendas y en sus mínimos históricos y un botón para pasarlo todo a «Lo quiero»; editor con buscador mientras se escribe, subir, bajar o escribir el puesto, comentario que se guarda solo y quitar, todo al momento con HTMX; «Añadir a una lista» en la ficha (desplegable con tus listas y «Nueva lista con este juego») y «En N listas», con una página con todas; pestaña Listas y sección del escaparate en el perfil; listas de prueba en seed_test_profile y 37 tests. De paso, las tarjetas de la biblioteca del perfil vuelven a pintar las estrellas de la nota (games/_card.html usaba un atributo que no existe). Falta la fase 2 (me gusta, comentarios y denuncias, y que crear una lista salga en la actividad y el feed) y la 3 (modo tier list, listas colaborativas y «Convertir en reto» cuando haya retos). 25/10/2026: fase 1 con commit en su rama (sin fusionar). Fase 2 hecha en la rama listas-me-gusta-y-comentarios, encima de la anterior (aún sin commit): «me gusta» (ListLike, con aviso en la campana una vez por persona), comentarios (ListComment, con library/_comments.html y comment_thread, como los de las reseñas y las publicaciones: avisan al dueño salvo si ya tiene uno igual sin leer y los borran quien los escribe y el dueño de la lista) y denuncias de la lista y de sus comentarios, con su admin; actividad de tipo lista (social.activity.list_created, list_games_added y list_game_removed: crearla y añadirle juegos seguido va junto, y quitar uno recién añadido lo descuenta) que sale en el feed y en la actividad del perfil con el mosaico, los «me gusta», los comentarios y lo que ha jugado quien mira; las privadas, solo para su dueño, y las vacías no salen. De paso, un comentario en blanco dice «Escribe algo en el comentario.» en vez del error genérico de Django (también en reseñas y publicaciones). «Me gusta», comentarios y actividad de prueba en seed_test_profile y 14 tests. Falta la fase 3 (modo tier list, listas colaborativas y «Convertir en reto» cuando haya retos). 26/10/2026: fase 2 con commit en su rama (sin fusionar). Fase 3 hecha en la rama listas-por-niveles-y-colaborativas, encima de la anterior (aún sin commit): el tipo de lista pasa a ser un campo (mode: lista, numerada o tier list; la migración convierte las numeradas); tier lists con hasta 8 niveles con nombre y color (S, A, B, C, D y F de serie, con los colores de NextPlay) guardados con un id fijo, botones de nivel en el editor y tablero por niveles en la página, con los sin clasificar al final; listas colaborativas: el dueño invita por nombre de usuario (hasta 10, con aviso en la campana y sugiriendo la gente a la

que sigue), quien acepta puede añadir, ordenar, poner en un nivel, comentar y quitar juegos aunque la lista sea privada, se dice quién colabora y quién añadió cada juego, sale en su pestaña Listas, en su «Añadir a una lista» y en su actividad, y puede irse cuando quiera. De paso, el color del texto sobre un color (accounts.models.text_on) sirve ya para el perfil y para los niveles. Tier list y colaboración de prueba en seed_test_profile y 13 tests. Falta «Convertir en reto», que espera a que existan los retos. 26/10/2026: antes de subir las listas, propuesto: que solo se pueda invitar a editar a quien te sigue (ahora se puede invitar a cualquiera y, creando varias listas, mandarle muchos avisos), avisar al dueño cuando alguien acepta la invitación (ahora no se entera) y que un color de nivel mal guardado desde el admin no rompa la página de la lista (accounts.models.text_on falla con un color que no sea «#rrggbb»). 26/10/2026: fase 3 con commit en su rama y «Convertir en reto» hecho con los retos, así que queda hecha. Esos tres arreglos pasan a su propia propuesta, «Listas colaborativas: invitar solo a quien te sigue y avisar al aceptar».

---

71. Listas: colocar las carátulas arrastrándolas

HECHA · Funcionalidad · propuesta el 28/10/2026

Qué es

En las tier lists, arrastrar cada carátula a su nivel (o a «Sin clasificar») y, dentro de él, a su sitio, en el tablero del editor y en el de la página de la lista, en vez de pulsar los botones de nivel de cada juego. En las listas normales y numeradas, arrastrar las filas del editor para ordenarlas en vez de subirlas y bajarlas una a una.

Por qué

Idea del usuario (28/10/2026): con botones, colocar una tier list de 20 o 30 juegos es lento y no se ve el tablero mientras se hace. Arrastrar es lo que se espera de una tier list y de un ranking.

Pasos previstos

1. GameList.place: pone un juego en un nivel y lo deja junto a sus vecinos de fila sin cambiar el orden de los demás; lists.views.tier lo usa cuando le llega orden=.
2. Un tablero compartido (lists/_tier_board.html) en el editor y en la página de la lista: quien puede editarla arrastra las carátulas con SortableJS (ratón y pantallas táctiles), y la fila «Sin clasificar» sale siempre para poder sacar juegos de su nivel.
3. En el editor de las listas normales y numeradas, un asa en cada fila para arrastrarla.
4. Los botones de nivel y los de subir y bajar se quedan para quien usa el teclado o un lector de pantalla.
5. Mientras se guarda no se puede arrastrar otra; si falla, la carátula vuelve a su sitio y sale el aviso.

Notas

28/10/2026: hecha en la rama listas-arrastrar y revisada por el usuario. SortableJS 1.15.6 se carga de jsDelivr solo en el editor y en la página de las tier lists de quien puede editarlas (lists/_drag.html), como htmx se carga de unpkg. Con el ratón se arrastra en cuanto se mueve y un clic sigue abriendo el juego; con el dedo hay que mantener pulsado un momento para que deslizar siga desplazando la página. Probado con el ratón en el navegador integrado (entre niveles, dentro de un nivel, a «Sin clasificar», en el editor y con un fallo simulado) y a 390 px; falta probarlo en un móvil de verdad. De paso, los fieldset ya no llevan el recuadro del navegador (ver «Detalles sueltos…»). 4 tests nuevos.

---

72. Emparejar los juegos por nombre sin contar signos: recuperar horas perdidas

HECHA · Funcionalidad · propuesta el 01/11/2026

Qué es

Que «inFAMOUS Second Son» encuentre «Infamous: Second Son» y «Outlast 2» encuentre «Outlast II»: comparar los nombres sin signos ni mayúsculas, también con los nombres alternativos y en España que ya se guardan, y aplicarlo a lo ya importado sin tener que volver a sincronizar.

Por qué

PlayStation y Xbox emparejan por nombre, y accounts.matching.match_by_name solo acepta nombres idénticos (normalize quita ™, ® y © y poco más). En tester_steam hay 33 títulos sin emparejar y 689 h que no cuentan en su biblioteca, ni en sus retos ni en sus estadísticas. Simulado sin guardar nada: con una comparación sin signos salen 21 títulos y unas 583 h, entre ellos Rainbow Six Siege (382,6 h; IGDB ahora lo llama así, sin «Tom Clancy's»), The Witcher 3 Remastered (61,4 h) y Sniper Ghost Warrior 3 (27,6 h y su platino). Es la primera impresión de cualquiera que importe de PlayStation o Xbox, y psn.trophy_key ya resuelve lo mismo para los trofeos.

Pasos previstos

1. Llevar la clave de psn.trophy_key a accounts.matching (sin signos ni espacios) y usarla en match_by_name, también al comparar lo que devuelve IGDB, con sus alternative_names, que ya llegan en GAME_FIELDS.
2. En el catálogo local, buscar también por name_es y search_names: un modelo GameAlias (juego y clave, con índice), relleno en upsert_games y por una migración, para no cargar el catálogo entero cuando crezca.
3. Desempates: _fits y la popularidad siguen decidiendo; si el título trae el año («resident evil 4 (2005)»), gana el juego de ese año, que es el original y no el remake.
4. Comando rematch_titles: vuelve a buscar los títulos automáticos sin juego y recoloca las horas con update_library, sin pedir NPSSO ni iniciar sesión en Microsoft. Respeta LAUNCHERS y lo corregido a mano.
5. Tests con estos casos reales.

Notas

Sale de medir el 01/11/2026. Seguirían sin salir Genshin Impact 6.º aniversario, Trilogía Mafia, Tina Chiquitina asalta la mazmorra del dragón, LittleBigPlanet3 (LATAM), Devil May Cry 5 Series y cosas que no son juegos (servidores de prueba, SHAREfactory). 01/11/2026: hecha a petición del usuario, aún sin commit, en la rama trofeos-por-id porque comparte accounts/psn.py con los trofeos. accounts.matching: name_key (sin signos, espacios, tildes ni mayúsculas; psn.trophy_key la usa), match_by_name compara también con el nombre en España, los alternativos y los títulos ya emparejados de otras plataformas (_seen_games), el año del título elige entre los que se llaman igual y LAUNCHERS y YEAR_OR_REGION pasan aquí, así que Xbox tampoco empareja el lanzador de Call of Duty. Comando rematch_titles, con --dry-run. No hizo falta el modelo GameAlias del paso 2: recorrer los nombres de los 2206 juegos tarda unos 60 ms y solo se hace si hay títulos que buscar, sin migración ni nada que se pueda quedar viejo; habrá que guardar las claves si el catálogo llega a cientos de miles. La prueba en seco enseñó un error: buscando en IGDB, The Witcher 3: Wild Hunt — Remastered (61,4 h en PS4, de 2020 y 2021) salía como The Witcher 3: Wild Hunt Remastered, una actualización de 2026 sin versión de PS4. Por eso lo que se encuentra por otro nombre o buscando tiene que salir en la plataforma en la que se jugó, y el mismo nombre ya emparejado en Steam da el juego bueno. Aplicado a tester_steam: 22 títulos emparejados, de 348 a 361 juegos, 585 h más de PlayStation, de 4 a 5 platinos (Sniper Ghost Warrior 3) y de 33 títulos y 689 h sin emparejar a 11 títulos y 104 h. De esos 11, a mano: Genshin Impact 6.º aniversario, Trilogía Mafia, Tina Chiquitina asalta la mazmorra del dragón y Devil May Cry 5 Series; el resto no son juegos (servidores de prueba, SHAREfactory y el lanzador de Call of Duty). 23 tests nuevos, 445 en total, todos bien. 02/11/2026: con commit y fusionada con main, junto con los trofeos por ID y las horas de cada plataforma.

REVISIÓN DEL 01/11/2026

Cómo está hoy

Hecha el 01/11/2026 y aplicada a tester_steam: 22 títulos más emparejados y 585 h más en su biblioteca. Los detalles, en sus notas.

Para profundizar

- Proponer en vez de esperar: en /cuenta/juegos-importados/ hay que pulsar para buscar cada título. Enseñar ya el candidato más parecido («¿Es Genshin Impact?») y confirmar con un clic; la corrección sirve para todos, como ya hace ExternalTitle.
- Números romanos y arábigos: «Outlast 2» y «Ghostrunner2» salen hoy gracias a los nombres alternativos; una regla que iguale 2 y II cubre los juegos que no los tengan.

- Medir el emparejado: el porcentaje por plataforma en el admin (hoy, un 84 % de 178 títulos en PlayStation y un 97 % de 269 en Steam), para saber si un cambio de Sony o de IGDB lo empeora.

73. Horas de cada plataforma, a la vista

HECHA · Funcionalidad · propuesta el 02/11/2026

Qué es

Decir de qué plataforma salen las horas de cada juego: en la ficha, una fila por plataforma con sus horas y su última partida; en las portadas del perfil, el logo de la plataforma junto a las horas o, si se ha jugado en varias, el total y debajo las de cada una; y en las cifras del perfil, el reparto de las horas jugadas.

Por qué

Idea del usuario (02/11/2026), al ver que Rainbow Six Siege sumaba 622,4 h tras emparejar sus títulos de PlayStation: no se sabía cuántas eran de Steam (239,8 h) y cuántas de PlayStation (382,6 h). LibraryEntry ya guardaba las horas y la última partida de cada plataforma por separado, pero la web solo enseñaba el total.

Pasos previstos

1. LibraryEntry.hours_by_platform: las horas de cada plataforma con algo jugado, de la que más a la que menos, con su última partida. Sustituye a steam_hours, que solo usaban los tests.
2. Logos dibujados una sola vez por página (library/_platform_symbols.html, en base.html) y reutilizados con <use> (library/_platform_mark.html): repetir el SVG entero en las 361 portadas de una biblioteca habría casi triplicado el peso del perfil.
3. Ficha: «Tus horas» en el widget de la biblioteca, antes de «Tus logros», con el total si son varias plataformas.
4. Portadas del perfil: el logo junto a las horas o, con varias plataformas, el total y debajo las de cada una, con el reparto también al pasar el ratón. «Jugando ahora» también lleva los logos.
5. Cifras del perfil: debajo de «Horas jugadas», las de cada plataforma cuando son de más de una (accounts.views._profile_stats).

Notas

Hecha el 02/11/2026 en la rama trofeos-por-id y fusionada con main. Comprobada en el navegador con tester_steam, en escritorio y a 360 y 375 px. De paso, las estrellas y las horas de una portada ya bajan de línea si no caben: en las tres columnas del móvil se salían 9 px de su hueco. 5 tests nuevos, 450 en total, todos bien.

REVISIÓN DEL 02/11/2026

Cómo está hoy

Hecha. En tester_steam hay 43 juegos con horas de más de una plataforma, y el perfil reparte sus 11.465 h en 7560 de PlayStation y 3905 de Steam.

Para profundizar

- Steam Deck dentro de Steam: ImportedTitle.deck_minutes ya dice cuánto se jugó en la Deck, así que la fila de Steam podría decir «de ellas, 12 h en Steam Deck».
- PS4 y PS5 por separado: cada título de PlayStation sabe en qué consola se jugó (ImportedTitle.platforms), así que la fila de PlayStation podría separar las dos.
- Filtrar la biblioteca del perfil por plataforma («lo que he jugado en PlayStation»), ordenada por las horas de esa plataforma.
- En las estadísticas, el reparto por plataforma de cada año, con PlaySession.platform.

74. Retos: crea los tuyos y únete a los de otros

HECHA · Funcionalidad · propuesta el 21/10/2026

Qué es

Que cualquiera cree un reto («Termina 5 juegos de terror en octubre», «Pásate todos los Souls», «20 juegos en 2027») con sus reglas y fechas, que otros se unan y que el progreso se cuente solo con la biblioteca de cada uno.

Por qué

El usuario quiere retos además de listas. Backloggd no tiene retos, y en NextPlay el progreso puede ir solo porque los estados y las horas ya se importan de Steam, PlayStation y Xbox.

Pasos previstos

1. Modelo de reto (creador, título, descripción, fechas, público o privado) con tres tipos: una lista de juegos concreta, un número de juegos que cumplan un filtro (género, plataforma o año) o un número de horas.
2. Unirse a un reto y barra de progreso calculada con la biblioteca: cuenta lo que se completa entre las fechas del reto.
3. Guardar la fecha en que un juego pasa a Completado (ahora solo hay updated_at), que también necesitan las estadísticas.
4. Insignia en el perfil al terminarlo y sección «Retos» en el escaparate del perfil.
5. Página de retos con los más populares y los de la gente a la que sigues.

Notas

El reto anual de «Retos del año y control del pendiente» pasa a ser un tipo más de este; lo del valor del pendiente y el asistente siguen allí. 25/10/2026: visto al revisar el código. El tipo «número de horas» necesita saber cuántas horas se jugaron entre las fechas del reto, y eso solo lo dan las sesiones del paso 1 de «Diario de partidas que se escribe solo» (ahora ImportedTitle.minutes se sobrescribe en cada importación), así que ese paso va antes o con los retos. La fecha de Completado es más fácil de lo que parecía: las importaciones solo ponen Pendiente, Jugando o Jugado (LibraryEntry.imported_status), así que Completado siempre lo pone el jugador y basta con guardar la fecha al cambiar el estado; la actividad ya la tiene desde el 25/10/2026. El tipo «lista de juegos concreta» se apoya en «Listas personalizadas de juegos», que conviene hacer antes. 26/10/2026: fase 1 hecha (aún sin commit; irá en su propia rama, encima de la fase 3 de las listas), en la app nueva challenges: modelos Challenge (tres tipos: juegos concretos, que se copian de una lista con «Convertir en reto»; número de juegos con filtros de género, plataforma y años; y horas; fechas opcionales; público o privado; las reglas no cambian después de crearlo) y Participation (con cuándo lo consiguió); el progreso se cuenta solo (challenges.progress) con LibraryEntry.completed_at (nuevo: lo pone set_status al pasar a Completado y la migración lo rellena con la actividad o el «terminado el» de las reseñas) y con las sesiones de juego del paso 1 del diario (library.models.PlaySession); páginas /retos/ (los tuyos con tu progreso, los de la gente que sigues, los más populares y los nuevos), la de cada reto (tu progreso, sus juegos y la clasificación, solo con perfiles visibles), crear, editar, borrar, unirse y dejarlo; «Retos» en la cabecera y en los menús; retos y sesiones de prueba en seed_test_profile y 16 tests. Falta la fase 2: insignias y sección «Retos» en el perfil, que unirse y conseguirlos salga en la actividad y el feed, y avisos. 26/10/2026: fase 1 con commit en la rama retos. 04/11/2026: fase 2 hecha en la rama retos-fase-2, encima de avisos-en-un-solo-sitio y sin subir. Sección «Retos» en el escaparate del perfil (SECTIONS), con las insignias de los conseguidos y los que lleva en marcha con su barra (challenges.progress.showcase); a los demás les cuenta el progreso en tercera persona y no les enseña los privados. progress.mark_completed devuelve ya a quién ha apuntado y a cada uno le avisa («¡Has conseguido el reto…!») y le apunta «consiguió el reto» en la actividad (Activity.Kind.CHALLENGE, con el reto en Activity.challenge). Crear un reto y unirse también salen en la actividad, junto con conseguirlo si va seguido, y dejarlo la borra; de los privados solo ve algo quien los crea. A quien crea un reto se le avisa de cada persona que se une, una vez por persona. Repaso de cada noche, el comando close_challenges dentro de nightly, justo después de sincronizar (challenges.closing): apunta a quien ha llegado al objetivo sin que se hubiera mirado, avisa a quien le quedan 3 días o menos en uno que aún no ha conseguido («Te quedan 3 días para el reto…: llevas 1 de 2 juegos»), y, pasados 7 días de margen tras su fin (progress.GRACE_DAYS), avisa a cada uno de cómo ha quedado («lo conseguiste y quedaste 1.º de 2») y lo cierra (Challenge.closed_at). progress.check cuenta ya las sincronizaciones que llegan en esos 7 días. La página del reto ya no guarda nada al abrirla: quien ha llegado al objetivo sale como conseguido aunque aún no esté apuntado. Los avisos de los retos tienen su grupo en Editar perfil › Avisos. 7 tests nuevos (522 en total, todos bien sin claves y con claves falsas).

REVISIÓN DEL 04/11/2026

Cómo está hoy

Hechas las dos fases. Queda fuera, porque es de otras propuestas: proponer «¿Lo has terminado?» al llegar a las horas o al logro del final (va con el repaso de «Retos del año y control del pendiente», que es lo que llena Completado) y juntar en uno los avisos de varias personas que se unen a un reto («Ana y 3 más…», en «Un solo sitio para crear los avisos»). El repaso de cada noche no corre hasta que programes nightly al desplegar; mientras, lo apuntan al momento la biblioteca, las importaciones y unirse.

Para profundizar

- Verificado o declarado: Completado lo pone el jugador. Con los logros y la duración, un sello en la clasificación para quien lo respalda con el logro final o con horas cerca de la duración normal, y un filtro «solo verificados». Sin prohibir nada.
- Retos de bingo: un cartón de 5 × 5 cuyas casillas son filtros (uno de antes del 2000, uno de menos de 5 h, uno en castellano…) y cada juego vale para una sola. _game_filters ya sabe evaluar género, plataforma y años, y con datos verificados es difícil de copiar.
- Retos de logros («consigue 10 logros de menos del 5 %») y retos anti-compras («no añadas nada a la biblioteca hasta terminar 3 de tu pendiente»), que se rompen solos si una importación trae juegos nuevos.
- Reto personal de la semana generado con tu pendiente («te quedan 5 juegos de menos de 8 h: termina uno») y duelos 1 contra 1 con la clasificación que ya existe.

---

75. Revisar los juegos que no se emparejaron al importar

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Una página con los juegos importados que no se encontraron en el catálogo (o que se emparejaron mal) para asignarlos a mano, y que esa corrección sirva para todos los usuarios.

Por qué

Steam solo avisa de cuántos no se encontraron y PlayStation y Xbox emparejan por nombre quedándose con el más popular, lo que falla con remakes que se llaman igual (God of War de 2005 y de 2018, Resident Evil 4, Dead Space...).

Pasos previstos

1. Modelo ExternalTitle (plataforma, ID externo, nombre, juego) donde se guarda cada título importado, emparejado o no.
2. Al importar, mirar primero esa tabla antes de buscar por nombre.
3. Página «Juegos sin emparejar» con un buscador para asignar cada uno, y botón «Este no es mi juego» en la biblioteca.
4. Las asignaciones de un usuario se aplican a los demás; el admin puede revisarlas y bloquear las malas.
5. Para remakes, desempatar por plataforma (un juego de PS5 no puede ser el de PS2).

Notas

Hecha el 18/10/2026: modelos ExternalTitle e ImportedTitle, página /cuenta/juegos-importados/ con buscador (catálogo y, si salen pocos, IGDB), «¿No es este el juego que jugaste?» en la ficha, ocultar herramientas y demos, admin para revisar, corregir al momento y bloquear, y desempate de remakes por plataforma (PS4/PS5 en PlayStation, dispositivos en Xbox). Las correcciones de un usuario llegan a los demás en su próxima importación; las del admin, al momento.

76. Lista de deseos con alertas de precio

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Marcar juegos como «Lo quiero» y recibir un aviso (en la web y por correo) cuando bajen de un precio o lleguen a su mínimo histórico.

Por qué

Ya se guardan precios de varias tiendas y el mínimo histórico cada día con update_prices; falta sacarle partido para el usuario.

Pasos previstos

1. Nuevo estado «Lo quiero» en la biblioteca o modelo Wishlist aparte con precio objetivo opcional.
2. Al terminar update_prices, comparar con las listas de deseos y crear avisos para los que crucen el umbral o igualen el mínimo histórico.
3. Icono de notificaciones en la cabecera y correo opcional (ya está configurado el envío para recuperar contraseña).
4. Página «Mi lista de deseos» ordenable por descuento.

Notas

Solo funciona en PC y Xbox, que es donde hay fuente de precios; en PlayStation y Nintendo se mostraría sin alerta. Ya estaba planeada antes de proponerla. Hecha el 18/10/2026: modelo WishlistItem aparte (no un estado de la biblioteca) con precio de aviso opcional, botón «Lo quiero» en la ficha, página /biblioteca/lista-de-deseos/ ordenable por descuento o precio, campana de avisos en la cabecera (modelo Notification, reutilizable para los avisos sociales) y un correo por usuario con sus avisos, que se desactiva en Editar perfil. Avisa al bajar del precio puesto o al estar rebajado a su mínimo histórico, una sola vez hasta que baje más o suba y vuelva a bajar, y nunca de juegos que ya tiene en la biblioteca. Probada con precios reales.

77. Historial de precios y «¿compro ahora o espero?»

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Una gráfica con la evolución del precio en PC y un veredicto sencillo: «está en su mínimo histórico», «buen precio» o «suele bajar más, espera».

Por qué

Ahora solo se muestra el mínimo histórico. IsThereAnyDeal también da el historial completo de precios con la misma clave que ya se usa.

Pasos previstos

1. Pedir a IsThereAnyDeal el historial de precios del juego (uno o dos años).
2. Gráfica pequeña en la ficha, sin librerías pesadas (SVG generado en el servidor).
3. Calcular el veredicto comparando el precio de hoy con el mínimo y con cuántas veces ha bajado de ahí en el último año.
4. Mostrar cuándo fue la última rebaja fuerte.

Notas

Solo funciona en PC, que es de donde viene el historial. Hecha el 18/10/2026: se piden dos años a IsThereAnyDeal y se dibuja el último en un SVG generado en el servidor, cargado con HTMX y guardado un día. Veredicto: «mínimo histórico», «buen precio» (no ha estado más barato en un año o está a menos de un 10 % de lo más barato del año) o «espera» (ha bajado de ese precio; las rebajas solapadas de varias tiendas cuentan como una), y fecha de la última rebaja de al menos un 50 %. El historial de ITAD a veces cuenta tiendas que no salen en los precios de hoy, así que la gráfica termina siempre en el precio que enseña la ficha.

78. Seguir a otros jugadores y feed de actividad

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Poder seguir a otros jugadores y ver lo que hacen en la portada (pestañas Siguiendo y Descubrir) y en una pestaña Actividad de su perfil: juegos que empiezan, terminan o abandonan, notas, reseñas, «Lo quiero» e importaciones agrupadas.

Por qué

NextPlay se presenta como red social, pero ahora mismo no hay ninguna relación entre usuarios. Es la pieza que convierte la biblioteca personal en comunidad.

Pasos previstos

1. Fase 1 · Seguir: app nueva social con el modelo Follow (seguidor, seguido y fecha; único y sin poder seguirse a uno mismo), botón Seguir / Siguiendo con HTMX en el perfil, «Te sigue», contadores y páginas de seguidores y seguidos, y aviso en la campana «X ha empezado a seguirte» (una sola vez por persona, como los «me gusta»).
2. Seguir es unidireccional y sin solicitudes. Los perfiles privados no se pueden seguir y su actividad solo la ve su dueño, como ahora su biblioteca. La visibilidad se decide en un solo sitio para poder añadir después «solo seguidores».
3. Fase 2 · Actividad: modelo Activity (autor, tipo, juego, reseña, estado, nota, datos y fecha) que se apunta desde las vistas (cambio de estado, nota, reseña publicada y «Lo quiero») y no con señales: update_import guarda juegos en cada sincronización nocturna (y a veces les cambia el estado solo), y con señales no se distinguiría lo que hace el usuario de lo automático.
4. Lo que se hace con un mismo juego en poco rato se junta en una sola actividad («Terminó Hades · □□□□½ · y lo reseñó»), y cada importación de Steam, PlayStation o Xbox es una sola («Importó 129 juegos de Steam», con las portadas de los más jugados). Si se borra la reseña, se borra su actividad.
5. Pestaña Actividad en el perfil. Hacia atrás solo se rellenan las reseñas ya publicadas (con su fecha de publicación): las fechas de la biblioteca no valen, porque updated_at cambia con cada sincronización.
6. Fase 3 · Feed: la portada pasa a tener las pestañas Siguiendo y Descubrir de las maquetas, con una tarjeta para cada tipo de actividad (las reseñas, con library/_feed_review.html) y «Cargar más» con HTMX; si no sigues a nadie, sugerencias de gente a la que seguir.
7. El feed se calcula al leer (la actividad de la gente que sigues, con un índice por autor y fecha), sin Celery ni Redis: a este tamaño no hacen falta.
8. Usuarios demo que se siguen y tienen actividad en seed_test_profile, y tests de visibilidad, de agrupación, de importaciones y de avisos.

Notas

Las insignias de PSN, Switch, Epic... del perfil encajan aquí: sirven para añadir como amigo en la plataforma a quien sigues. Ya estaba planeada antes de proponerla. 25/10/2026: el usuario quiere empezar lo social por aquí; plan en tres fases, cada una en un PR que se puede probar por separado. Lo que hace que seguir a alguien sirva en la ficha y en los perfiles va en «La gente que sigues en la ficha y en los perfiles»; bloquear y silenciar, y el nivel «solo seguidores», en sus propias propuestas. 25/10/2026: fase 1 hecha en la rama seguir-a-jugadores (aún sin commit): app social con el modelo Follow (único y sin seguirse a uno mismo, también en la base de datos), botón Seguir / Siguiendo / Seguir también con HTMX en el perfil, «Te sigue», contadores en el perfil y en la portada, listas /u/<usuario>/seguidores/ y /u/<usuario>/siguiendo/ (los perfiles privados salen sin enlace ni biografía), aviso en la campana con la foto del nuevo seguidor (campo actor en Notification; una vez por persona), la regla de visibilidad en Profile.visible_to y Profile.visible_filter, seguidores de prueba en seed_test_profile y 24 tests. Faltan la fase 2 (actividad) y la 3 (feed). 25/10/2026: fase 1 con commit en su rama (sin fusionar, por decisión del usuario). Fase 2 hecha en la rama actividad-de-jugadores, encima de la anterior (aún sin commit): modelo Activity (tipos juego e importación) que apunta social/activity.py desde set_status, rate, remove, la primera publicación de una reseña y las importaciones de Steam, PlayStation y Xbox (update_library con record_activity; las correcciones del emparejamiento no la piden); lo hecho con un mismo juego en menos de una hora se junta; quitar la nota enseguida lo deshace; quitar un juego olvida su actividad salvo las reseñas; borrar una reseña deja lo demás (señal que borra las que se quedan

vacías). Pestaña Actividad en el perfil de 20 en 20 con «Cargar más» (HTMX y cursor por fecha e id). La migración pasa a la actividad las reseñas ya publicadas. «Lo quiero» no sale en la actividad porque la lista de deseos es privada: si se quiere, con un interruptor en Editar perfil. Actividad de prueba en seed_test_profile y 15 tests. Falta la fase 3 (feed en la portada). 25/10/2026: fase 2 con commit en su rama. Fase 3 hecha en la rama feed-en-la-portada, encima de las otras dos (aún sin commit): la portada pasa a tener el feed con las pestañas Siguiendo (lo tuyo y lo de la gente que sigues) y Descubrir (toda la comunidad), con las tarjetas de la actividad diciendo quién fue (y las imágenes y lo mejor y lo peor de las reseñas) y «Cargar más»; lo de perfiles privados no sale aunque los sigas; si no sigues a nadie, hasta 5 jugadores a los que seguir (primero los que tienen más juegos en común y después los más activos del mes, con subconsultas); sin sesión, populares y lo último de la comunidad. En la portada con sesión ya no salen «Populares» (están en el catálogo y en En tendencia) ni «Últimas reseñas», que ahora son parte del feed (se quita library/_feed_review.html). Índice por fecha para Descubrir y 4 tests. Las tres ramas están sin fusionar con main.

79. Rediseño de la ficha: cabecera con arte y carrusel de imágenes y vídeos

HECHA · Diseño · propuesta el 22/10/2026

Qué es

Aprovechar las imágenes y los vídeos de IGDB, Steam y la tienda de Microsoft en la ficha: una cabecera fija con el mejor arte apaisado y el logo del juego encima, y un carrusel que se pasa a mano con el tráiler, las capturas y los artes, que se abre en un visor a pantalla completa.

Por qué

Idea del usuario al empezar el rediseño de las fichas. Ahora la ficha solo usa la carátula (desenfocada de fondo) y hay material de sobra: en 100 juegos del catálogo repartidos por popularidad, la mediana es de 18 imágenes por juego entre los 25 más populares y de 8 en torno al puesto 2000, y 99 de cada 100 tienen vídeo. Pero no todo sirve: uno de cada seis artes de IGDB es un logo, un icono o una portada, y solo 15 de cada 100 juegos tienen un arte principal sin logo para la cabecera.

Pasos previstos

1. Filtrar los artes por artwork_type de IGDB: al carrusel van los artes, los conceptuales y los key art; fuera logos, iconos, portadas e infografías. Arregla de paso el selector de cabecera del perfil (IGDBClient.artworks), que ahora ofrece logos e iconos en los 5 juegos más populares (el icono de 256 × 256 de GTA V, el logo de The Witcher 3...).
2. Cabecera fija con la mejor imagen: library_hero.jpg de Steam con su logo.png encima, como en la biblioteca de Steam (salen del CDN con el steam_app_id, sin llamar a ninguna API); si no, el SuperHeroArt de la tienda de Microsoft o el key art sin logo de IGDB; si no, la carátula desenfocada de ahora. El título sigue siendo texto para lectores de pantalla y buscadores.
3. Carrusel manual «Imágenes y vídeos»: tira con scroll-snap (en el móvil se pasa con el dedo), flechas en escritorio, la siguiente imagen asomando, contador y filtros Todo / Vídeos / Capturas / Arte. Sin avance automático: casi nadie mira más allá de la primera, distrae de la biblioteca y los precios y obliga a poner un botón de pausa por accesibilidad.
4. El tráiler, primero y como miniatura: el vídeo de YouTube (youtube-nocookie.com) solo se carga al pulsar.
5. Visor a pantalla completa con <dialog>: flechas, teclado, deslizar y Esc; la imagen a 1080p solo se descarga ahí. Miniaturas con srcset y carga diferida.
6. Cargar la sección con HTMX después de pintar la ficha, como el historial de precios, con caché de un día (game_arts ya lo hace para el perfil) y por la noche para los juegos más populares.
7. Botón «Poner de cabecera en mi perfil» en el visor, que guarda ese juego y esa imagen en banner_game y banner_art.
8. Más adelante: los tráileres en castellano de Steam (pidiendo filters=basic,screenshots,movies en la misma llamada que ya hace localize_games, sin peticiones de más) y las capturas de la comunidad cuando existan las publicaciones.

Notas

Forma parte de «Rediseño de toda la web con aspecto de red social» y recoge el tráiler, la galería y la cabecera de «Ficha del juego más completa». Comprobado el 22/10/2026: 1297 de los 2175 juegos del catálogo tienen ID de Steam y 1442 el de Steam o el de Microsoft; library_hero.jpg y logo.png estaban en los 6 juegos de Steam probados. La tienda de Microsoft da, en la misma respuesta que ya pide fetch_descriptions, 14 capturas a 1920 × 1080, SuperHeroArt y TitledHeroArt (GTA V). Los vídeos de Steam vienen en HLS o DASH (hace falta un reproductor como hls.js) y en algunos juegos hay versiones en castellano: en GTA V salen con «(ES)» o «PEGI SP» en el nombre. 22/10/2026: maqueta con God of War (escritorio, móvil y visor) y Zelda: Breath of the Wild como juego sin Steam, en el lienzo «Maqueta de la ficha de juego» (https://claude.ai/artifact/99d2PMDrVMrnsuEyvMqRja); el carrusel se puede ver como tira o como vista grande con miniaturas. El usuario eligió la tira. 23/10/2026: montada en la rama ficha-cabecera-y-carrusel y fusionada con main: filtro de tipos de arte en IGDBClient.artworks (arregla el selector de cabecera del perfil), games/media.py con una sola consulta a IGDB guardada en Game.media 30 días, comprobación de la imagen y el logo de la biblioteca de Steam y recorte del margen transparente de los logos, cabecera con el logo encima, tira con filtros y flechas, visor con teclado, deslizar y tráileres de YouTube que se cargan al pulsar, y el botón «Poner de cabecera en mi perfil» (solo con juegos de la biblioteca). Solo se usan logos con transparencia (el de Zelda: Breath of the Wild en IGDB trae fondo blanco y saldría como un recuadro) y el primer vídeo es el tráiler de lanzamiento si lo hay. 10 tests nuevos; probada en el navegador en escritorio y en móvil con God of War y con Zelda. Falta: tráileres en castellano de Steam y capturas de la comunidad.

---

80. Fichas en castellano: doblaje, subtítulos y sinopsis en español

HECHA · Funcionalidad · propuesta el 21/10/2026

Qué es

Enseñar en cada juego si tiene voces, textos o subtítulos en castellano (o solo en español de Latinoamérica), poner la sinopsis en español, encontrar los juegos por su nombre en España y filtrar el catálogo y el pendiente por «doblado al castellano».

Por qué

Backloggd está en inglés y elegir el idioma de las reseñas sigue pendiente en su hoja de ruta. NextPlay ya enseña los precios de la tienda española, pero la sinopsis de la ficha sale de IGDB en inglés y la pregunta más típica del jugador de aquí, «¿viene doblado?», no la contesta ningún tracker. Los datos salen de fuentes que ya se usan: IGDB, Steam y la tienda de Microsoft.

Pasos previstos

1. Pedir a IGDB language_supports (idioma y tipo: voces, subtítulos o interfaz) en GAME_FIELDS; distingue el español de España del de México.
2. Completar con Steam en los juegos con steam_app_id: appdetails con l=spanish trae supported_languages, que marca con un asterisco los idiomas con voces.
3. Sinopsis en español: la descripción corta de Steam en español o la de la tienda de Microsoft, que fetch_products ya pide en es-ES aunque ahora solo lee el precio. Si no hay ninguna, la de IGDB con la etiqueta «en inglés».
4. Nombres de España (los de Nintendo cambian mucho) desde los nombres alternativos de IGDB, para que el buscador también los encuentre.
5. Insignias en la ficha («Voces en castellano», «Textos en español», «Solo en inglés») y filtro en el catálogo, la biblioteca y la lista de deseos.
6. Portadas de aquí: game_localizations de IGDB trae nombre y portada por región (también Europa), como las portadas personalizadas de Backloggd; usar la europea por defecto y dejar elegir región en el perfil.

Notas

Se puede hacer en la misma reimportación que «Ficha del juego más completa». La API appdetails de Steam admite pocas peticiones seguidas (unas 200 cada 5 minutos): pedirla en la importación o en la tarea nocturna, nunca al abrir la ficha. Hecha el 21/10/2026 (aún sin commit): games/spanish.py, comando localize_games (también en nightly, 200 juegos por noche, con reintentos si Steam corta la conexión), etiquetas y tabla de idiomas en la ficha, sinopsis de Steam o de Xbox con la de IGDB plegada, nombre en España, portada europea cuando IGDB la tiene, filtro ?idioma=

en el catálogo, la biblioteca y la lista de deseos, y el buscador encuentra los nombres alternativos (siglas como «tw3» o «gta 5»). En el catálogo local: 2175 juegos revisados en IGDB, 496 con voces en castellano y 1383 con textos en castellano; sinopsis en español de los 154 más populares (137 de Steam y 17 de la tienda de Xbox). El resto lo irá completando nightly (o localize_games --limit N de una vez). No hecho: elegir en el perfil la región de las portadas (aporta poco: solo unos 1 de cada 9 juegos tienen portada europea en IGDB).

---

81. Un solo sitio para crear los avisos, con preferencias por tipo

HECHA · Infraestructura · propuesta el 01/11/2026

Qué es

Que todos los avisos de la campana pasen por una sola función, que decida si se crean: no repetir, respetar los bloqueos y silencios cuando existan y lo que cada usuario haya elegido recibir, en la campana o por correo.

Por qué

Hoy hay 10 llamadas a Notification.objects.create repartidas por lists/views.py, social/views.py, social/models.py, library/views.py, library/wishlist.py y library/reminders.py, cada una con su propia regla para no repetirse. Ocho propuestas abiertas añaden avisos nuevos (retos, lanzamientos, juegos gratis, la gente que sigues, amigos, solicitudes, listas colaborativas y Game Pass) y «Bloquear, silenciar y quitar seguidores» tendría que tocarlas todas. La única preferencia que existe es el correo de los precios (price_alert_emails).

Pasos previstos

1. accounts/notices.py con notify(user, kind, text, url, game=None, actor=None): toda la lógica común en un sitio.
2. Las reglas de repetición, por tipo y declaradas en una tabla: una vez por persona (los «me gusta»), hasta que lo lea (los comentarios) o una por cosa (las reseñas que se han quedado atrás).
3. Pasar las 10 llamadas a notify sin cambiar el comportamiento, con los tests actuales como red de seguridad.
4. Profile.notification_prefs (JSON con campana y correo por tipo) y una sección «Avisos» en Editar perfil. Lo que no se ha tocado sigue encendido, y price_alert_emails se migra.
5. Punto de entrada para bloqueos y silencios: cuando exista el bloqueo, solo se toca aquí.

Notas

Sale de la revisión del 01/11/2026. Va antes de la fase 2 de «Retos: crea los tuyos y únete a los de otros» y de «Bloquear, silenciar y quitar seguidores». 04/11/2026: hecha en la rama avisos-en-un-solo-sitio. accounts/notices.py con notify(user, kind, text, url, game=, actor=, since=): nadie recibe avisos de lo que hace él mismo, cada tipo tiene en la tabla REPEAT su regla para no repetirse (siempre: las bajadas de precio y las invitaciones; una vez por persona y cosa: los «me gusta» y los seguidores; mientras no se lea el anterior: los comentarios y las denuncias; una por reseña hasta que se vuelve a guardar: las reseñas que se han quedado atrás) y solo llega lo que cada uno deja encendido. Las 10 llamadas pasan por ahí y se portan igual, salvo un detalle: los avisos de «me gusta» y comentarios de las reseñas guardan ya quién los ha provocado, como los demás, y dejan de distinguirse por el texto (antes, si alguien cambiaba su nombre visible, su «me gusta» volvía a avisar). Profile.notification_prefs (un JSON por grupos) sustituye a price_alert_emails, con una migración que deja el correo apagado a quien lo tenía apagado. Sección «Avisos» en Editar perfil, separada de «Privacidad», con seis interruptores: bajadas de precio (con el correo aparte), nuevos seguidores, «me gusta», comentarios, invitaciones a editar listas y reseñas que se han quedado atrás. Tocarlos no pide la vista previa, y un envío del formulario sin la sección no apaga nada. Las denuncias no se pueden apagar y, por correo, de momento solo pueden llegar las bajadas de precio. Si alguien apaga las bajadas de precio en la campana y en el correo, la bajada no cuenta como avisada: al volver a encenderlas, se avisa del precio de entonces. La lista de deseos dice por dónde te avisamos (o cómo encender los avisos si están apagados), y la página de Avisos y el correo de precios enlazan a la sección. Un tipo de aviso nuevo necesita su regla y su grupo, y un test lo comprueba. 14 tests nuevos (515 en total). 04/11/2026: ya la usan los avisos de la fase 2 de los retos, que han añadido su grupo «Retos».

REVISIÓN DEL 04/11/2026

Cómo está hoy

Hecha en la rama avisos-en-un-solo-sitio: los avisos solo se crean en accounts.notices.notify y la sección «Avisos» de Editar perfil decide cuáles llegan. Lo siguiente que se apoya en ella: la fase 2 de los retos, el aviso a quien abre una tier list a la comunidad, «La gente que sigues» y los bloqueos. Para cada tipo nuevo basta con su Notification.Kind, su regla en REPEAT y su grupo en GROUPS (o uno nuevo). Por correo solo van las bajadas de precio: para los demás tipos haría falta un resumen agrupado, para no mandar un correo por cada «me gusta».

Para profundizar

- Agrupar en la campana: «A Ana y 3 más les ha gustado tu reseña», en vez de cuatro avisos seguidos. Con actor ya en el modelo, solo falta juntarlos al enseñarlos.
- Resumen semanal por correo, opcional: rebajas de tu lista, lanzamientos y lo que ha hecho tu gente, con baja en un clic (cabecera List-Unsubscribe). Hace volver sin ser pesado.
- Avisos del navegador (Web Push) cuando la web sea instalable: la misma función decide si también salen ahí.

---

82. Partir las vistas más grandes por pantalla

HECHA · Infraestructura · propuesta el 03/11/2026

Qué es

Cambiar accounts/views.py (44 funciones), lists/views.py (45) y library/views.py (29) por una carpeta views/ con un archivo por pantalla o por grupo de botones, para que el nombre del archivo ya diga a qué pantalla afecta cada función.

Por qué

Ahora cada función dice sus pantallas, pero en los views.py grandes van mezcladas las de pantallas distintas: accounts/views.py tiene Registro, Perfil, Editar perfil, la vinculación de Steam, PlayStation y Xbox, Juegos importados y Avisos. Es la parte de «reestructurar el proyecto» que mueve código, que el 03/11/2026 no se hizo.

Pasos previstos

1. accounts/views/ con perfil.py, editar_perfil.py, vincular.py (Steam, PlayStation, Xbox y otras plataformas), importados.py y avisos.py, y un __init__.py que lo reexporte todo para que urls.py y quien importa de accounts.views no cambien.
2. Lo mismo con lists/views.py (página de la lista, editor, colaboradores y tier lists) y library/views.py (los botones de la ficha, las reseñas y la lista de deseos).
3. Cambiar las rutas de los @patch de los tests (accounts.views.steam pasa a accounts.views.vincular.steam) y lanzar los tests sin claves y con claves falsas.
4. Regenerar el PDF de funciones: el mapa de pantallas ya lo agrupa por archivo, así que se verá cada pantalla con sus archivos.

Notas

Hecha el 03/11/2026, en la misma rama que la documentación de las pantallas: accounts/views/ (registro, perfil, editar_perfil, cuentas_vinculadas, juegos_importados, avisos, ficha_del_juego y comun), lists/views/ (pagina_de_la_lista, editar_lista, colaboradores, nueva_lista, listas_del_juego, tarjetas y comun) y library/views/ (ficha_del_juego, escribir_resena, pagina_de_la_resena, lista_de_deseos y comentarios_y_denuncias). Los nombres de los archivos van en español porque son los de las pantallas de docs/pantallas.json. Ninguna función ha cambiado: se movieron con un script que comprobó que el código de cada una seguía igual. Los __init__.py reexportan las vistas, así que urls.py y las otras apps no cambian; solo dos tests importan ahora del archivo nuevo (accounts.views.perfil y lists.views.tarjetas).

REVISIÓN DEL 03/11/2026

Cómo está hoy

Hecha. social/views.py (28 funciones: seguir, el feed, la actividad y las publicaciones) sigue de una pieza; se puede partir igual si crece.

Para profundizar

- Ir más allá de las vistas: los modelos y los formularios se usan en muchas pantallas, así que ahí partir por pantalla no ayuda; mejor dejarlos por app.
- Las plantillas ya van por app y por pantalla; los trozos que salen en varias (la tarjeta de juego, el avatar) podrían ir a una carpeta común para que se vea que no son de una sola.

---

83. Reseñas con horas verificadas y spoilers según tu progreso

HECHA · Funcionalidad · propuesta el 21/10/2026

Qué es

Que cada reseña diga con cuántas horas se escribió y de qué plataforma salen, poder ver solo las de quien lo ha jugado de verdad o lo ha terminado, y plegar las que van más adelante que tú en el juego.

Por qué

En Backloggd cualquiera puede valorar un juego sin haberlo jugado. En NextPlay las horas que salen en las reseñas ya vienen de las importaciones, pero son las de hoy y no las de cuando se escribió, y no se puede filtrar por ellas. Es una confianza que Backloggd no puede dar porque no conoce las horas reales.

Pasos previstos

1. Guardar en LibraryEntry las horas que tenía al publicar o editar la reseña y enseñar «escrita a las 12 h · ahora lleva 80 h».
2. Logo de la plataforma de la que vienen las horas (Steam, PlayStation o Xbox) como sello de horas verificadas.
3. Filtros en la ficha: «Todas», «Más de 10 h» y «Lo han terminado», y una nota de la comunidad solo con quienes tienen horas.
4. Spoilers según tu progreso: si llevas 5 h, las reseñas escritas con muchas más horas que tú salen plegadas («escrita a las 60 h: puede contar lo que te queda»).
5. Aviso al autor cuando ha jugado mucho más desde su reseña: «llevas 40 h más, ¿la actualizas?».

Notas

El plegado por progreso sirve también para «Publicaciones con imágenes asociadas a un juego». Backloggd tiene pendiente en su hoja de ruta marcar spoilers dentro del texto; aquí sería automático. 23/10/2026: hecho en la rama resenas-completas: cada reseña guarda al publicarla (y al editarla) las horas importadas y de qué plataformas salen («Escrita a las 24,2 h · Steam») y cómo iba el autor con el juego («Lo terminó», «Aún jugándolo»); filtros «Lo han terminado» y «Con más de 10 h» en la ficha; las reseñas escritas con 10 h o más de ventaja sobre quien las lee (si no lo ha terminado) salen plegadas. Faltan la nota de la comunidad solo con quien tiene horas, los logos de plataforma en vez del nombre y el aviso «llevas 40 h más, ¿la actualizas?». 24/10/2026: hecho lo que faltaba en la rama resenas-imagenes-y-denuncias (aún sin commit): la valoración de la comunidad enseña también la media solo con quienes tienen horas importadas, el sello de horas lleva los logos de Steam, PlayStation y Xbox, y quien lleva el doble de horas que al escribir su última reseña del juego (y al menos 10 más) ve «Llevas 40 h más: ¿la actualizas?» en la ficha y en la reseña y recibe un aviso en la campana tras la importación (uno por reseña hasta que la vuelva a guardar).

84. Importar logros y trofeos

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Traer el porcentaje de logros de Steam, trofeos de PlayStation y logros de Xbox y mostrarlo en la biblioteca y el perfil (platinos, 100 %).

Por qué

Las tres integraciones ya están hechas y sus APIs dan los logros con poco trabajo extra; es de lo que más presumen los jugadores.

Pasos previstos

1. Steam: GetPlayerAchievements por juego (más lento: una llamada por juego, hacerlo en segundo plano).
2. PlayStation: la lista de títulos de trofeos ya trae progreso y platino.
3. Xbox: endpoint de logros por título.
4. Campo de progreso en LibraryEntry e insignia de platino / 100 % en las tarjetas.

Notas

Conviene antes tener tareas en segundo plano (Celery o django-q) porque la sincronización de Steam sería larga. Ya estaba planeada antes de proponerla. 28/10/2026: hecha a petición del usuario en la rama importar-logros, encima de listas-arrastrar. Sin tareas en segundo plano: cada importación de Steam lee los logros de los 40 juegos jugados hace menos de los que haya algo nuevo, con 8 peticiones a la vez (unos segundos más), y la sincronización de cada noche lee el resto. Se guardan por título importado (ImportedTitle: conseguidos, totales, platino y cuándo se leyeron), no en LibraryEntry, porque cada plataforma lleva los suyos. Xbox los trae con su historial (decoración achievement) y PlayStation con su lista de trofeos, emparejada por nombre; como no se guarda el NPSSO ni el token de Microsoft, se actualizan al sincronizar a mano. La ficha enseña «Tus logros» por plataforma con su barra; las portadas del perfil, «Platino» o «100 %», y la cabecera, cuántos platinos y juegos al 100 % tiene. Probado con respuestas simuladas de las tres APIs (8 tests nuevos) y con datos de prueba en seed_test_profile. Steam, también con la cuenta de verdad de tester_steam: 133 juegos leídos en 11,6 s (127 con logros, 6 sin, 1 que no se pudo leer; 5 al 100 %). Faltan PlayStation y Xbox con una sincronización de verdad. Siguiente paso posible: «¿Cuánta gente lo termina?», que usa los logros de Steam. 29/10/2026: primera sincronización de verdad de PlayStation (tester_steam): trofeos de 129 de los 176 juegos y 4 platinos en el perfil; los 47 que faltan, en la propuesta «Trofeos de PlayStation emparejados por el ID del juego». Xbox sigue sin probarse con una cuenta de verdad.

85. Importar la lista de deseos de Steam

HECHA · Funcionalidad · propuesta el 21/10/2026

Qué es

Un botón (y una pregunta al vincular Steam) que trae a «Lo quiero» todos los juegos de la lista de deseos de Steam, para que desde el primer minuto el usuario sepa cuáles están en su mínimo histórico o más baratos en otra tienda y reciba los avisos.

Por qué

Es la forma más rápida de que alguien nuevo vea lo que vale NextPlay aunque todavía no haya comunidad: Steam solo avisa de sus propias rebajas, y NextPlay compara todas las tiendas, sabe el mínimo histórico y dice si conviene esperar. Probado el 21/10/2026: la API de Steam devuelve la lista sin clave (93 juegos de la cuenta vinculada en local, con appid, fecha y prioridad).

Pasos previstos

1. Leer IWishlistService/GetWishlist de cada cuenta de Steam vinculada (no necesita la STEAM_API_KEY).
2. Emparejar los appid con _match_games de accounts/steam.py, que ya busca en el catálogo y, si falta, en IGDB; crear los WishlistItem que no existan, sin los juegos que ya están en la biblioteca.

3. Guardar la prioridad de Steam y la fecha en que se añadió, para poder ordenar la lista de deseos igual que en Steam.
4. Pedir en ese momento los precios de los juegos importados (update_prices solo con ellos), sin esperar a la noche, y enseñar un resumen: «de tus 93 deseados, 12 están hoy en su mínimo histórico y 5 son más baratos fuera de Steam».
5. Opción para volver a importarla cada noche junto a la biblioteca, recordando los que el usuario haya quitado para no volver a añadirlos.

Notas

Es una API sin documentar, como otras que ya se usan, y solo funciona si el perfil deja ver sus detalles de juego (igual que la biblioteca). Conviene tenerla antes de las rebajas de invierno de Steam (del 17 de diciembre al 4 de enero). 21/10/2026: aparcada; de momento el usuario prefiere centrarse en el perfil, las listas, los retos y las reseñas. 24/10/2026: hecha a petición del usuario (aún sin commit). Cada vez que se importa Steam (al vincular, con «Sincronizar» y en la sincronización de cada noche) se lee IWishlistService/GetWishlist de todas sus cuentas y los juegos se buscan como los de la biblioteca, respetando lo emparejado a mano. Se crean en «Lo quiero» los que no tiene ya, con el puesto y la fecha de Steam, que dan el orden «Como en tu lista de Steam» y una etiqueta de Steam en la lista de deseos. No se traen los de su biblioteca ni los que quitó a mano (Profile.steam_wishlist_skip, con «Volver a traerlos» en el panel de Steam); los que trajo Steam se quitan cuando dejan de estar allí, pero solo si se han podido leer todas sus listas (una privada o vacía no borra nada), y lo añadido a mano en NextPlay nunca se toca. Al sincronizar se piden ya a IsThereAnyDeal los precios de los nuevos y el aviso dice cuántos están hoy en su mínimo histórico y cuántos son más baratos fuera de Steam. Se puede desactivar en el panel de Steam. 7 tests nuevos. De los que no están en el catálogo solo se da el número: la API no trae sus nombres.

---

86. Vincular otras plataformas sin importar (PSN, Switch, Epic, GOG, Battle.net)

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Que el usuario pueda poner en su perfil su PSN ID, código de amigo de Switch y usuarios de Epic, GOG y Battle.net, mostrados como insignias con su icono.

Por qué

Estas plataformas no tienen API pública para leer la biblioteca (o solo no oficial), pero vincularlas sirve para que otros jugadores te encuentren.

Pasos previstos

1. Modelo PlatformAccount (usuario, plataforma, nombre o código) con una cuenta por plataforma.
2. Validar el formato de cada una (p. ej. SW-XXXX-XXXX-XXXX para Switch).
3. Sección en Editar perfil para añadirlas y quitarlas.
4. Insignias en el perfil con el logo de cada plataforma (ya hay SVG de PlayStation, Nintendo, Epic y GOG).

Notas

No se verifica que la cuenta sea del usuario: es lo que él escribe, como en la mayoría de webs. Hecha el 17/10/2026 con Xbox (gamertag), Nintendo Switch, Epic Games, GOG y Battle.net; PlayStation quedó fuera porque ya tiene su propia importación.

---

87. Importar juegos y horas de PlayStation (experimental)

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Importar los juegos de PS4/PS5 con sus horas y trofeos usando la API no oficial de la app de PlayStation.

Por qué

Sony no ofrece API pública, pero la API que usa su app devuelve los juegos jugados y el tiempo de juego, y la usan webs conocidas de trofeos.

Pasos previstos

1. El usuario copia su código NPSSO desde la web de PlayStation (con su sesión iniciada) y lo pega en NextPlay.
2. Cambiar ese código por un token de acceso y leer la lista de juegos jugados con su duración.
3. Emparejar con el catálogo por nombre (IGDB no guarda IDs de PlayStation Store de forma fiable) y crear las entradas.
4. Marcarlo como experimental en la interfaz.

Notas

Riesgo: es no oficial y Sony puede romperlo en cualquier momento; además el NPSSO da acceso a la cuenta, así que habría que guardarlo cifrado o no guardarlo. Hacer solo si compensa. Hecha el 17/10/2026: el NPSSO no se guarda (se pega en cada sincronización), una cuenta de PSN por usuario, las horas se suman con las de Steam y el lanzador «Call of Duty®» de PS5 se ignora. Probada con una cuenta real: 129 juegos importados.

---

88. Varias cuentas de Steam por usuario

HECHA · Funcionalidad · propuesta el 17/10/2026

Qué es

Que cada usuario pueda vincular más de una cuenta de Steam a su perfil y que la importación junte los juegos de todas.

Por qué

La idea es que todo el mundo pueda conectar una o varias cuentas. Ahora el SteamID se guarda en el propio perfil, así que solo cabe una.

Pasos previstos

1. Nuevo modelo SteamAccount (usuario, SteamID único, nombre, avatar, URL del perfil y fecha de la última sincronización).
2. Migración que pase la cuenta ya vinculada del perfil a la nueva tabla y quite los campos viejos.
3. Importar de cada cuenta o de todas a la vez; si un juego está en varias cuentas se suman sus horas en vez de duplicarlo.
4. En Editar perfil, una lista de cuentas con «Añadir otra cuenta» y, en cada una, sincronizar y desvincular.
5. Una insignia por cuenta en el perfil y tests nuevos.

Notas

No hace falta una API key por usuario: la clave de Steam es de NextPlay y sirve para leer cualquier cuenta con los «Detalles del juego» públicos. Hecha el 17/10/2026: hasta 5 cuentas por usuario; la sincronización es conjunta (no por cuenta) para que las horas sumadas cuadren, y las cuentas privadas se saltan avisando.